

João Pedro Baza Garcia Rodrigues

**DCOU — Dimensionamento Computacional de
Operações Unitárias: uma ferramenta didática
para Engenharia Química**

Campo Grande – MS

2026

João Pedro Baza Garcia Rodrigues

**DCOU — Dimensionamento Computacional de Operações
Unitárias: uma ferramenta didática para Engenharia
Química**

Trabalho de Conclusão de Curso apresentado
no Instituto de Química como requisito básico
para a conclusão do Curso de Engenharia
Química.

Universidade Federal de Mato Grosso do Sul – UFMS
Instituto de Química – INQUI
Engenharia Química

Orientador: Prof. Dr. Celso Murilo dos Santos

Campo Grande – MS
2026

João Pedro Baza Garcia Rodrigues

DCOU — Dimensionamento Computacional de Operações Unitárias: uma ferramenta didática para Engenharia Química

Trabalho de Conclusão de Curso apresentado
no Instituto de Química como requisito básico
para a conclusão do Curso de Engenharia
Química.

Trabalho aprovado. Campo Grande – MS, 13 de julho de 2026:

Prof. Dr. Celso Murilo dos Santos
Docente Orientador

Danieli Cristine Anversa
Docente Doutora

Vinícius Basso de Godoy
Mestre

Campo Grande – MS
2026

Agradecimentos

Agradeço a Deus, por me conceder forças nos momentos difíceis e iluminar meu caminho ao longo desta jornada acadêmica.

Aos meus pais, Cassiano e Jacquecislene, por me darem não apenas a vida, mas também o dom do estudo e do senso crítico.

À minha namorada, Carolyne, que, com sua paciência, carinho e incentivo, foi essencial para que eu visse propósito no que eu fazia, como a luz que guia a direção da folha.

*“Não deseje que seja mais fácil, deseje ser melhor.
Não deseje menos problemas, deseje mais habilidades.
Não deseje menos desafios, deseje mais sabedoria.”*
(Jim Rohn)

Resumo

Este trabalho apresenta o desenvolvimento do DCOU (*Dimensionamento Computacional de Operações Unitárias*), um *software web* de código aberto concebido como ferramenta didática para o ensino de Engenharia Química. O sistema automatiza cálculos de operações unitárias a partir de equações consolidadas, contemplando módulos de tubulações, escoamento, bombas, propriedades termodinâmicas, reatores ideais e balanços de massa. Além dos resultados numéricos, a aplicação integra recursos voltados ao aprendizado conceitual, como glossário técnico com equações renderizadas, explicações teóricas embutidas em cada calculadora, cenários de exemplo fornecidos pelo *backend* e visualizações interativas dos fenômenos físicos. A arquitetura cliente-servidor expõe os cálculos e os modelos de visualização por meio de uma *API* (*Application Programming Interface*) aberta, enquanto o *frontend* moderno em *React* organiza esses recursos em uma experiência navegável e responsiva, com adaptação consistente a diferentes tamanhos de tela, incluindo dispositivos móveis, desktops e monitores *ultrawide*. O código permanece publicamente disponível, promovendo flexibilidade, colaboração e aprimoramento contínuo por parte da comunidade acadêmica.

Palavras-chave: Ferramenta didática. Operações unitárias. *Software educacional*. Código aberto. Engenharia Química.

Lista de ilustrações

Figura 1 – Interface gráfica do módulo de cálculos de tubulação (<i>Piping</i>)	28
Figura 2 – Código da função <i>get_calculated_diameter()</i>	33
Figura 3 – Código da função <i>get_real_diameter()</i>	34
Figura 4 – Código da função <i>determine_limiting_reagent()</i> – Parte 1/2	36
Figura 5 – Código da função <i>determine_limiting_reagent()</i> – Parte 2/2	37
Figura 6 – Código da função <i>_calculate_concentration_and_rate()</i> – Trecho da velocidade da reação	38
Figura 7 – Código da função <i>_calculate_concentration_and_rate()</i> – Trecho da concentração dos componentes	39
Figura 8 – Código da função <i>_calculate_dilution_factor()</i>	40
Figura 9 – Código da função <i>_conversion_and_kinetics_in_cstr()</i> – Parte 1/2	42
Figura 10 – Código da função <i>_conversion_and_kinetics_in_cstr()</i> – Parte 2/2	43
Figura 11 – Código da função <i>_initialize_reactor()</i> – Parte 1/2	44
Figura 12 – Código da função <i>_initialize_reactor()</i> – Parte 2/2	45
Figura 13 – Código da função <i>_volume_and_kinetics_in_cstr()</i> – Parte 1/2	46
Figura 14 – Código da função <i>_volume_and_kinetics_in_cstr()</i> – Parte 2/2	47
Figura 15 – Código da função <i>_residence_time_and_kinetics_in_cstr()</i> – Parte 1/2	48
Figura 16 – Código da função <i>_residence_time_and_kinetics_in_cstr()</i> – Parte 2/2	49
Figura 17 – Código da função objetivo para o <i>PFR</i>	50
Figura 18 – Especificações internas para tubulações	52
Figura 19 – Cálculo da perda de carga por Darcy–Weisbach com acessórios	53
Figura 20 – Resultados do balanço de massa com reciclo e reação	55
Figura 21 – Cálculo do Reynolds – Parte 1/3	57
Figura 22 – Cálculo do Reynolds – Parte 2/3	58
Figura 23 – Cálculo do Reynolds – Parte 3/3	58
Figura 24 – Interface de cálculo do fator de atrito	59
Figura 25 – Cálculo do fator de atrito – Parte 1/2	60
Figura 26 – Cálculo do fator de atrito – Parte 2/2	61
Figura 27 – Cálculo da perda de carga – Parte 1/3	62
Figura 28 – Cálculo da perda de carga – Parte 2/3	63
Figura 29 – Cálculo da perda de carga – Parte 3/3	64
Figura 30 – Cálculo do <i>head</i> – Parte 1/2	65
Figura 31 – Cálculo do <i>head</i> – Parte 2/2	66
Figura 32 – Cálculo do <i>NPSH</i> – Parte 1/2	67
Figura 33 – Cálculo do <i>NPSH</i> – Parte 2/2	68
Figura 34 – Tipos de dutos de escoamento	69

Figura 35 – Exemplo da identificação e critérios de diâmetro hidráulico	70
Figura 36 – Obtenção de propriedades pela biblioteca <i>CoolProp</i>	71
Figura 37 – Obtenção de propriedades críticas pela biblioteca <i>CoolProp</i>	72
Figura 38 – Interface de cálculo para as propriedades de uma mistura	73
Figura 39 – Obtenção de propriedades de misturas pela biblioteca <i>CoolProp</i> – Parte 1/2	74
Figura 40 – Obtenção de propriedades de misturas pela biblioteca <i>CoolProp</i> – Parte 2/2	75
Figura 41 – Inicialização da biblioteca de componentes via <i>Python</i>	77
Figura 42 – Inicialização da biblioteca de componentes via <i>frontend</i>	78
Figura 43 – Listagem parcial de componentes disponíveis na biblioteca termodinâmica	78
Figura 44 – Propriedades termofísicas da água calculadas a 300K e 101,3kPa.	79
Figura 45 – Mapeamento de chaves de propriedades e suas unidades padronizadas.	80
Figura 46 – Propriedades críticas e do ponto triplo calculadas para a água.	81
Figura 47 – Propriedades termodinâmicas disponíveis para cálculo em misturas.	82
Figura 48 – Propriedades termodinâmicas disponíveis para cálculo em misturas.	83
Figura 49 – Código de validação do Balanço de Massa (Misturador)	89
Figura 50 – Resultados expandidos da validação de Balanço de Massa (Misturador, Reator e <i>Splitter</i>)	92
Figura 51 – Lista parcial de acessórios disponíveis (JSON)	93
Figura 52 – Especificações do acessório Cotovelo 90° Raio Longo (JSON)	94
Figura 53 – Lista parcial de materiais de tubulação (JSON)	95
Figura 54 – Especificações do material Aço Comercial (JSON)	96
Figura 55 – Lista de classes de pressão (<i>Schedules</i>) disponíveis (JSON)	96
Figura 56 – Lista parcial de diâmetros para SCH40 (JSON)	97
Figura 57 – Especificações do tubo SCH40 DN50 (JSON)	98
Figura 58 – Glossário técnico com busca e equações renderizadas	102
Figura 59 – Exemplo de explicação teórica embutida em uma calculadora	102
Figura 60 – Diagrama de <i>Moody</i> com o ponto operacional do problema destacado	106
Figura 61 – Diagrama de <i>Levenspiel</i> comparando reatores <i>CSTR</i> e <i>PFR</i>	106
Figura 62 – Trilhas de aprendizagem apresentadas na página inicial	107
Figura 63 – Tela inicial com trilhas de aprendizagem e cartões de acesso rápido	112
Figura 64 – Aba Composições do módulo de tubulações com exemplo carregado	113
Figura 65 – Aba Schedules e Diâmetros com diâmetro comercial do exemplo	114
Figura 66 – Aba Conexões com acessório carregado pelo exemplo	115
Figura 67 – Aba Diâmetro Calculado com exemplo executado	116
Figura 68 – Aba Diâmetro Real com seleção de diâmetro comercial	117
Figura 69 – Aba Número de Reynolds com resultado e régua de regime	118
Figura 70 – Aba Fator de Atrito com diagrama de <i>Moody</i>	119

Figura 71 – Aba Diâmetro Hidráulico com pré-visualização geométrica	120
Figura 72 – Aba Perda de Carga com exemplo e curva do sistema	121
Figura 73 – Aba NPSH Disponível com indicador de margem	122
Figura 74 – Aba Altura Manométrica com decomposição dos termos de energia .	123
Figura 75 – Aba Propriedades Críticas com fluido do exemplo	124
Figura 76 – Aba Fluido Puro com propriedades e superfície T-P	125
Figura 77 – Aba Misturas com exemplo e diagrama ternário	126
Figura 78 – Aba Equilíbrio Binário com diagrama T-x-y	127
Figura 79 – Bloco McCabe-Thiele gerado a partir do exemplo binário	128
Figura 80 – Aba Envelope de Fase com curva gerada pelo exemplo	129
Figura 81 – Aba CSTR com exemplo calculado	130
Figura 82 – Aba PFR com exemplo calculado	131
Figura 83 – Aba Levenspiel comparando CSTR e PFR	132
Figura 84 – Aba Arrhenius com curva semilogarítmica do exemplo	133
Figura 85 – Aba Componentes do balanço de massa com exemplo carregado . . .	134
Figura 86 – Aba Correntes com alimentação, produto e reciclo do exemplo	135
Figura 87 – Aba Reações com estequiometria do exemplo	136
Figura 88 – Aba Splits / Reciclo com configuração do exemplo	137
Figura 89 – Aba Resultados do balanço de massa após cálculo do exemplo	138
Figura 90 – Glossário com busca por Reynolds	139

Lista de tabelas

Tabela 1	– Bibliotecas e <i>frameworks</i> utilizados no <i>backend</i>	25
Tabela 2	– Bibliotecas e <i>frameworks</i> utilizados no <i>frontend</i>	29
Tabela 3	– Validação Balanço de Massa (Misturador)	90
Tabela 4	– Validação <i>CSTR</i> ($X = 0,85$)	99
Tabela 5	– Validação <i>PFR</i> ($X = 0,90$)	100
Tabela 6	– Validação <i>PFR</i> com Reciclo ($R = 2,0; X = 0,90$)	101

Lista de abreviaturas e siglas

<i>API</i>	<i>Application Programming Interface</i>
<i>ASGI</i>	<i>Asynchronous Server Gateway Interface</i>
<i>BEP</i>	<i>Best Efficiency Point</i>
<i>CD</i>	<i>Continuous Delivery / Continuous Deployment</i>
<i>CI</i>	<i>Continuous Integration</i>
<i>CSS</i>	<i>Cascading Style Sheets</i>
<i>CSTR</i>	<i>Continuous Stirred Tank Reactor</i>
DCOU	Dimensionamento Computacional de Operações Unitárias
EDO	Equação diferencial ordinária
<i>HTML</i>	<i>HyperText Markup Language</i>
<i>LHS</i>	<i>Left-Hand Side</i>
<i>PFR</i>	<i>Plug Flow Reactor</i>
<i>NPSH</i>	<i>Net Positive Suction Head</i>
<i>NPSH_d</i>	<i>Net Positive Suction Head</i> disponível
<i>NPSH_r</i>	<i>Net Positive Suction Head</i> requerido
<i>REST</i>	<i>Representational State Transfer</i>
<i>RHS</i>	<i>Right-Hand Side</i>
<i>SVG</i>	<i>Scalable Vector Graphics</i>
<i>UI</i>	<i>User Interface</i>
<i>UX</i>	<i>User Experience</i>
<i>Q</i>	Vazão volumétrica
<i>D</i>	Diâmetro
<i>D_h</i>	Diâmetro hidráulico
<i>H</i>	Altura manométrica

Re	Número de Reynolds
T	Temperatura
P	Pressão
P_c	Pressão crítica
T_c	Temperatura crítica
X	Conversão
Z	Fator de compressibilidade
h_f	Perda de carga
k	Constante cinética
ρ	Massa específica do fluido
γ	Peso específico do fluido
μ	Viscosidade dinâmica
ν	Viscosidade cinemática
τ	Tempo de residência
ε	Coeficiente de expansão volumétrico
ψ	Fator de diluição
g	Aceleração da gravidade

Sumário

1	INTRODUÇÃO	21
2	JUSTIFICATIVA	22
3	OBJETIVOS	23
3.1	Geral	23
3.2	Específico	23
4	METODOLOGIA	25
4.1	Desenvolvimento do <i>Software</i>	25
4.1.1	<i>Backend</i>	25
4.1.2	<i>Frontend</i>	27
4.1.3	<i>Deploy</i>	29
4.1.4	Integração Contínua e Entrega Contínua	30
4.1.5	Distribuição Desktop Multiplataforma	30
4.1.6	Versionamento	31
4.2	Funções de Engenharia	31
4.2.1	Dimensionamento de Tubulações	31
4.2.1.1	Diâmetro Calculado	32
4.2.1.2	Diâmetro Real	33
4.2.2	Reatores	34
4.2.2.1	Reator do Tipo <i>CSTR</i>	40
4.2.2.1.1	Conversão e Cinética (Cálculo do Volume)	41
4.2.2.1.2	Volume e Cinética (Cálculo da Conversão)	45
4.2.2.1.3	Tempo de Residência e Cinética	47
4.2.2.2	Reator do Tipo <i>PFR</i>	49
4.2.3	Tubulações	50
4.2.4	Balanço de Massa	54
4.2.5	Escoamento, Perdas de Carga e Diâmetros Hidráulicos	55
4.2.5.1	Número de Reynolds	56
4.2.5.2	Fator de Atrito	59
4.2.5.3	Perda de Carga – Darcy–Weisbach e Hazen–Williams	61
4.2.5.4	Altura Manométrica entre Dois Pontos	64
4.2.5.5	<i>NPSH</i> Disponível	66
4.2.5.6	Diâmetro Hidráulico para Geometrias Diversas	68
4.2.6	Propriedades Termofísicas	70

4.3	API	76
4.4	Validação de Cálculos e Especificação de Retornos	77
4.4.1	Componentes	77
4.4.1.1	Listar Componentes	78
4.4.1.2	Valor de Propriedade	79
4.4.1.3	Nome de Propriedades	79
4.4.1.4	Propriedades Críticas	80
4.4.1.5	Opções de Propriedades de Misturas	81
4.4.1.6	Valor de Propriedades de Misturas	82
4.4.2	Hidráulicos	83
4.4.2.1	Perda de Carga	83
4.4.2.2	Reynolds	84
4.4.2.3	Fator de Atrito	84
4.4.2.4	Diâmetro Real	85
4.4.2.5	Diâmetro Calculado	85
4.4.2.6	<i>NPSH</i> Disponível	86
4.4.2.7	Head	86
4.4.2.8	Diâmetro Hidráulico	86
4.4.3	Balanços de Massa	87
4.4.3.1	Misturador Ideal	87
4.4.3.2	Reator de Conversão	90
4.4.3.3	Divisor de Corrente (<i>Splitter</i>)	90
4.4.4	Tubulações	93
4.4.4.1	Acessórios	93
4.4.4.2	Especificação de Acessórios	94
4.4.4.3	Composição	94
4.4.4.4	Especificação da Composição	95
4.4.4.5	<i>Schedules</i>	96
4.4.4.6	Diâmetros	97
4.4.4.7	Especificação de Diâmetros	98
4.4.5	Reatores	98
4.4.5.1	<i>CSTR</i> : Reator Tanque Agitado Continuamente	99
4.4.5.2	<i>PFR</i> : Reator Tubular	99
4.4.5.3	<i>PFR</i> com Reciclo	100
4.5	Recursos Didáticos	101
4.5.1	Glossário	101
4.5.2	Explicações Teóricas Embutidas	102
4.5.3	Visualizações Interativas	102
4.5.3.1	Diagrama de Moody	103

4.5.3.2	Curva da Bomba, Margem de <i>NPSH</i> e Mapa de Eficiência	103
4.5.3.3	Diagrama de Levenspiel	104
4.5.3.4	Gráfico de Arrhenius	104
4.5.3.5	Perfis Espaciais no <i>PFR</i>	104
4.5.3.6	Diagrama <i>T-x-y</i> Binário	105
4.5.3.7	Diagrama de McCabe-Thiele	105
4.5.3.8	Envelope de Fase e Curva de Pressão de Vapor	105
4.5.3.9	Diagrama Ternário	105
4.5.3.10	Superfície <i>T-P</i> de Propriedades	106
4.5.4	Cenários de Exemplo	107
4.5.5	Trilhas de Aprendizagem	107
5	RESULTADOS E DISCUSSÕES	109
6	GUIA DE USO DA FERRAMENTA	111
6.1	Acesso e Tela Inicial	111
6.2	Tubulações e Acessórios	112
6.3	Dimensionamento	115
6.4	Escoamento Interno	117
6.5	Perda de Carga e Bombas	120
6.6	Propriedades de Componentes	123
6.7	Reatores Ideais	129
6.8	Balanço de Massa	133
6.9	Glossário e Validações	138
7	CONCLUSÕES	141
	REFERÊNCIAS	143

1 Introdução

O dimensionamento de equipamentos em processos químicos é uma etapa crítica no projeto de plantas industriais, envolvendo princípios fundamentais da engenharia química, como balanços de massa e energia, termodinâmica, transferência de calor e de massa e cinética química. Esses conceitos são aplicados para definir características operacionais e tamanhos de equipamentos, como trocadores de calor, colunas de destilação, reatores, bombas e compressores. O domínio das operações unitárias é essencial para garantir desempenho seguro, eficiente e otimizado em termos de custos nos processos industriais (FELDER; ROUSSEAU, 2005).

Apesar de *softwares* consagrados como *Aspen Plus*®, *HYSYS*® e *ChemCAD*® serem amplamente utilizados na indústria, seu uso é limitado por custos elevados de licenciamento e pela falta de acesso em ambientes acadêmicos (ASPEN TECHNOLOGY, 2026b; ASPEN TECHNOLOGY, 2026a; DATACOR, 2026; ALVES et al., 2022).

Essas ferramentas, embora poderosas, nem sempre priorizam o aprendizado conceitual, o que pode dificultar a compreensão dos cálculos pelos estudantes e engenheiros em formação. Há, portanto, espaço para soluções acessíveis, flexíveis e didáticas. Este trabalho propõe o desenvolvimento do DCOU (*Dimensionamento Computacional de Operações Unitárias*), um *software web* próprio e gratuito que vai além da execução de cálculos isolados: o sistema foi concebido como uma ferramenta didática, articulando rotinas de engenharia com recursos que apoiam diretamente o aprendizado — explicações teóricas, visualizações dos fenômenos físicos, um glossário técnico e cenários de exemplo reutilizáveis —, integrando, assim, conhecimento teórico e programação moderna.

2 Justificativa

O desenvolvimento de um *software* gratuito para dimensionamento de operações unitárias integra a teoria da engenharia química com ferramentas de programação e automação, democratizando o acesso a simulações para instituições de ensino, profissionais autônomos e pequenas empresas desprovidas de licenças comerciais. Isso favorece o aprendizado ativo, permitindo visualizações em tempo real de como parâmetros operacionais afetam o desempenho dos equipamentos. No DCOU, esse propósito didático é materializado por recursos como explicações teóricas embutidas em cada calculadora, visualizações interativas dos fenômenos (regimes de escoamento, diagrama de *Moody*, curvas de *Levenspiel*, diagrama de *McCabe-Thiele*, envelope de fase e outros), um glossário técnico de apoio e cenários de exemplo fornecidos pelo *backend*, aproximando o estudante de fluxos de trabalho reais da engenharia.

No contexto da Indústria 4.0, o domínio de linguagens de programação e bancos de dados é um diferencial para engenheiros químicos, dada a crescente digitalização de processos e a necessidade de análise de dados em tempo real.

Softwares proprietários como *Aspen Plus*® e *HYSYS*® apresentam custo elevado e acesso restrito ([ASPEN TECHNOLOGY, 2026b](#); [ASPEN TECHNOLOGY, 2026a](#); [ALVES et al., 2022](#)), enquanto alternativas *open-source* como o nacional *DWSIM* ([DWSIM, 2026](#)) ainda apresentam limitações quando o objetivo é combinar cálculo, leitura didática dos fenômenos e acoplamento entre módulos em uma experiência customizável. Planilhas em *Excel*® ou rotinas em *MATLAB*®, por sua vez, oferecem baixa modularização e carecem de interfaces amigáveis para simulação de processos complexos. Identifica-se, assim, uma lacuna em soluções abertas, escaláveis e focadas no aprendizado, que combinem rigor técnico com facilidade de uso. Este trabalho busca preencher essa lacuna com um sistema modular e extensível, servindo tanto ao ensino quanto à prática profissional.

Sendo de código aberto, espera-se que estudantes com formação interdisciplinar em engenharia química e desenvolvimento de *software* possam contribuir para expandir e aprimorar o repertório da ferramenta, servindo como precursora para uso em maior escala.

3 Objetivos

3.1 Geral

Desenvolver o DCOU, um *software web* modular para cálculo e apoio didático em operações unitárias, integrando rotinas de tubulações, escoamento, bombas, propriedades termodinâmicas, reatores ideais e balanços de massa, simultaneamente oferecendo recursos que apoiem a compreensão dos conceitos envolvidos. O sistema deve adotar arquitetura cliente-servidor com interface *web* acessível por meio de *API* aberta, mantendo o código disponível publicamente.

3.2 Específico

- Implementar rotinas de cálculo baseadas em equações clássicas para operações unitárias como reatores, tubulações e demais correlatos, com perda de carga, *NPSH* etc;
- Desenvolver uma interface *web* interativa e responsiva para inserção e interpretação de dados em diversos dispositivos, favorecendo a experiência do usuário;
- Integrar um banco de dados local inicial com propriedades físico-químicas de substâncias comuns (coeficientes de difusão, calor específico, entalpia de vaporização etc.);
- Disponibilizar recursos didáticos integrados — glossário técnico, explicações teóricas embutidas e visualizações interativas dos fenômenos físicos — que conectem cada cálculo ao seu fundamento conceitual;
- Disponibilizar cenários de exemplo e entradas de referência que encadeiem múltiplos módulos, reproduzindo fluxos de trabalho reais de engenharia;
- Estruturar trilhas de aprendizagem que organizem o uso dos módulos em sequência didática;
- Validar os resultados comparando-os com dados da literatura, assegurando precisão e confiabilidade dos cálculos.

4 Metodologia

4.1 Desenvolvimento do *Software*

A metodologia adotada combina fundamentos da engenharia química com técnicas modernas de desenvolvimento de *software*, visando garantir precisão nos cálculos e acessibilidade por meio de uma interface *web* intuitiva. O projeto foi estruturado em etapas que abrangem desde o levantamento bibliográfico das equações de dimensionamento até a implementação e validação do sistema completo. Para fins de organização, o desenvolvimento é descrito a seguir em seus principais eixos técnicos: *backend*, *frontend*, *deploy*, automação de integração e entrega, distribuição *desktop* e versionamento.

4.1.1 *Backend*

O *backend* foi desenvolvido em *Python*, utilizando o *framework FastAPI*, escolhido por sua alta *performance*, suporte a tipagem forte e facilidade na criação de *APIs RESTful*. Essa camada é responsável por processar os cálculos, gerenciar as rotas de comunicação (*endpoints*) e servir os resultados ao *frontend*. Além disso, por se tratar de uma linguagem de alto nível, sua sintaxe é simples e intuitiva para iniciantes.

A Tabela 1 resume as principais bibliotecas e *frameworks* utilizados, sua funcionalidade e a correspondência com aplicações típicas da engenharia química.

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
<i>FastAPI</i>	<i>Framework web</i> moderno para criação de <i>APIs RESTful</i> em <i>Python</i> , com validação automática de dados e documentação interativa.	<i>Backend</i> para expor os cálculos de engenharia química através de <i>endpoints</i> HTTP, permitindo que a interface <i>web</i> ou outros sistemas acessem as funções de dimensionamento, balanço de massa, cálculos de reatores etc.
<i>Uvicorn</i>	Servidor <i>ASGI</i> para executar aplicações <i>web</i> assíncronas em <i>Python</i> .	Executa o servidor da <i>API FastAPI</i> , processando as requisições dos cálculos.

(Continua na próxima página)

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*
(continuação)

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
<i>Pydantic</i>	Biblioteca para validação de dados e gerenciamento de configurações usando <i>Python type hints</i> .	Valida os dados de entrada dos cálculos (vazões, temperaturas, pressões, propriedades físicas etc.), garantindo que os parâmetros fornecidos estejam no formato e tipo corretos antes de executar os cálculos.
<i>CoolProp</i>	Biblioteca especializada em propriedades termodinâmicas de fluidos, com banco de dados extenso de substâncias.	Cálculo de propriedades termodinâmicas (densidade, viscosidade, entalpia, entropia, calor específico etc.) de compostos e misturas.
<i>Pint</i>	Gerenciamento de unidades físicas, conversões e cálculos dimensionais	Permite trabalhar com diferentes sistemas de unidades, garantindo consistência dimensional nos cálculos.
<i>Matplotlib</i>	Biblioteca de visualização de dados e criação de gráficos 2D/3D.	Gera gráficos de processos químicos como curvas de reação, perfis de temperatura, diagramas de fases, curvas de bomba, e visualizações de resultados de simulações.
<i>NumPy</i>	Biblioteca fundamental para computação científica com <i>arrays</i> multidimensionais e operações matemáticas vetorizadas	Realiza cálculos matriciais, interpolações, operações vetoriais para balanços de massa e energia, resoluções de sistemas de equações lineares em processos químicos
<i>SciPy</i>	Conjunto de algoritmos científicos para otimização, integração numérica, álgebra linear, estatística	Resolução de EDOs, otimiza processos químicos, realiza integrações numéricas para cálculos de conversão, ajuste de curvas experimentais

(Continua na próxima página)

Tabela 1 – Bibliotecas e *frameworks* utilizados no *backend*
(continuação)

Nome	Funcionalidade	Como se enquadra no uso da Engenharia Química
<i>Pytest</i>	<i>Framework</i> de testes, com suporte a testes unitários, testes de integração, <i>fixtures</i> , parametrização e relatórios detalhados.	Permite validar automaticamente a confiabilidade dos cálculos de engenharia química, garantindo que equações, modelos matemáticos e rotinas numéricas estejam corretos e continuem funcionando após alterações no código.

Fonte: Elaboração própria (2026).

4.1.2 Frontend

O *frontend* corresponde à interface gráfica do usuário, sendo a parte visual e interativa do sistema, acessada diretamente pelo navegador. Ele é responsável por exibir os resultados dos cálculos, receber dados de entrada e enviar as informações ao servidor (*backend*).

O *frontend* foi desenvolvido com *React*, *TypeScript* e *Vite*, combinação que favorece a composição da interface em componentes reutilizáveis, a tipagem dos contratos entre formulários e serviços e um ciclo de desenvolvimento ágil para compilação e distribuição da aplicação. A camada de navegação é estruturada com *React Router*, o que permite organizar os módulos de tubulações, escoamento, bombas, componentes, reatores, balanço de massa e glossário em rotas aninhadas e previsíveis, sob um mesmo *shell* de aplicação.

Na camada visual, a aplicação utiliza *Tailwind CSS* para estilização utilitária, em conjunto com primitivas acessíveis de *@base-ui/react* e componentes compostos no padrão da biblioteca *shadcn*. Os ícones são fornecidos por *Lucide React*, as notificações rápidas utilizam *Sonner* e a renderização matemática é realizada com *KaTeX*, o que permite exibir expressões em $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ diretamente no navegador. Para as visualizações, a interface adota predominantemente componentes vetoriais em *SVG* especializados para o contexto de engenharia química, apoiados por uma camada compartilhada de grades, eixos, marcadores, rótulos numéricos e legendas coloridas. Com isso, diagramas como *Moody*, *Levenspiel*, *McCabe-Thiele*, envelope de fase, curva de pressão de vapor e mapas de processo podem compartilhar contratos de dados e convenções visuais sem replicar a lógica de apresentação em cada página.

Na organização da experiência, a interface foi concebida como um painel educacional único, com navegação lateral persistente, página inicial orientada por trilhas de aprendiza-

gem e áreas de acesso rápido aos principais módulos. A disposição dos elementos busca explicitar o propósito pedagógico de cada seção, reduzindo a fricção de uso e favorecendo a leitura imediata dos resultados, dos gráficos e dos indicadores de processo. A Figura 1 ilustra a interface de usuário desenvolvida para o módulo de *Piping*, evidenciando as escolhas de *design* descritas.

Figura 1 – Interface gráfica do módulo de cálculos de tubulação (*Piping*)

The screenshot displays the 'Chemical Engineering Calculator' interface, specifically the 'Piping Calculations' section. The header includes the title 'Chemical Engineering Calculator' and the subtitle 'Comprehensive toolkit for chemical engineering calculations'. A navigation bar at the top lists various modules: Piping, Sizing, Flow, Pump, Components, Reactor, and Mass Balance. The 'Piping' module is currently selected.

The 'Piping Calculations' section is divided into two main panels:

- Composition Selection:** This panel includes a dropdown menu for 'Select Composition' set to 'Commercial steel'. Below it, a table titled 'Composition Details' provides further information.
- Schedule & Diameter:** This panel includes a dropdown menu for 'Select Schedule' set to 'SCH10' and another dropdown for 'Select Diameter' set to '6 mm'. Below these, a table titled 'Diameter Details' provides further information.

Property	Value	Units
Name	Commercial steel	-
Description	Standard carbon steel piping used in many industrial applications.	-
Applications	Water, gas, oil, steam transport in industrial settings.	-
Specifications - Roughness	0.0600000	millimeter
Specifications - Roughness Coefficient	135.000	dimensionless

Property	Value	Units
External Diameter	10.3000	millimeter
Thickness	1.24500	millimeter
Weight	0.277000	kilogram / meter

Fonte: Elaboração própria (2026).

Tabela 2 – Bibliotecas e *frameworks* utilizados no *frontend*

Nome	Funcionalidade
<i>Vite</i>	Ferramenta de <i>build</i> e ambiente de desenvolvimento utilizada para empacotar a aplicação <i>React</i> , com inicialização rápida e geração otimizada dos arquivos estáticos.
<i>React</i>	Biblioteca declarativa para composição de interfaces em componentes reutilizáveis e encapsulados.
<i>TypeScript</i>	Superset tipado de <i>JavaScript</i> utilizado para formalizar contratos de dados, reduzir erros de integração e melhorar a manutenção.
<i>React Router</i>	Biblioteca de roteamento usada para organizar os módulos do sistema em rotas navegáveis dentro da aplicação <i>web</i> .
<i>Tailwind CSS</i>	<i>Framework</i> utilitário que facilita a criação de interfaces responsivas e personalizadas por meio de classes pré-definidas de estilo.
<i>@base-ui/react</i>	Conjunto de primitivas acessíveis para construção de botões, diálogos, menus e outros controles de interface, sobre as quais são compostos os componentes visuais do sistema.
<i>shadcn</i>	Camada de componentes reaproveitáveis, estilizados com utilitários e composta sobre primitivas acessíveis, empregada para estruturar cartões, botões, painéis e áreas de navegação.
<i>Lucide React</i>	Biblioteca de ícones vetoriais modulares para navegação, estados visuais e ações contextuais na interface.
<i>Sonner</i>	Biblioteca para notificações breves e não bloqueantes associadas às ações do usuário na interface.
<i>KaTeX</i>	Biblioteca <i>JavaScript</i> para renderização rápida de equações em $\text{\LaTeX}$ diretamente no navegador.
<i>Geist Variable</i>	Família tipográfica variável utilizada para padronizar a leitura visual e reforçar a identidade da interface.

Fonte: Elaboração própria (2026).

Além da pilha principal, a interface utiliza uma camada de componentes visuais próprios para o contexto da Engenharia Química, incluindo cartões de módulo, painéis didáticos, blocos de métricas, indicadores de contexto e visualizações auxiliares para apoiar a interpretação dos fenômenos representados. Essa camada complementa a comunicação com a *API* sem modificar os contratos de dados, ao mesmo tempo em que preserva a consistência visual e a legibilidade dos módulos. O mesmo *frontend* também consome *endpoints* de exemplo, o que permite pré-preencher entradas realistas para módulos como escoamento, dimensionamento, bombas e componentes antes da execução normal dos cálculos no *backend*.

4.1.3 Deploy

O *deploy* do sistema foi realizado por meio de *Docker*, tecnologia que permite empacotar a aplicação e suas dependências em contêineres portáteis e reproduzíveis. Essa abordagem

elimina problemas de compatibilidade entre ambientes e facilita o uso do *software* em diferentes sistemas operacionais.

Foram criadas imagens separadas para o *backend* e o *frontend*, que podem ser executadas em conjunto ou individualmente. No caso da interface, o processo de empacotamento compila a aplicação com *Vite* e distribui os arquivos gerados por meio de um servidor *Nginx*, preservando a possibilidade de distribuição independente da *API*. O uso do *Docker* também simplifica futuras implementações em servidores remotos e integração com orquestradores, garantindo escalabilidade e isolamento dos serviços. A aplicação encontra-se disponível para acesso público em <<https://tcc.joao.baza.dev.br>>.

4.1.4 Integração Contínua e Entrega Contínua

As práticas de *Continuous Integration (CI)* e *Continuous Delivery/Continuous Deployment (CD)* foram incorporadas ao projeto como mecanismos de apoio à confiabilidade e à rastreabilidade do desenvolvimento. Em termos conceituais, a integração contínua consiste na validação automatizada de alterações submetidas ao repositório, mediante execução recorrente de rotinas como compilação, testes e verificações de empacotamento. Já a entrega contínua corresponde à preparação sistemática de artefatos distribuíveis, de modo que o código validado possa ser convertido em versões aptas para publicação com menor variabilidade operacional.

No repositório do DCOU, essa automação foi estruturada por meio de fluxos em *GitHub Actions*. O fluxo principal de *CI* executa testes do *backend* em *Python*, testes e geração de *build* do *frontend*, verificações *end-to-end* com *Playwright* e uma validação de fumaça da distribuição *desktop*. Complementarmente, há fluxos dedicados à publicação de imagens *Docker* do *backend* e do *frontend* no *GitHub Container Registry*, bem como um fluxo específico para geração e disponibilização de instaladores *desktop* em diferentes sistemas operacionais a partir de versões etiquetadas do projeto.

Sob a perspectiva metodológica, a adoção de *CI/CD* mostra-se particularmente pertinente em um projeto acadêmico de código aberto. Como o DCOU combina um *backend* científico, uma interface reativa em *React/TypeScript*, rotinas de *deploy* e uma frente de empacotamento *desktop*, a automação das verificações reduz o risco de regressões entre camadas distintas e amplia a reprodutibilidade das entregas. Além disso, a existência desses fluxos fortalece a transparência do desenvolvimento colaborativo, pois estabelece critérios objetivos para validação do código e para correspondência entre o estado do repositório e os artefatos efetivamente distribuídos.

4.1.5 Distribuição Desktop Multiplataforma

Além da disponibilização da aplicação em ambiente *web*, foi iniciada uma frente de desenvolvimento orientada à distribuição do DCOU como aplicação *desktop* multiplataforma

por meio de *Electron*. Essa iniciativa busca ampliar o alcance do sistema sem introduzir uma segunda interface independente, preservando como base a mesma aplicação principal já desenvolvida para o navegador. Nessa abordagem, a camada de apresentação, a navegação e os componentes visuais permanecem centralizados na base do *frontend*, enquanto a adaptação para o ambiente *desktop* concentra-se no empacotamento e na orquestração local dos serviços necessários.

Do ponto de vista técnico, essa estratégia apresenta duas vantagens principais. A primeira é a redução de duplicação estrutural, uma vez que não se torna necessário manter implementações paralelas para *web* e *desktop*. A segunda é o controle da complexidade arquitetural, pois a evolução funcional dos módulos de cálculo, das visualizações e dos fluxos da interface continua concentrada em uma única base de código. No estado atual do projeto, a pasta dedicada ao ambiente *desktop* já contempla configuração de empacotamento com *electron-builder*, scripts de construção do *frontend* e do *backend*, além de testes específicos para o hospedeiro local e para a validação dos artefatos gerados.

Em termos de distribuição, a estrutura adotada permite preparar instaladores para Linux, Windows e macOS, reutilizando o *bundle* compilado do *frontend* e uma versão empacotada do *backend*. Tal direção é coerente com os objetivos gerais do projeto, pois amplia a portabilidade da ferramenta e preserva consistência entre plataformas sem elevar proporcionalmente o custo de manutenção. Desse modo, a frente *desktop* representa uma extensão arquitetural do DCOU, e não uma bifurcação independente de seu desenvolvimento.

4.1.6 Versionamento

O controle de versão foi implementado com o *Git* para rastrear alterações no código-fonte, permitir colaboração e garantir segurança no desenvolvimento. O repositório foi criado no *GitHub* como público, permitindo a colaboração entre diferentes contribuidores. O uso de versionamento também favorece a rastreabilidade científica do projeto, assegurando transparência e reprodutibilidade dos resultados obtidos. O repositório original pode ser acessado em <https://github.com/joao-baza/TCC>.

4.2 Funções de Engenharia

Para cada função existente no projeto, as entradas, saídas e definições de retorno são descritas a seguir. Algumas rotinas recuperam informações do banco de dados local ou das bibliotecas utilizadas, enquanto outras calculam diretamente parâmetros de projeto.

4.2.1 Dimensionamento de Tubulações

Tubulações são componentes por onde o fluido escoar, permitindo a transferência de matéria entre equipamentos, desde a entrada de matéria-prima até a saída de produtos.

Seu correto dimensionamento é imprescindível para a segurança e a eficiência do processo.

O dimensionamento consiste em definir o diâmetro nominal e o *schedule* capazes de conduzir as vazões dentro de faixas de velocidade e perda de carga economicamente ótimas. As grandezas envolvidas incluem vazão (mássica ou volumétrica), velocidade admissível, propriedades do fluido (massa específica, viscosidade), comprimento equivalente (devido a acessórios) e características do material (rugosidade, pressão de projeto).

Na prática, calcula-se um diâmetro a partir da vazão e velocidade alvo e seleciona-se o diâmetro comercial imediatamente superior disponível, verificando em seguida a perda de carga pelo método de Darcy–Weisbach e a integridade mecânica conforme normas aplicáveis. Temperatura e pressão afetam propriedades físicas do fluido e critérios de espessura da tubulação; adicionalmente, a topografia da planta e o número de acessórios elevam o comprimento equivalente total.

Subdimensionar uma linha implica grandes perdas de carga e custos energéticos; superdimensionar acarreta CAPEX elevado e pode gerar velocidades muito baixas (favorecendo sedimentação em linhas com sólidos). O embasamento teórico combina a equação de continuidade, correlações de atrito e seleção de materiais, apoiado no uso do diagrama de Moody (WHITE, 2018).

4.2.1.1 Diâmetro Calculado

O diâmetro calculado é baseado na vazão e na velocidade média do fluido escoando em um duto circular, de acordo com a Equação 4.1:

$$D = \sqrt{\frac{4 \cdot Q}{\pi \cdot V}} \quad (4.1)$$

Em que:

- Q – vazão volumétrica do fluido;
- V – velocidade do fluido;
- D – diâmetro hidráulico do duto circular.

Essa relação advém diretamente da equação da continuidade para escoamento em regime permanente (WHITE, 2018). No código, a função `get_calculated_diameter` presente na classe dos cálculos hidráulicos utiliza o controle de unidades via *Pint* e retorna o resultado em milímetros, assegurando consistência dimensional.

Figura 2 – Código da função `get_calculated_diameter()`

```

def get_calculated_diameter(self, parameters: Dict[str, object]):
    """
    Hydraulic diameter from flow rate and velocity.

    Parameters
    -----
    parameters : dict
        ``{"flow_rate": <m³/s>, "velocity": <m/s>}``

    Returns
    -----
    pint.Quantity
        Calculated diameter (mm).
    """
    req = ["flow_rate", "velocity"]
    self._require_keys(parameters, req)
    self._validate_numeric(parameters, req)

    Q = parameters["flow_rate"] * self.ureg.m**3 / self.ureg.s
    V = parameters["velocity"] * self.ureg.m / self.ureg.s

    # Use **0.5 instead of math.sqrt to keep units intact
    D = (4 * Q / (np.pi * V)) ** 0.5
    D_mm = D.to(self.ureg.mm)

    if D_mm <= 0 * self.ureg.mm:
        raise ValueError(f"Diameter must be positive, got {D_mm}.")

    return D_mm

```

Fonte: Elaboração própria (2026).

4.2.1.2 Diâmetro Real

O diâmetro real corresponde ao diâmetro nominal da tubulação a ser utilizada. Fabricantes produzem tubos em geometrias padronizadas (padrões de espessura como SCH 10, SCH 40, SCH 80 etc.). Deve-se calcular inicialmente o diâmetro necessário pelo método anterior e então escolher, em tabela de catálogo, a tubulação comercial com diâmetro interno imediatamente superior, no material e *schedule* desejados. Essa lógica é implementada na função `get_real_diameter`, garantindo que o diâmetro selecionado comportará a vazão com velocidade dentro do admissível. Após a seleção, verifica-se a perda de carga resultante e, se necessário, itera-se para outro diâmetro.

Figura 3 – Código da função `get_real_diameter()`

```

def get_real_diameter(self, parameters: Dict[str, object]):
    """
    Pick the next available nominal diameter above the calculated one.

    Parameters
    -----
    parameters : dict
        ``{"calculated_diameter": <mm>, "schedule": <str>}``

    Returns
    -----
    pint.Quantity
        Nominal diameter (mm).
    """
    req = ["calculated_diameter", "schedule"]
    self._require_keys(parameters, req)
    self._validate_numeric(parameters, ["calculated_diameter"])

    d_calc = parameters["calculated_diameter"]
    schedule = parameters["schedule"]

    diameters = sorted(Piping().diameters(schedule))
    for dn in diameters:
        if dn > d_calc:
            return dn * self.ureg.mm

    raise ValueError(
        f"No diameter in schedule '{schedule}' exceeds {d_calc} mm."
    )

```

Fonte: Elaboração própria (2026).

4.2.2 Reatores

Reatores são equipamentos projetados para promover reações químicas, convertendo reagentes em produtos sob condições controladas de operação.

Modelos ideais como *CSTR*, *PFR* e batelada fornecem limites úteis para projeto e análise de desempenho. As variáveis centrais envolvidas no projeto de reatores incluem conversão dos reagentes, seletividade e rendimento (em caso de múltiplas reações), velocidade de reação, tempo de residência, volume do reator e condições de operação (temperatura, pressão). Industrialmente, o correto dimensionamento de reatores afeta diretamente a capacidade produtiva, a qualidade do produto e a segurança, especialmente no controle de remoção de calor em reações exotérmicas. A cinética química depende fortemente da temperatura (lei de Arrhenius) e da forma funcional da taxa de reação (ordem de reação, efeitos de inibição, presença de inertes, variação volumétrica em fase gasosa etc.). Erros na

previsão de conversão ou no cálculo do volume reacional podem levar a baixa seletividade, formação de pontos quentes (*hot spots*) e riscos operacionais. A base teórica conjuga balanços de massa por componente com leis cinéticas apropriadas, integradas no espaço (*PFR*) ou no tempo de residência (*CSTR*), podendo incluir balanço de energia quando o regime térmico é relevante (FOGLER, 2009; LEVENSPIEL, 2000).

No escopo deste projeto, são considerados reatores isotérmicos e monofásicos (todos os reagentes e produtos em uma única fase, líquida ou gasosa). O código rejeita misturas de componentes em estados físicos diferentes, a fim de garantir a validade dos balanços e do fator de expansão volumétrica utilizado para o estado atual dos módulos de cálculo para reatores.

Algumas lógicas gerais se aplicam a todos os tipos de reatores implementados. Por exemplo, a determinação do reagente limitante, onde: identifica-se aquele reagente cuja razão entre a vazão molar de entrada e seu coeficiente estequiométrico é a menor dentre os reagentes presentes. Em termos de cálculo, percorrem-se os reagentes (coeficientes estequiométricos negativos), calculando conforme a Equação 4.2:

$$razao_i = \frac{N_{i0}}{N_{u0}} \quad (4.2)$$

em que N_{i0} é a vazão molar de entrada do componente i e N_{u0} é seu coeficiente estequiométrico. O reagente com menor valor de $razao_i$ é o limitante (FELDER; ROUSSEAU, 2005).

Figura 4 – Código da função *determine_limiting_reagent()* – Parte 1/2

```
def determine_limiting_reagent(self, parameters):  
    """Determine the limiting reagent based on stoichiometric coefficients and  
    component concentrations."""  
    self._require_keys(parameters, ["components", "stoichiometric_coefficients"])  
  
    components = parameters["components"]  
    stoichiometric_coefficients = parameters["stoichiometric_coefficients"]  
  
    if len(components) != len(stoichiometric_coefficients):  
        raise ValueError("Components and stoichiometric coefficients must have  
the same length.")  
  
    required_keys = ["flow_rate_inlet", "molar_concentration_inlet",  
"component_name"]  
    for component in components:  
        for key in required_keys:  
            if key not in component:  
                raise KeyError(f"Missing key '{key}' in component {component}")  
  
    limiting_index = None  
    min_ratio = float('inf')  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 5 – Código da função `determine_limiting_reagent()` – Parte 2/2

```

def determine_limiting_reagent(self, parameters):
    """Determine the limiting reagent based on stoichiometric coefficients and
    component concentrations."""
    ...

    for i, component in enumerate(components):
        coef = stoichiometric_coefficients[i]

        if coef == 0:
            raise ValueError(f"Stoichiometric coefficient for component
{component['component_name']} is zero.")

        if coef < 0: # It's a reactant
            flow_rate = component["flow_rate_inlet"] * self.ureg.m**3 /
self.ureg.s
            molar_concentration = (component["molar_concentration_inlet"] *
self.ureg.mol / self.ureg.L).to(self.ureg.mol / self.ureg.m**3)

            ratio = (flow_rate * molar_concentration / abs(coef)).magnitude #
mol/s per mol

            if ratio < min_ratio:
                min_ratio = ratio
                limiting_index = i # Store the index of the limiting reagent

    if limiting_index is None:
        raise ValueError("No limiting reagent found. Check stoichiometric
coefficients and components.")

    return limiting_index

```

Fonte: Elaboração própria (2026).

A velocidade de reação é modelada, em geral, por uma lei de potência do tipo:

$$r = k \cdot \prod C_i^{m_i} \quad (4.3)$$

Em que:

- $C_i^{m_i}$ – concentração do reagente i elevada à sua ordem de velocidade;
- r – taxa de reação por unidade de volume;
- k – constante de velocidade.

Figura 6 – Código da função `_calculate_concentration_and_rate()` – Trecho da velocidade da reação

```
def _calculate_concentration_and_rate(self, components,
    stoichiometric_coefficients, reaction_rate_params,
    limiting, C0_i, C_lim0, X, dilution_factor):
    """Calculate the reactor concentrations and reaction rate for a given
    conversion."""
    # --[Cálculo de concentrações omitidas para reduzir imagem]--

    # Calculate the reaction rate  $r = k \prod C_i^{n_i}$ 
    rate_expression = 1.0 # Initialize variable
    for i, n_i in enumerate(reaction_rate_params["reaction_orders"]):
        if n_i != 0: # Only consider terms with non-zero order
            rate_expression *= concentration_values[i] ** n_i

    k_unit = self.ureg.Unit('mol/m³/s') / rate_expression.units
    k = reaction_rate_params["k"] * k_unit
    r = k * rate_expression

    return outlet_concentrations, k, r
```

Fonte: Elaboração própria (2026).

A integração da cinética com a estequiometria é feita via conversão X do reagente limitante. Os balanços estequiométricos relacionam as concentrações de saída de cada componente à conversão X . Para cada componente i na reação, têm-se a seguinte relação geral, sendo o valor do mais ou menos positivo para formação (produtos), negativo para consumo (reagentes) e, claro, uma conversão igual a zero para inertes.

$$C_i = \frac{C_{i0} \pm \left(\frac{n_i}{n_A} \cdot C_A \cdot X\right)}{\varepsilon} \quad (4.4)$$

Em que:

- C_i – concentração do componente na saída;
- C_{i0} – concentração do componente na entrada;
- ε – coeficiente de expansão volumétrico;
- X – conversão do reagente limitante;
- C_A – concentração do reagente limitante;
- n_i – coeficiente estequiométrico do componente;
- n_A – coeficiente estequiométrico do reagente limitante.

Figura 7 – Código da função `_calculate_concentration_and_rate()` – Trecho da concentração dos componentes

```
def _calculate_concentration_and_rate(self, components,
    stoichiometric_coefficients, reaction_rate_params,
    limiting, C0_i, C_lim0, X, dilution_factor):
    """Calculate the reactor concentrations and reaction rate for a given
    conversion."""
    # --[Cálculo da velocidade da reação omitida para reduzir imagem]--

    outlet_concentrations = {}
    concentration_values = []
    for i in range(len(components)):
        if stoichiometric_coefficients[i] < 0: # reactant
            if i == limiting:
                # For the limiting reactant
                Ci = C0_i[i] * (1 - X) / dilution_factor
            else:
                # For other reactants
                theta_i = C0_i[i]/C_lim0
                stoichiometric_ratio = abs(stoichiometric_coefficients[i]) /
                abs(stoichiometric_coefficients[limiting])
                X_i = X / stoichiometric_ratio
                if X_i > 1: # Cannot have more than 100% conversion
                    X_i = 1
                Ci = C0_i[i] * (1 - X_i) / dilution_factor
        else:
            if stoichiometric_coefficients[i] > 0: # product
                # For products
                stoichiometric_ratio = stoichiometric_coefficients[i] /
                abs(stoichiometric_coefficients[limiting])
                # Ensure C_lim0 is a single value, not a sequence
                lim_conc = C_lim0 if isinstance(C_lim0, (int, float)) or
                hasattr(C_lim0, 'magnitude') else C_lim0[0]
                Ci = (C0_i[i] + stoichiometric_ratio * lim_conc * X) /
                dilution_factor
            else: # Inert (stoichiometric_coefficients[i] = 0)
                # For inerts, just dilution
                Ci = C0_i[i] / dilution_factor

        component_name = components[i]["component_name"]
        outlet_concentrations[component_name] = Ci
        concentration_values.append(Ci)

    return outlet_concentrations, k, r
```

Fonte: Elaboração própria (2026).

Para determinação do coeficiente de expansão volumétrico para reações gasosas, a Equação 4.5 deve ser utilizada.

$$\varepsilon = y_{A0} \cdot \frac{\sum n_P - \sum n_R}{|n_A|} \quad (4.5)$$

Em que:

- $\sum n_P$ – somatório dos coeficientes estequiométricos dos produtos gasosos;

- $\sum n_R$ – somatório dos coeficientes estequiométricos dos reagentes gasosos;
- y_{A0} – fração molar do reagente limitante na alimentação;
- n_A – coeficiente estequiométrico do reagente limitante.

$$\varphi = (1 + \epsilon X) \frac{P}{P_0} \frac{T_0}{T} \quad (4.6)$$

Figura 8 – Código da função `_calculate_dilution_factor()`

```
def _calculate_dilution_factor(self, components, stoichiometric_coefficients, limiting, y_A0):
    """Calculate the dilution factor based on the stoichiometric coefficients and yield."""
    has_gas_phase = any(c["state"] == "gaseous" for c in components)

    if has_gas_phase:
        mols_prod = sum(coef for coef in stoichiometric_coefficients if coef > 0)
        mols_reag = sum(abs(coef) for coef in stoichiometric_coefficients if coef < 0)
        ε = (mols_prod - mols_reag) / abs(stoichiometric_coefficients[limiting]) * y_A0
    else:
        ε = 0.0

    return ε
```

Fonte: Elaboração própria (2026).

Onde ϵ representa a variação de número de mols por mol de A reagido. O fator ψ agrega a expansão/contração estequiométrica (via ϵX) e as correções de pressão e temperatura em relação às condições iniciais (LEVENSPIEL, 2000). Essas relações estequiométricas permitem calcular as concentrações de saída de todos os componentes a partir da conversão X do reagente limitante, sendo a base para os balanços nos diferentes modelos de reatores.

4.2.2.1 Reator do Tipo *CSTR*

O reator do tipo *CSTR* é um tanque agitado operando em regime contínuo. Assume-se mistura completa no interior do tanque (composição uniforme em todo o volume). A conversão é desenvolvida integralmente dentro do volume do reator, dependendo do balanço entre a cinética de reação e o tempo de residência dos reagentes no tanque.

No sistema implementado, são suportados três casos de uso: (i) conversão desejada e cinética conhecidas (calcula-se o volume do reator necessário), (ii) volume disponível e cinética conhecidas (calcula-se a conversão atingida) e (iii) tempo de residência e cinética conhecidos (calcula-se a conversão atingida). As saídas calculadas pelo algoritmo incluem, entre outros: conversão do reagente limitante X , vazão molar do reagente limitante na entrada F_{A0} , vazão volumétrica total na saída Q_T , volume do reator V , taxa de reação r

no interior, concentrações de saída $C_{i,\text{saída}}$, fator de diluição ψ , coeficiente de expansão ε e tempo de residência τ .

4.2.2.1.1 Conversão e Cinética (Cálculo do Volume)

Passos principais do cálculo:

- Identificação do reagente limitante (conforme descrito anteriormente);
- Cálculo das vazões molares de entrada pela Equação 4.7:

$$N_{i0} = Q_i \cdot C_{i0e} \quad (4.7)$$

para cada componente, bem como da vazão volumétrica total pela Equação 4.8:

$$Q_T = \sum Q_i \quad (4.8)$$

As concentrações iniciais (após mistura das correntes de alimentação) são dadas pela Equação 4.9:

$$C_{i0} = \frac{N_{i0}}{Q_T} \quad (4.9)$$

- Cálculo do fator de diluição total ψ com a conversão alvo X (Equação 4.6);
- Cálculo da taxa de reação r a partir da constante cinética k e das concentrações na saída (considerando X), supondo regime isotérmico (FOGLER, 2009);
- Aplicação do balanço no *CSTR* para o reagente limitante.

Para conversão X conhecida, a equação de *design* é dada por:

$$V = \frac{F_{A0} \cdot X}{-r_A} \quad (4.10)$$

Em que:

- F_{A0} – vazão molar de A na entrada;
- r – taxa de consumo de A por unidade de volume no interior do reator.

Cálculo do tempo de residência:

$$\tau = \frac{Q_T}{V} \quad (4.11)$$

Figura 9 – Código da função `_conversion_and_kinetics_in_cstr()` – Parte 1/2

```
def _conversion_and_kinetics_in_cstr(self, parameters):  
    """Calculates the volume of a CSTR based on conversion and kinetics."""  
    self._require_keys(parameters, ["conversion"])  
    self._validate_numeric(parameters, ["conversion"])  
  
    init_data = self._initialize_reactor(parameters)  
    X = parameters["conversion"] * self.ureg.dimensionless  
  
    # Unpack needed variables  
    components = init_data["components"]  
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]  
    reaction_rate_params = init_data["reaction_rate_params"]  
    limiting = init_data["limiting"]  
    F_A0 = init_data["F_A0"]  
    C0_i = init_data["C0_i"]  
    C_lim0 = init_data["C_lim0"]  
    Q_tot = init_data["Q_tot"]  
    epsilon = init_data["epsilon"]  
    var_operation_conditions = init_data["var_operation_conditions"]  
  
    # Combined factor: effect of volumetric variation and operating conditions  
    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions  
  
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(  
        components, stoichiometric_coefficients, reaction_rate_params,  
        limiting, C0_i, C_lim0, X, dilution_factor  
    )  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 10 – Código da função `_conversion_and_kinetics_in_cstr()` – Parte 2/2

```
def _conversion_and_kinetics_in_cstr(self, parameters):
    """Calculates the volume of a CSTR based on conversion and kinetics."""
    ...

    if any(C.magnitude < 0 for C in outlet_concentrations.values()):
        raise ValueError("Too high conversion – negative concentration
calculated.")

    # Check if r has the correct units (mol/m³/s)
    if r.units != self.ureg.Unit('mol/m³/s'):
        raise ValueError(f"Incorrect unit for reaction rate: {r.units}. Expected:
mol/m³/s")

    if r.magnitude == 0:
        raise ValueError("The reaction rate cannot be zero.")

    # Consumption rate of the limiting reagent
    rate_lim = abs(stoichiometric_coefficients[limiting]) * r # mol/m³/s

    # Calculate the reactor volume using  $V/FA_0 = X/r$ 
    V = F_A0 * X / rate_lim # m³

    residence_time = V / Q_tot

    # Return the volume with the unit in cubic meters
    return {
        "volume": V,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor": epsilon * self.ureg.dimensionless,
        "molar_rate_inlet_(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "residence_time": residence_time,
        "conversion": X,
    }
```

Fonte: Elaboração própria (2026).

A fim de simplificar as rotinas, utiliza-se a função `_initialize_reactor()`, que calcula as concentrações e vazões de cada componente na entrada, bem como o coeficiente de expansão volumétrico, o fator de diluição, as temperaturas e pressões iniciais e finais e o reagente limitante.

Figura 11 – Código da função `_initialize_reactor()` – Parte 1/2

```
def _initialize_reactor(self, parameters):  
    """Standardizes the initialization of reactor calculations."""  
    # Common keys required for all calculations  
    self._require_keys(parameters, ["components", "stoichiometric_coefficients",  
    "operation_conditions"])  
  
    components = parameters["components"]  
    stoichiometric_coefficients = parameters["stoichiometric_coefficients"]  
    operation_conditions = parameters["operation_conditions"]  
  
    # Validate component states  
    last_component_state = components[0]["state"]  
    for component in components:  
        if last_component_state != component["state"]:  
            raise ValueError("Use only liquid or only gaseous components")  
  
    limiting = self.determine_limiting_reagent(parameters)  
  
    # Initialize variables  
    F0_i = []  
    Q_tot = 0 * self.ureg.m**3 / self.ureg.s  
  
    # Calculate inlet molar flow rates and total volumetric flow rate  
    for c in components:  
        flow_rate = c["flow_rate_inlet"] * self.ureg.m**3 / self.ureg.s  
        conc = (c["molar_concentration_inlet"] * self.ureg.mol /  
self.ureg.L).to(self.ureg.mol / self.ureg.m**3)  
        F0_i.append(flow_rate * conc)  
        Q_tot += flow_rate  
  
    F_A0 = F0_i[limiting]  
    C0_i = [F0 / Q_tot for F0 in F0_i]  
    C_lim0 = C0_i[limiting]  
    y_A0 = F_A0 / sum(F0_i)  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 12 – Código da função `_initialize_reactor()` – Parte 2/2

Fonte: Elaboração própria (2026).

4.2.2.1.2 Volume e Cinética (Cálculo da Conversão)

Nesse caso, são conhecidos o volume do reator V e a cinética (expressão da taxa $r(C_i)$, incluindo k). Deseja-se determinar a conversão X atingida no *CSTR*. Isolando X na equação de *design* do *CSTR* obtém-se uma equação implícita. Define-se então uma função objetivo:

$$f(x) = \frac{|v_A| \cdot r_A \cdot V}{F_{A0}} - X \quad (4.12)$$

Então encontra-se sua raiz na faixa $0 < X < 1$, tipicamente utilizando o método de Brent, dada a monotonicidade esperada de $f(x)$. A raiz X que satisfaz $f(x) = 0$ corresponde à conversão no reator.

Figura 13 – Código da função `_volume_and_kinetics_in_cstr()` – Parte 1/2

```
def _volume_and_kinetics_in_cstr(self, parameters):  
    """Calculates the conversion of a CSTR based on volume and kinetics."""  
    self._require_keys(parameters, ["volume"])  
    self._validate_numeric(parameters, ["volume"])  
  
    init_data = self._initialize_reactor(parameters)  
    V = parameters["volume"] * self.ureg.m**3  
  
    # Unpack needed variables  
    components = init_data["components"]  
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]  
    reaction_rate_params = init_data["reaction_rate_params"]  
    limiting = init_data["limiting"]  
    F_A0 = init_data["F_A0"]  
    C0_i = init_data["C0_i"]  
    C_lim0 = init_data["C_lim0"]  
    Q_tot = init_data["Q_tot"]  
    epsilon = init_data["epsilon"]  
    var_operation_conditions = init_data["var_operation_conditions"]  
  
    def objective(X_val):  
        if not (0 < X_val < 1):  
            raise ValueError("Conversion out of bounds")  
  
        dilution_factor = (1 + (epsilon * X_val)) * var_operation_conditions  
  
        outlet_concentrations, k, r = self._calculate_concentration_and_rate(  
            components, stoichiometric_coefficients, reaction_rate_params,  
            limiting, C0_i, C_lim0, X_val, dilution_factor  
        )  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 14 – Código da função `_volume_and_kinetics_in_cstr()` – Parte 2/2

```

def _volume_and_kinetics_in_cstr(self, parameters):
    ...

    if any(Ci.magnitude < 0 for Ci in outlet_concentrations.values()):
        raise ValueError("Ci is negative")

    rate_lim = abs(stoichiometric_coefficients[limiting]) * r
    X_calc = (rate_lim * V / F_A0).to_base_units().magnitude
    return X_calc - X_val

    result = root_scalar(objective, bracket=[1e-9, 1.0 - 1e-9], method='brentq')
    if not result.converged:
        raise ValueError("Failed to converge to a valid conversion value.")

    X = result.root * self.ureg.dimensionless

    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(
        components, stoichiometric_coefficients, reaction_rate_params,
        limiting, C0_i, C_lim0, X, dilution_factor
    )

    return {
        "conversion": X,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor": epsilon * self.ureg.dimensionless,
        "molar_rate_inlet(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "residence_time": V / Q_tot,
        "volume": V,
    }

```

Fonte: Elaboração própria (2026).

4.2.2.1.3 Tempo de Residência e Cinética

Conhecido o tempo de residência τ , calcula-se inicialmente pela Equação 4.13:

$$V = Q_T \cdot \tau \quad (4.13)$$

Em seguida, o problema se reduz ao caso anterior: aplica-se a mesma função objetivo $f(X)$ da Equação 4.12 e resolve-se numericamente para X , obtendo a conversão atingida no *CSTR* com tempo de residência τ especificado.

Figura 15 – Código da função `_residence_time_and_kinetics_in_cstr()` – Parte 1/2

```
def _residence_time_and_kinetics_in_cstr(self, parameters):
    """Calculates the conversion of a CSTR based on residence time and
    kinetics."""
    self._require_keys(parameters, ["residence_time"])
    self._validate_numeric(parameters, ["residence_time"])

    init_data = self._initialize_reactor(parameters)
    residence_time = parameters["residence_time"] * self.ureg.s

    # Unpack needed variables
    components = init_data["components"]
    stoichiometric_coefficients = init_data["stoichiometric_coefficients"]
    reaction_rate_params = init_data["reaction_rate_params"]
    limiting = init_data["limiting"]
    F_A0 = init_data["F_A0"]
    C0_i = init_data["C0_i"]
    C_lim0 = init_data["C_lim0"]
    Q_tot = init_data["Q_tot"]
    epsilon = init_data["epsilon"]
    var_operation_conditions = init_data["var_operation_conditions"]

    # Reactor volume based on residence time
    V = Q_tot * residence_time # m³

    def objective(X_val):
        if not (0 < X_val < 1):
            raise ValueError("Conversion out of bounds")

        dilution_factor = (1 + (epsilon * X_val)) * var_operation_conditions

        outlet_concentrations, k, r = self._calculate_concentration_and_rate(
            components, stoichiometric_coefficients, reaction_rate_params,
            limiting, C0_i, C_lim0, X_val, dilution_factor
        )

    ...
```

Fonte: Elaboração própria (2026).

Figura 16 – Código da função `_residence_time_and_kinetics_in_cstr()` – Parte 2/2

```

def _residence_time_and_kinetics_in_cstr(self, parameters):
    ...

    if any(Ci.magnitude < 0 for Ci in outlet_concentrations.values()):
        raise ValueError("Ci is negative")

    rate_lim = abs(stoichiometric_coefficients[limiting]) * r
    X_calc = (rate_lim * V / F_A0).to_base_units().magnitude
    return X_calc - X_val

    result = root_scalar(objective, bracket=[1e-9, 1.0 - 1e-9], method='brentq')
    if not result.converged:
        raise ValueError("Failed to converge to a valid conversion value.")

    X = result.root * self.ureg.dimensionless

    dilution_factor = (1 + (epsilon * X)) * var_operation_conditions
    outlet_concentrations, k, r = self._calculate_concentration_and_rate(
        components, stoichiometric_coefficients, reaction_rate_params,
        limiting, C0_i, C_lim0, X, dilution_factor
    )

    return {
        "conversion": X,
        "molar_rate_inlet_(limitant)": F_A0,
        "flow_rate_outlet": Q_tot,
        "volume": V,
        "reaction_rate": r,
        "outlet_concentrations": outlet_concentrations,
        "dilution_factor_(1+e * P0*T/P*T0)": dilution_factor *
self.ureg.dimensionless,
        "residence_time": V / Q_tot,
        "dilution_factor": epsilon * self.ureg.dimensionless
    }

```

Fonte: Elaboração própria (2026).

4.2.2.2 Reator do Tipo *PFR*

Um reator tubular é caracterizado por escoamento pistonado, sem mistura axial significativa: cada elemento de fluido que entra percorre o reator como se fosse um reator batelada em movimento, evoluindo em conversão conforme o comprimento.

Para uma conversão alvo X , o volume requerido de um *PFR* ideal é dado pela integração da equação de *design* diferencial ao longo da conversão (LEVENSPIEL, 2000). Matematicamente:

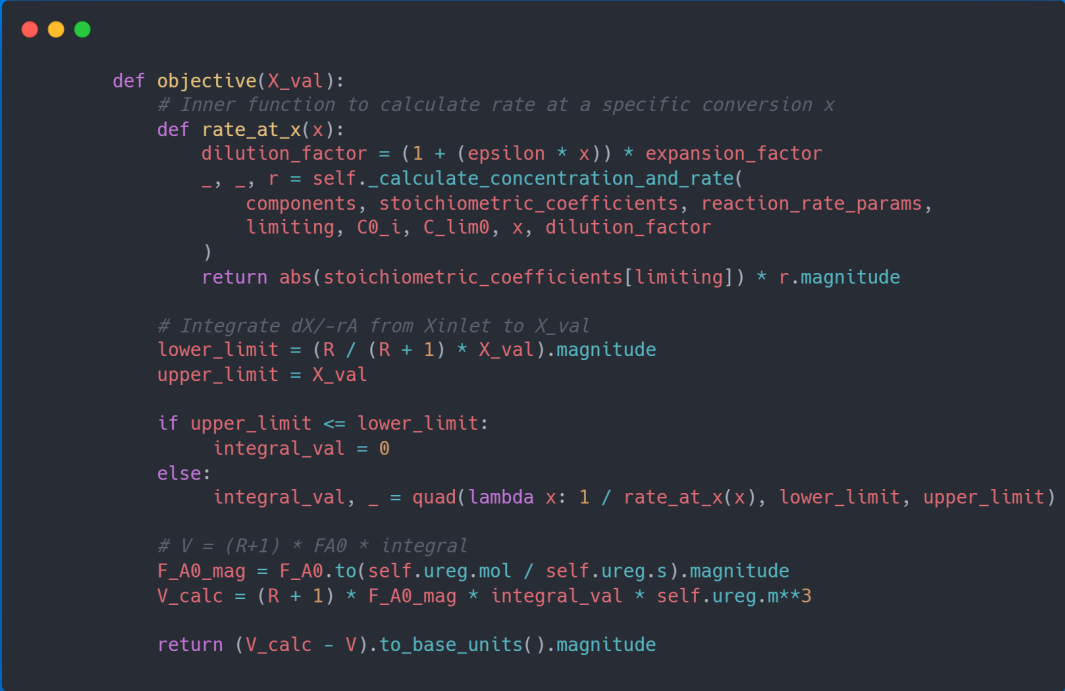
$$V = (R + 1) \cdot F_{A0} \cdot \int_{\frac{R}{R+1}X}^X \frac{dX}{-r_A} \quad (4.14)$$

Tendo que R representa a razão de reagentes não limitantes inertes em relação ao

limitante (se houver variação de vazão volumétrica) e o termo $(R + 1)$ acomoda a variação de vazão na integração. No código, essa integral é avaliada numericamente (por exemplo, via `scipy.integrate.quad`). Para o caso inverso, em que V é conhecido e se busca X , utiliza-se novamente um procedimento iterativo (método de Brent) aplicando a equação de *design* diferencial até encontrar a conversão cujo volume calculado corresponda ao especificado (FOGLER, 2009).

Considerando a similaridade estrutural e semântica com as rotinas de cálculo do *CSTR*, destaca-se a diferença na função objetivo usada para o cálculo de volume, representada na Figura a seguir.

Figura 17 – Código da função objetivo para o *PFR*



```
def objective(X_val):
    # Inner function to calculate rate at a specific conversion x
    def rate_at_x(x):
        dilution_factor = (1 + (epsilon * x)) * expansion_factor
        _, _, r = self._calculate_concentration_and_rate(
            components, stoichiometric_coefficients, reaction_rate_params,
            limiting, C0_i, C_lim0, x, dilution_factor
        )
        return abs(stoichiometric_coefficients[limiting]) * r.magnitude

    # Integrate dX/-rA from Xinlet to X_val
    lower_limit = (R / (R + 1) * X_val).magnitude
    upper_limit = X_val

    if upper_limit <= lower_limit:
        integral_val = 0
    else:
        integral_val, _ = quad(lambda x: 1 / rate_at_x(x), lower_limit, upper_limit)

    # V = (R+1) * FA0 * integral
    F_A0_mag = F_A0.to(self.ureg.mol / self.ureg.s).magnitude
    V_calc = (R + 1) * F_A0_mag * integral_val * self.ureg.m**3

    return (V_calc - V).to_base_units().magnitude
```

Fonte: Elaboração própria (2026).

4.2.3 Tubulações

Para um correto cálculo de perdas de carga e seleção de bombas, o *software* incorpora dados de catálogo de tubulações, rugosidades de materiais e comprimentos equivalentes de acessórios.

Rugosidade de materiais. A rugosidade absoluta do material interno do tubo influencia o fator de atrito em regime turbulento por meio da rugosidade relativa ε/D . Materiais metálicos, plásticos ou com incrustações apresentam valores típicos de rugosidade

tabelados ([WHITE, 2018](#)). Com o tempo de operação, depósitos e corrosão podem elevar a rugosidade efetiva, devendo-se considerar fatores de segurança.

Catálogo de diâmetros. São armazenadas dimensões padrão de tubulações (diâmetro externo, espessura e diâmetro interno resultante) para diferentes *schedules* (por exemplo, SCH 10, 40 e 80) e materiais. Também podem ser incluídos dados de pressão máxima de trabalho e peso por metro. Isso permite que, dado um diâmetro calculado, o programa selecione o diâmetro nominal comercial mais próximo disponível.

Figura 18 – Especificações internas para tubulações

```
class Piping:
    def __init__(self):
        """
        Contains piping specifications.
        Each number key is the nominal diameter in mm, which retrieve:
        ----external_diameter: mm
        ----thickness: mm
        ----weight: kg/m
        ----pressure: psi
        """

        self.ureg = UnitRegistry()

        self.data = {
            "dimensions":{
                "SCH10": {
                    6: {
                        "external_diameter": 10.30,
                        "thickness": 1.245,
                        "weight": 0.277,
                        "max_pressure": None
                    },
                    ...
                },
                "SCH40": {
                    ...
                },
                ...
            },
            "composition":{
                "Commercial steel":{
                    "roughness":0.06,
                    "roughness_coefficient": 135
                },
                ...
            },
            "fittings": {
                "180 degrees Return": {
                    "equivalentLength": 28
                },
                ...
            }
        }
```

Fonte: Elaboração própria (2026).

Coefficientes de perda em acessórios. Curvas, válvulas, expansões, contrações e conexões diversas introduzem perdas de carga locais. No modelo implementado, cada

acessório pode ser contabilizado por um comprimento equivalente L_{eq} ou por um fator adimensional de perda K . O comprimento equivalente total da tubulação é obtido pela soma dos acessórios e acrescentado ao comprimento linear. Esses dados, como fatores K ou L_{eq} típicos para cada acessório padrão, foram compilados de literatura de engenharia de escoamento (PERRY; GREEN, 2007).

No cálculo de perdas de carga totais, o comprimento L da tubulação reta é acrescido do comprimento equivalente $\sum L_{eq}$ das singularidades presentes. Uma estimativa inadequada da rugosidade ou do efeito de acessórios pode produzir erros significativos na previsão de perda de carga, resultando em superdimensionamento ou subdimensionamento de bombas. Assim, o uso de dados acurados de catálogo e correlações de atrito é fundamental para a confiabilidade do dimensionamento. A Figura a seguir exemplifica o cálculo da perda de carga por Darcy–Weisbach utilizando acessórios.

Figura 19 – Cálculo da perda de carga por Darcy–Weisbach com acessórios

```
def _equivalent_length(
    self, fittings: List[Dict[str, object]] | None, diameter_m: "pint.Quantity"
) -> "pint.Quantity":
    """Return the total equivalent length of all fittings."""
    if not fittings:
        return 0 * self.ureg.m

    total = 0 * self.ureg.m
    for f in fittings:
        spec = self.piping.fitting_specifications(f["fitting"])
        total += spec["specifications"]["equivalentLength"].magnitude * diameter_m * f["quantity"]
    return total

def head_loss(self, parameters: Dict[str, object]):
    ...
    if method == "Darcy-Weisbach":
        req = ["friction_factor", "velocity", "pipe_length", "diameter"]
        self._require_keys(parameters, req)
        self._validate_numeric(parameters, req)

        f = parameters["friction_factor"]
        L = parameters["pipe_length"] * self.ureg.m
        V = parameters["velocity"] * self.ureg.m / self.ureg.s
        D = parameters["diameter"] * self.ureg.mm
        D_m = D.to(self.ureg.m)

        Leq = self._equivalent_length(parameters.get("fittings"), D_m)

        hl = f * (L + Leq) * V**2 / (2 * D_m * self.g)
        if hl < 0:
            raise ValueError(f"Head loss cannot be negative, got {hl}.")
        return hl.to(self.ureg.m)
```

Fonte: Elaboração própria (2026).

4.2.4 Balanço de Massa

O balanço de massa aplica o princípio da conservação da massa a sistemas de processo: em regime estacionário, a massa que entra em um sistema deve ser igual à massa que sai, ajustada pela geração ou consumo devido a reações químicas (FELDER; ROUSSEAU, 2005). Para cada componente, a quantidade produzida ou consumida deve ser contabilizada segundo a estequiometria.

$$\text{Acúmulo} = \text{Entradas} - \text{Saídas} + \text{Geração} - \text{Consumo} \quad (4.15)$$

No estado estacionário sem reação, o acúmulo é zero, de modo que:

$$\sum \dot{m}_{\text{entrada}} = \sum \dot{m}_{\text{saída}} \quad (4.16)$$

Por componente i , incluindo reação química:

$$\sum (\dot{m}_{\text{entrada},i} + \dot{m}_{\text{geracao},i}) = \sum (\dot{m}_{\text{saída},i} + \dot{m}_{\text{consumo},i}) \quad (4.17)$$

Em que:

- $\dot{m}$ é a vazão mássica.

Em problemas industriais, frequentemente há reciclo e purgas envolvidos nos balanços. Nesses casos, para divisores de fluxo (*splitters*), define-se por exemplo:

$$F_R = f \cdot F_P \quad (4.18)$$

$$F_G = (1 - f) \cdot F_P \quad (4.19)$$

Em que:

- f é a fração de reciclo;
- F_G é a vazão total do produto bruto;
- F_R é a vazão do reciclo;
- F_P é a vazão da purga.

Esses parâmetros permitem fechar balanços em sistemas com recirculação e calcular a acumulação de inertes ou subprodutos. Na Figura 20, é apresentado o resultado gerado pelo *software* para um balanço de massa contendo reciclo e reação.

Figura 20 – Resultados do balanço de massa com reciclo e reação

Mass Balance Results				
Flow rates are in consistent units throughout. Compositions are mass fractions (when using mass flow units) or molar fractions (when using molar flow units).				
Process Metrics - Fresh Feed: 100.00 (mass or mol)/time - Product Flow: 50.00 (mass or mol)/time - Recycle Ratio: 0.75				
Fresh_Feed Flow Rate: 100.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.8000 B: 0.2000 C: 0.0000 D: 0.0000				
Reactor_Out Flow Rate: 125.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				
Recycle Flow Rate: 75.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				
Product Flow Rate: 50.00 (mass or mol)/time Compositions (mass or molar fractions): A: 0.1574 B: 0.0183 C: 0.6426 D: 0.1817				

Fonte: Elaboração própria (2026).

O balanço de massa fundamenta o dimensionamento de equipamentos como misturadores, separadores, reatores e redes de processo, além de embasar análises econômicas (minimizando perdas de matérias-primas) e de segurança (evitando acúmulo indesejado de compostos). Vale notar que temperatura e pressão influenciam o balanço apenas através de propriedades físicas (densidade, volumes específicos), ou mudanças de fase que afetem as vazões mássicas e molares de cada corrente (FELDER; ROUSSEAU, 2005).

4.2.5 Escoamento, Perdas de Carga e Diâmetros Hidráulicos

O comportamento do escoamento em dutos é governado, principalmente, pelo número de Reynolds, que permite classificar o regime como laminar, de transição ou turbulento. Variáveis-chave incluem o diâmetro hidráulico, a velocidade média, a viscosidade e a densidade do fluido, bem como a rugosidade relativa da parede. Perfis de velocidade, mecanismos de mistura e dissipação de energia por atrito dependem desse regime de escoamento. Para quantificar a perda de carga (queda de pressão) devida ao atrito no escoamento, emprega-se a equação universal de Darcy–Weisbach (WHITE, 2018).

No regime turbulento, o fator de atrito f depende de Re e da rugosidade relativa ε/D , não possuindo solução analítica explícita; por isso, utilizam-se relações empíricas ou aproximadas. Já no regime laminar ($Re < 2300$), o perfil de velocidades é parabólico e f admite a expressão simples dada pela Equação 4.20:

$$f = \frac{64}{Re} \quad (4.20)$$

Em serviços de água a escoamento turbulento, a fórmula empírica de Hazen–Williams é comumente empregada como aproximação prática (PERRY; GREEN, 2007). Temperatura e pressão afetam densidade e viscosidade do fluido, modificando o valor de Re e, consequentemente, as perdas de carga.

Além disso, o material e condições da tubulação (nova, envelhecida) determinam a rugosidade efetiva. Todos esses efeitos impactam diretamente o balanço de energia em uma linha de fluxo e a potência requerida em bombas ou compressores (PERRY; GREEN, 2007; WHITE, 2018).

4.2.5.1 Número de Reynolds

O número de Reynolds (Re) é adimensional e relaciona as forças inerciais com as forças viscosas no escoamento dentro de um duto:

Usando viscosidade dinâmica μ :

$$Re = \frac{D \cdot V \cdot \rho}{\mu} \quad (4.21)$$

Usando viscosidade cinemática ν :

$$Re = \frac{D \cdot V}{\nu} \quad (4.22)$$

Onde D é o diâmetro hidráulico, ρ é a densidade do fluido, V é a velocidade média e μ é a viscosidade dinâmica. Tipicamente, $Re < 2300$ indica regime laminar, $2300 < Re < 4000$ indica regime de transição, e $Re > 4000$ indica regime turbulento plenamente desenvolvido (WHITE, 2018).

Figura 21 – Cálculo do Reynolds – Parte 1/3



Fonte: Elaboração própria (2026).

Figura 22 – Cálculo do Reynolds – Parte 2/3

```

def reynolds(self, parameters: Dict[str, object]):
    ...

    # ----- dynamic viscosity supplied ----- #
    if all(k in parameters for k in model):
        self._validate_numeric(parameters, model)

        D = parameters["characteristic_diameter"] * self.ureg.mm
        V = parameters["velocity"] * self.ureg.m / self.ureg.s
        rho = parameters["density"] * self.ureg.kg / self.ureg.m**3
        mu = parameters["dynamic_viscosity"] * self.ureg.kg / (
            self.ureg.m * self.ureg.s
        )

        Re = (D * rho * V / mu).to_base_units()
        if Re <= 0:
            raise ValueError(f"Reynolds number must be positive, got {Re}.")
        return Re
    ...

```

Fonte: Elaboração própria (2026).

Figura 23 – Cálculo do Reynolds – Parte 3/3

```

def reynolds(self, parameters: Dict[str, object]):
    ...

    # ----- kinematic viscosity supplied ----- #
    if all(k in parameters for k in mode2):
        self._validate_numeric(parameters, mode2)

        D = parameters["characteristic_diameter"] * self.ureg.mm
        V = parameters["velocity"] * self.ureg.m / self.ureg.s
        nu = parameters["kinematic_viscosity"] * self.ureg.m**2 / self.ureg.s

        Re = (D * V / nu).to_base_units()
        if Re <= 0:
            raise ValueError(f"Reynolds number must be positive, got {Re}.")
        return Re

    raise ValueError(
        "Invalid parameters: provide either (D, V, ρ, μ) or (D, V, ν).")

```

Fonte: Elaboração própria (2026).

4.2.5.2 Fator de Atrito

No regime laminar, o fator de atrito de Darcy pode ser obtido diretamente por $f = 64/Re$. No regime turbulento, f satisfaz implicitamente a equação de Colebrook–White (COLEBROOK, 1939):

$$\frac{1}{\sqrt{f}} = -2 \log_{10} \left(\frac{\varepsilon}{3,7D_h} + \frac{2,51}{Re\sqrt{f}} \right) \quad (4.23)$$

Sendo ε a rugosidade absoluta. Para evitar a resolução iterativa, existem correlações explícitas aproximadas amplamente usadas, por exemplo:

Swamee–Jain (SWAMEE; JAIN, 1976):

$$f = \frac{0,25}{\left[\log_{10} \left(\frac{\varepsilon}{3,7D} + \frac{5,74}{Re^{0,9}} \right) \right]^2} \quad (4.24)$$

Haaland (HAALAND, 1983):

$$\frac{1}{\sqrt{f}} = -1,8 \log_{10} \left[\left(\frac{\varepsilon/D}{3,7} \right)^{1,11} + \frac{6,9}{Re} \right] \quad (4.25)$$

O *software* implementa as equações acima para cálculo de f conforme o regime e dados disponíveis (WHITE, 2018). A interface para a obtenção do fator de atrito e as opções disponibilizadas ao usuário estão exibidas na Figura 24.

Figura 24 – Interface de cálculo do fator de atrito

Friction Factor

Roughness Type
☒ Material Composition ☐ Custom Value

Material Composition
Commercial steel

Diameter Type
☒ Pipe Schedule ☐ Custom Value

Pipe Schedule
SCH40

Diameter (mm)
15 mm

Reynolds Number
100000

Method
ColebrookWhite

[Calculate Friction Factor](#) [Limpar campos](#)

Property	Value	Units
Diameter	0.0294801	dimensionless

Fonte: Elaboração própria (2026).

Figura 25 – Cálculo do fator de atrito – Parte 1/2

```
def friction_factor(self, parameters: Dict[str, object]):  
    """  
    Darcy friction factor.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"method": "ColebrookWhite" | "SwameeJain" | "Haaland",  
         "roughness": <mm>,  
         "diameter": <mm>,  
         "reynolds": <float>}``  
  
    Returns  
    -----  
    pint.Quantity  
        Darcy friction factor (dimensionless).  
    """  
    if "method" not in parameters:  
        return ["ColebrookWhite", "SwameeJain", "Haaland"]  
  
    req = ["roughness", "diameter", "reynolds", "method"]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, ["roughness", "diameter", "reynolds"])  
  
    eps = parameters["roughness"]  
    D = parameters["diameter"]  
    Re = float(parameters["reynolds"])  
    eps_over_D = eps / D # dimensionless ratio  
  
    method = parameters["method"]  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 26 – Cálculo do fator de atrito – Parte 2/2



Fonte: Elaboração própria (2026).

4.2.5.3 Perda de Carga – Darcy–Weisbach e Hazen–Williams

A perda de carga distribuída h_f em uma tubulação de comprimento L é calculada pela equação de Darcy–Weisbach:

$$h_f = f \cdot \frac{(L + \sum L_{eq}) V^2}{D \cdot 2g} \quad (4.26)$$

Em que $\sum L_{eq}$ é o comprimento equivalente total dos acessórios, em unidades de comprimento, e g é a aceleração da gravidade. Essa fórmula é universal e independe do fluido, desde que f seja adequadamente determinado (PERRY; GREEN, 2007).

Para escoamento de água em tubulações de diâmetro moderado (tipicamente $D > 50$ mm) e regime turbulento plenamente estabelecido, a fórmula empírica de Hazen–

Williams pode ser empregada como simplificação:

$$h_f = \frac{4,73L}{C^{1,852} D^{4,87}} Q^{1,852} \quad (4.27)$$

Em que Q é a vazão volumétrica e C é o coeficiente de Hazen–Williams do material, adimensional e tabelado para tubulações novas de ferro fundido, aço, PVC etc. Apesar de não possuir fundamentação teórica geral, essa equação fornece boa precisão para água a 20 °C em tubulações de transporte e é consagrada no projeto de redes de água devido à sua simplicidade (PERRY; GREEN, 2007; WHITE, 2018).

Figura 27 – Cálculo da perda de carga – Parte 1/3

```
def head_loss(self, parameters: Dict[str, object]):
    """
    Head loss by Darcy-Weisbach or Hazen-Williams.

    Parameters
    -----
    parameters : dict
        **Darcy-Weisbach**

        ``{"method": "Darcy-Weisbach",
         "friction_factor": <float>,
         "pipe_length": <m>,
         "diameter": <mm>,
         "velocity": <m/s>,
         "fittings": [{"fitting": <str>, "quantity": <int>}, ...]}``

        **Hazen-Williams**

        ``{"method": "Hazen-Williams",
         "flow_rate": <m³/s>,
         "roughness_coefficient": <float>,
         "pipe_length": <m>,
         "diameter": <mm>,
         "fittings": [...]} ``

    Returns
    -----
    pint.Quantity
        Head loss (m).
    """
    if "method" not in parameters:
        return ["Darcy-Weisbach", "Hazen-Williams"]

    method = parameters["method"]

    ...
```

Figura 28 – Cálculo da perda de carga – Parte 2/3

```
def head_loss(self, parameters: Dict[str, object]):  
    ...  
  
    # ----- Darcy-Weisbach ----- #  
    if method == "Darcy-Weisbach":  
        req = ["friction_factor", "velocity", "pipe_length", "diameter"]  
        self._require_keys(parameters, req)  
        self._validate_numeric(parameters, req)  
  
        f = parameters["friction_factor"]  
        L = parameters["pipe_length"] * self.ureg.m  
        V = parameters["velocity"] * self.ureg.m / self.ureg.s  
        D = parameters["diameter"] * self.ureg.mm  
        D_m = D.to(self.ureg.m)  
  
        Leq = self._equivalent_length(parameters.get("fittings"), D_m)  
  
        hl = f * (L + Leq) * V**2 / (2 * D_m * self.g)  
        if hl < 0:  
            raise ValueError(f"Head loss cannot be negative, got {hl}.")  
        return hl.to(self.ureg.m)  
  
    ...
```

Fonte: Elaboração própria (2026).

Figura 29 – Cálculo da perda de carga – Parte 3/3

```

def head_loss(self, parameters: Dict[str, object]):
    ...

    # ----- Hazen-Williams ----- #
    if method == "Hazen-Williams":
        req = ["flow_rate", "roughness_coefficient", "diameter", "pipe_length"]
        self._require_keys(parameters, req)
        self._validate_numeric(parameters, req)

        Q = parameters["flow_rate"] * self.ureg.m**3 / self.ureg.s
        C = parameters["roughness_coefficient"]
        L = parameters["pipe_length"] * self.ureg.m
        D = parameters["diameter"] * self.ureg.mm

        if D.magnitude < 50:
            raise ValueError("For Hazen-Williams the diameter must exceed 50
mm.")

        D_m = D.to(self.ureg.m)
        Leq = self._equivalent_length(parameters.get("fittings"), D_m)

        hl = (
            10.67 * (L + Leq) * Q**1.85 / (C**1.85 * D_m**4.87)
        ) * self.ureg.s**1.85 / self.ureg.m**0.68
        if hl <= 0:
            raise ValueError(f"Head loss must be positive, got {hl}.")
        return hl

    raise ValueError('Invalid method. Use "Darcy-Weisbach" or "Hazen-Williams".')

```

Fonte: Elaboração própria (2026).

4.2.5.4 Altura Manométrica entre Dois Pontos

A forma estendida da equação de Bernoulli é utilizada para avaliar as variações de carga (energia por unidade de peso do fluido) entre dois pontos de uma linha de escoamento, incluindo perdas:

$$H = \frac{P_2 - P_1}{\rho g} + \frac{V_2^2 - V_1^2}{2g} + z_2 - z_1 \quad (4.28)$$

Em que H é a altura manométrica adicionada (se positiva) ou removida (se negativa) entre os pontos 1 e 2. Nessa equação, $\frac{P_2 - P_1}{\rho g}$ representa a variação de pressão estática convertida em altura de coluna de fluido, $z_2 - z_1$ é a variação de cota geométrica, o termo de velocidades representa a variação de energia cinética e h_f engloba todas as perdas de carga (distribuídas e localizadas) entre os pontos (WHITE, 2018).

Essa equação é fundamental para avaliar a necessidade de bombeamento: uma bomba deve fornecer uma H suficiente para superar as perdas e elevar o fluido da condição 1 até 2; inversamente, uma turbomáquina geradora extrairá essa energia do fluido.

Figura 30 – Cálculo do *head* – Parte 1/2

```
def head(self, parameters: Dict[str, object]):  
    """  
    Calculate the manometric head between two points in a pipe system.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"pressure1": <Pa>,  
          "pressure2": <Pa>,  
          "elevation1": <m>,  
          "elevation2": <m>,  
          "velocity1": <m/s>,  
          "velocity2": <m/s>,  
          "specific_mass": <kg/m3          "friction_factor": <m>}```  
  
    Returns  
    -----  
    pint.Quantity  
        Manometric head (m).  
    """  
    req = [  
        "pressure1",  
        "pressure2",  
        "elevation1",  
        "elevation2",  
        "velocity1",  
        "velocity2",  
        "specific_mass",  
        "friction_factor"  
    ]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, req)  
    ...
```

Fonte: Elaboração própria (2026).

Figura 31 – Cálculo do *head* – Parte 2/2

```

def head(self, parameters: Dict[str, object]):
    ...

    # Get parameters with proper units
    p1 = parameters["pressure1"] * self.ureg.Pa
    p2 = parameters["pressure2"] * self.ureg.Pa
    z1 = parameters["elevation1"] * self.ureg.m
    z2 = parameters["elevation2"] * self.ureg.m
    v1 = parameters["velocity1"] * self.ureg.m / self.ureg.s
    v2 = parameters["velocity2"] * self.ureg.m / self.ureg.s
    specific_mass = parameters["specific_mass"] * self.ureg.kg / self.ureg.m**3
    friction_factor = parameters["friction_factor"] * self.ureg.m

    # Equation terms
    pressure_term = (p2 - p1) / (specific_mass * self.g)
    elevation_term = (z2 - z1)
    velocity_term = (v2**2 - v1**2) / (2 * self.g)

    # Head calculation: (p2-p1)/specific_mass*g + (z2-z1) + (v2-v1)/(2g) +
    # friction_factor
    head = pressure_term + elevation_term + velocity_term + friction_factor

    # Return in meters
    return head.to(self.ureg.m)

```

Fonte: Elaboração própria (2026).

4.2.5.5 *NPSH* Disponível

Para evitar cavitação em bombas, calcula-se o *NPSH* disponível na sucção, que deve exceder o *NPSH* requerido pelo fabricante. A expressão para o *NPSH* disponível, em termos de altura de líquido, é:

$$NPSH_{disp} = \frac{P_{suc} + P_{atm} - p_{vap}}{\rho g} + \frac{V_{suc}^2}{2g} + (z_{suc} - z_0) - h_{suc} \quad (4.29)$$

Sendo P_{suc} a pressão manométrica na linha de sucção da bomba, P_{atm} a pressão atmosférica, z_{suc} a elevação do ponto de sucção em relação à referência, v_{suc} a velocidade do fluido na sucção, h_{suc} as perdas de carga na linha de sucção e p_{vap} a pressão de vapor do líquido na temperatura de bombeamento (PERRY; GREEN, 2007).

O cálculo de *NPSH* disponível permite verificar se há margem de segurança sobre o *NPSH* requerido da bomba (fornecido pelo fabricante para aquela vazão), garantindo que o líquido não vaporize na entrada da bomba – o que causaria cavitação, danos e redução de desempenho.

Figura 32 – Cálculo do *NPSH* – Parte 1/2

```
def npsht_available(self, parameters: Dict[str, object]):  
    """  
    Calculate available NPSH.  
  
    Parameters  
    -----  
    parameters : dict  
        ``{"manometric_pressure": <kgf/cm2>,  
         "atmospheric_pressure": <kgf/cm2>,  
         "vapor_pressure": <kgf/cm2>,  
         "specific_mass": <kg/m3>,  
         "friction_factor": <m>,  
         "pump_inlet_velocity": <m/s>,  
         "gauge_elevation": <m>}``  
  
    Returns  
    -----  
    pint.Quantity  
        Available NPSH (m).  
    """  
    req = [  
        "manometric_pressure",  
        "atmospheric_pressure",  
        "vapor_pressure",  
        "specific_mass",  
        "friction_factor",  
        "pump_inlet_velocity"  
    ]  
    self._require_keys(parameters, req)  
    self._validate_numeric(parameters, req)  
    ...
```

Fonte: Elaboração própria (2026).

Figura 33 – Cálculo do *NPSH* – Parte 2/2

```

def npsh_available(self, parameters: Dict[str, object]):
    ...

    # Get parameters with proper units
    Ps = parameters["manometric_pressure"] * self.ureg.kgf / self.ureg.cm**2
    Patm = parameters["atmospheric_pressure"] * self.ureg.kgf / self.ureg.cm**2
    Pv = parameters["vapor_pressure"] * self.ureg.kgf / self.ureg.cm**2
    specific_mass = parameters["specific_mass"] * self.ureg.kg / self.ureg.m**3
    friction_factor = parameters["friction_factor"] * self.ureg.m
    pump_inlet_velocity = parameters["pump_inlet_velocity"] * self.ureg.m /
self.ureg.s
    height_term = parameters["gauge_elevation"] * self.ureg.m

    # Convert pressures
    # Convert to N/m²
    Ps_Pa = Ps.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)
    Patm_Pa = Patm.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)
    Pv_Pa = Pv.to(self.ureg.kgf / self.ureg.cm**2).to(self.ureg.Pa)

    # Equation terms
    pressure_term = (Ps_Pa + Patm_Pa) / (specific_mass * self.g)
    vapor_pressure_term = Pv_Pa / (specific_mass * self.g)
    velocity_term = (pump_inlet_velocity**2) / (2 * self.g)

    # New equation: (Pmanometric+Patm)/gamma + V²/(2g) + h - friction_factor -
    Pvap/gamma
    npsh = pressure_term + height_term + velocity_term - friction_factor -
    vapor_pressure_term

    # Return in meters
    return npsh.to(self.ureg.m)

```

Fonte: Elaboração própria (2026).

4.2.5.6 Diâmetro Hidráulico para Geometrias Diversas

O conceito de diâmetro hidráulico generaliza o cálculo do número de Reynolds e do fator de atrito para seções transversais não circulares. O diâmetro hidráulico é definido como:

$$D_h = \frac{4A}{P} \quad (4.30)$$

onde A é a área da seção transversal de escoamento e P o perímetro molhado (contorno em contato com o fluido). Assim:

Para dutos circulares, conforme a Equação 4.31:

$$D_h = D \quad (4.31)$$

(recupera o diâmetro interno). Para canais ou abertos, conforme a Equação 4.32:

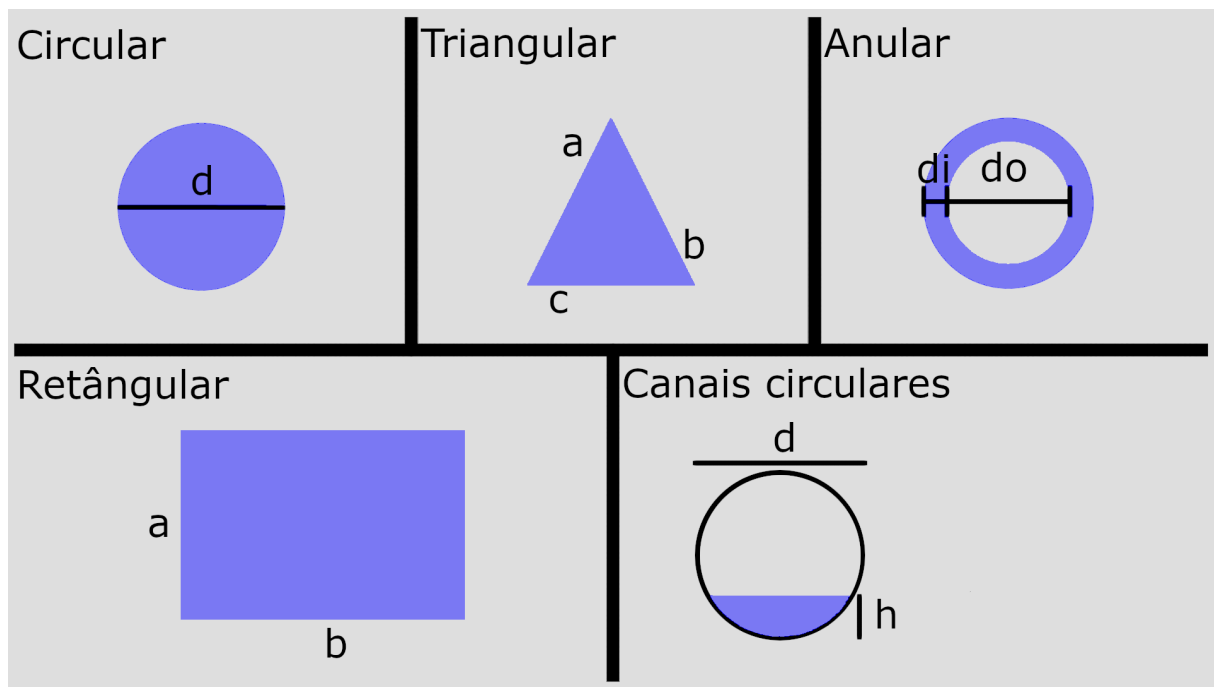
$$D_h = \frac{4ab}{2(a+b)} \quad (4.32)$$

onde a e b são os lados do retângulo. Para anulares (espaço entre dois tubos concêntricos), conforme a Equação 4.33:

$$D_h = D_o - D_i \quad (4.33)$$

(diferença entre diâmetro externo interno e diâmetro externo do interno). Para seções triangulares (por ex., condutos abertos triangulares), calcula-se A e P conforme a geometria e aplica-se $4A/P$. Para geometrias mais complexas (ex.: calhas semicirculares), também se aplica $4A/P$ considerando o perímetro molhado adequadamente.

Figura 34 – Tipos de dutos de escoamento



Fonte: Elaboração própria (2026).

O programa permite ao usuário informar A e P para cálculo de D_h em situações especiais, garantindo flexibilidade para tratar geometrias arbitrárias (WHITE, 2018).

Figura 35 – Exemplo da identificação e critérios de diâmetro hidráulico

```

def hydraulic_diameter(self, parameters: Dict[str, object]):
    """
    Calculate the hydraulic diameter for different geometric shapes.

    Parameters
    -----
    parameters : dict
        ``{"shape": <str>, ...shape-specific parameters...}``
        **Circular**
        ``{"shape": "circular", "diameter": <mm>}``
        **Rectangular**
        ``{"shape": "rectangular", "width": <mm>, "height": <mm>}``
        **Annular**
        ``{"shape": "annular", "outer_diameter": <mm>, "inner_diameter": <mm>}``
        **Triangular**
        ``{"shape": "triangular", "side_a": <mm>, "side_b": <mm>, "side_c": <mm>}``
        **Circular Cap**
        ``{"shape": "circularCap", "diameter": <mm>, "height": <mm>}``

    Returns
    -----
    pint.Quantity
        Hydraulic diameter (mm).
    """
    if "shape" not in parameters:
        return ["circular", "rectangular", "annular", "triangular",
                "circularCap"]

    shape = parameters["shape"]

    if shape == "circular": ...
    elif shape == "rectangular": ...
    elif shape == "annular": ...
    elif shape == "triangular": ...
    elif shape == "circularCap": ...

```

Fonte: Elaboração própria (2026).

4.2.6 Propriedades Termofísicas

Propriedades termofísicas – como densidade, viscosidade, calor específico, condutividade térmica, entalpia e entropia – são insumos essenciais em praticamente todos os cálculos de processos. Em misturas, a predição de propriedades requer modelos mais complexos do que simples somas ponderadas, considerando interações moleculares e desvios da idealidade. Tais propriedades alimentam balanços de massa e energia, o dimensionamento térmico de trocadores, o cálculo de Reynolds e de fatores de atrito, o desempenho de separações, entre outros. Temperatura e pressão são variáveis determinantes: utilizam-se correlações empíricas (por exemplo, equação de Antoine para pressão de vapor) ou equações de estado (gás ideal, Peng-Robinson, SRK) conforme o sistema. Erros na estimativa de viscosidade ou densidade afetam o cálculo de Reynolds, fator de atrito e potência de bombeamento;

imprecisões em calores específicos ou entalpias propagam-se em balanços de energia e projeto de utilidades.

Figura 36 – Obtenção de propriedades pela biblioteca *CoolProp*

```
def get_property(self, fluid, property_name, temperature, pressure):
    """
    Returns a specific property for a fluid at given temperature and pressure

    Parameters:
    -----
    fluid : str
        Name of the fluid
    property_name : str
        Name of the property to retrieve (CoolProp format)
    temperature : float
        Temperature in K
    pressure : float
        Pressure in Pa

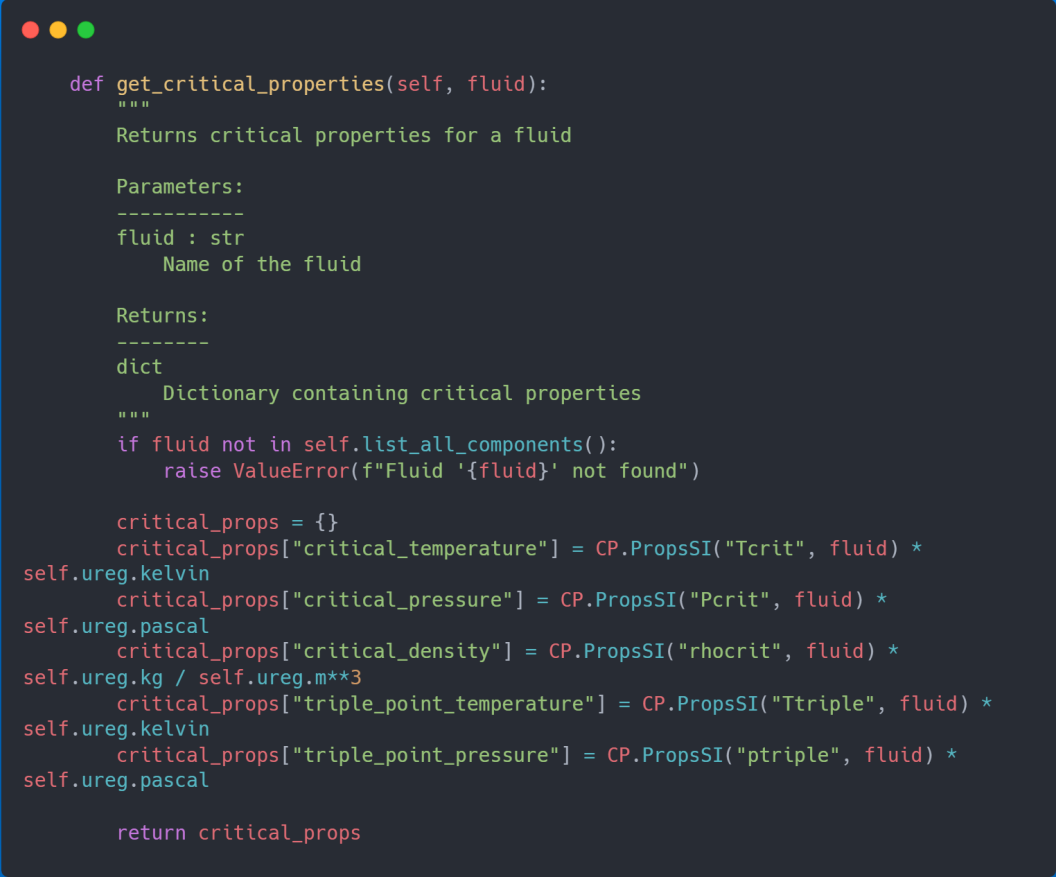
    Returns:
    -----
    float with units
        The requested property with appropriate units
    """
    if fluid not in self.list_all_components():
        raise ValueError(f"Fluid '{fluid}' not found")

    # Map of properties to their units
    units_map = {
        "D": self.ureg.kg / self.ureg.m**3,          # Density
        "C": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Specific
        "V": self.ureg.pascal * self.ureg.second,    # Viscosity
        "L": self.ureg.watt / (self.ureg.meter * self.ureg.kelvin), #
        "H": self.ureg.joule / self.ureg.kg,          # Enthalpy
        "S": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Entropy
        "M": self.ureg.kg / self.ureg.mol,           # Molecular weight
        "I": self.ureg.newton / self.ureg.meter,      # Surface tension
        "P": self.ureg.pascal,                        # Pressure
        "T": self.ureg.kelvin,                        # Temperature
        "T_bubble": self.ureg.kelvin,                # Bubble point temperature
        "T_dew": self.ureg.kelvin,                   # Dew point temperature
        "P_bubble": self.ureg.pascal,                 # Bubble point pressure
        "P_dew": self.ureg.pascal                    # Dew point pressure
    }

    # Check if the fluid is pure (not a mixture)
    # For bubble and dew points, we need a mixture
    if property_name in ["T_bubble", "T_dew", "P_bubble", "P_dew"]:
        # For pure fluids, return the saturation temperature/pressure
        try:
            if property_name == "T_bubble" or property_name == "T_dew":
                # Saturation temperature at given pressure
                value = CP.PropsSI("T", "P", pressure, "Q", 0, fluid)
            elif property_name == "P_bubble" or property_name == "P_dew":
                # Saturation pressure at given temperature
                value = CP.PropsSI("P", "T", temperature, "Q", 0, fluid)
        except Exception as e:
            return None
    else:
        # For other properties, use standard method
        try:
            value = CP.PropsSI(property_name, "T", temperature, "P", pressure,
                                fluid)
        except Exception as e:
            raise ValueError(f"CoolProp error for property '{property_name}' of fluid '{fluid}' at T={temperature}K, P={pressure}Pa: {str(e)}")

    if property_name in units_map:
        return value * units_map[property_name]
    else:
        return value # Return without units if not in the map
```

Fonte: Elaboração própria (2026).

Figura 37 – Obtenção de propriedades críticas pela biblioteca *CoolProp*

```
def get_critical_properties(self, fluid):  
    """  
    Returns critical properties for a fluid  
  
    Parameters:  
    -----  
    fluid : str  
        Name of the fluid  
  
    Returns:  
    -----  
    dict  
        Dictionary containing critical properties  
    """  
    if fluid not in self.list_all_components():  
        raise ValueError(f"Fluid '{fluid}' not found")  
  
    critical_props = {}  
    critical_props["critical_temperature"] = CP.PropsSI("Tcrit", fluid) *  
self.ureg.kelvin  
    critical_props["critical_pressure"] = CP.PropsSI("Pcrit", fluid) *  
self.ureg.pascal  
    critical_props["critical_density"] = CP.PropsSI("rhocrit", fluid) *  
self.ureg.kg / self.ureg.m**3  
    critical_props["triple_point_temperature"] = CP.PropsSI("Ttriple", fluid) *  
self.ureg.kelvin  
    critical_props["triple_point_pressure"] = CP.PropsSI("ptriple", fluid) *  
self.ureg.pascal  
  
    return critical_props
```

Fonte: Elaboração própria (2026).

No *software* desenvolvido, um módulo de propriedades utiliza a biblioteca *CoolProp* para recuperar propriedades de fluidos puros (e misturas) a partir de dados de entrada de T e P . São obtidas propriedades como densidade, viscosidade, calor específico (c_p e c_v), condutividade térmica, entalpia, entropia, massa molar e tensão superficial, além de propriedades críticas e pontos de saturação (bolha/orvalho) quando aplicável. A visualização desse cálculo de propriedades, com foco no resultado para uma mistura, é apresentada na Figura 38.

Figura 38 – Interface de cálculo para as propriedades de uma mistura

Mixture Properties

Temperature (K)

300

Pressure (Pa)

101325

Fluid Fractions

Water

0,5

Methanol

0,5

+ Add Fluid

Properties to Calculate

Density

Specific heat

Leave empty to calculate all available properties

Calculate Mixture Properties

Mixture Properties

Fluid Composition:

Water: 0.5

Methanol: 0.5

Property	Value	Units
Density	753.601	kilogram / meter ** 3
Specific Heat	3474.14	joule / kelvin / kilogram

Fonte: Elaboração própria (2026).

Para misturas, a *API* do *CoolProp* permite montar *strings* com o padrão abaixo, usando as frações especificadas para cada componente (BELL et al., 2014):

HEOS : : Comp1 [x1] &Comp2 [x2] & . . .

Todas as grandezas retornam já associadas a unidades do SI (via integração com a biblioteca *Pint*), garantindo consistência nos cálculos.

Figura 39 – Obtenção de propriedades de misturas pela biblioteca *CoolProp* – Parte 1/2

```

def get_mixture_properties(self, fluid_fractions, temperature, pressure,
properties=None):
    """
    Returns properties for a mixture of fluids at given temperature and pressure

    Parameters:
    -----
    fluid_fractions : dict
        Dictionary with fluid names as keys and their mass fractions as values
        Example: {"Water": 0.7, "Ethanol": 0.3} for a mixture of 70% water and
30% ethanol
    temperature : float
        Temperature in K
    pressure : float
        Pressure in Pa
    properties : list, optional
        List of property keys to retrieve. If None, returns all available
properties.

    Returns:
    -----
    dict
        Dictionary containing the requested properties for the mixture
    """
    # Validate inputs
    for fluid in fluid_fractions.keys():
        if fluid not in self.list_all_components():
            raise ValueError(f"Fluid '{fluid}' not found")

    # Check that fractions sum to approximately 1
    total_fraction = sum(fluid_fractions.values())
    if not 0.99 <= total_fraction <= 1.01:
        raise ValueError(f"Sum of fluid fractions should be 1.0, got
{total_fraction}")

    # Create the mixture string for CoolProp
    # Format: "HEOS::fluid1[fraction1]&fluid2[fraction2]&..."
    mixture_string = "HEOS::"
    for i, (fluid, fraction) in enumerate(fluid_fractions.items()):
        if i > 0:
            mixture_string += "&"
            mixture_string += f"{fluid}[{fraction}]"

    # Get properties
    result = {}

    # Define which properties to retrieve
    if properties is None:
        properties = ["D", "C", "V", "L", "H", "S", "M", "I", "P", "T"]

    ...

```

Fonte: Elaboração própria (2026).

Figura 40 – Obtenção de propriedades de misturas pela biblioteca *CoolProp* – Parte 2/2

```

def get_mixture_properties(self, fluid_fractions, temperature, pressure,
properties=None):
    ...

    # Map of property keys to result dictionary keys
    property_key_map = {
        "D": "density",
        "C": "specific_heat",
        "V": "viscosity",
        "L": "thermal_conductivity",
        "H": "enthalpy",
        "S": "entropy",
        "M": "molecular_weight",
        "I": "surface_tension",
        "P": "pressure",
        "T": "temperature"
    }

    # Map of properties to their units (same as in get_property method)
    units_map = {
        "D": self.ureg.kg / self.ureg.m**3,          # Density
        "C": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Specific
heat
        "V": self.ureg.pascal * self.ureg.second,    # Viscosity
        "L": self.ureg.watt / (self.ureg.meter * self.ureg.kelvin), #
Conductivity
        "H": self.ureg.joule / self.ureg.kg,          # Enthalpy
        "S": self.ureg.joule / (self.ureg.kg * self.ureg.kelvin), # Entropy
        "M": self.ureg.kg / self.ureg.mol,            # Molecular weight
        "I": self.ureg.newton / self.ureg.meter,       # Surface tension
        "P": self.ureg.pascal,                        # Pressure
        "T": self.ureg.kelvin                         # Temperature
    }

    # Retrieve each property
    for prop in properties:
        try:
            # For mixtures or other properties
            value = CP.PropsSI(prop, "T", temperature, "P", pressure,
mixture_string)

            if prop in units_map:
                result[property_key_map.get(prop, prop)] = value *
units_map[prop]
            else:
                result[property_key_map.get(prop, prop)] = value
        except Exception as e:
            raise ValueError(f"CoolProp mixture error for property '{prop}':
{str(e)}")

    return result

```

Fonte: Elaboração própria (2026).

O uso de uma biblioteca de propriedades de qualidade referência como o *CoolProp* assegura que o *software* tenha um *backend* termodinâmico rigoroso e abrangente, reforçando o caráter didático ao permitir explorar diferentes cenários com fidelidade física.

4.3 API

Com o objetivo de ampliar os casos de uso da ferramenta desenvolvida, a integração entre as camadas de *frontend* e *backend* é realizada por meio de uma *API* pública, permitindo que sistemas externos também possam consumir as funcionalidades disponibilizadas. Essa abordagem favorece a interoperabilidade do *software* e possibilita sua aplicação em diferentes contextos computacionais, como simulações, automações de processos e integração com outras plataformas utilizadas na engenharia química.

Para o consumo da *API*, a documentação encontra-se disponível no *endpoint /docs* do *backend*. Essa documentação foi desenvolvida utilizando o *framework FastAPI*, que possibilita a geração automática e padronizada da interface de documentação por meio da especificação *OpenAPI*, facilitando a compreensão, o teste e a integração dos *endpoints* disponibilizados.

No *frontend*, o consumo dessa mesma *API* é feito por um cliente tipado em *TypeScript*, centralizado em uma camada única de serviços e exposto à interface por meio de chamadas ao caminho */api*, preservando a mesma origem no ambiente de desenvolvimento e no ambiente publicado. Dessa forma, a interface desenvolvida com *React* e *Vite* mantém a separação entre apresentação e dados, enquanto os módulos de tubulações, escoamento, dimensionamento, bombas, propriedades, reatores e balanço de massa acessam os mesmos *endpoints* sem duplicar regras de negócio.

Um aspecto importante do estado atual do sistema é que a *API* não entrega apenas resultados escalares. Em vários módulos, o *backend* também monta estruturas de dados prontas para visualização, como modelos de gráfico com eixos, séries, marcadores e metadados de unidades. Esse padrão é empregado, por exemplo, no diagrama de *Moody*, nas curvas de perda de carga e altura manométrica, no indicador de margem de *NPSH*, no diagrama de *Levenspiel*, no gráfico de *Arrhenius*, nos perfis espaciais e de reciclo em *PFR*, no diagrama ternário, no diagrama de *McCabe-Thiele*, no envelope de fase, na curva de pressão de vapor e no gráfico nativo do balanço de massa.

Esse desenho arquitetural desloca para o *backend* a responsabilidade por cálculos, amostragem numérica, construção de séries e definição dos pontos operacionais que precisam ser destacados. Ao *frontend* cabe interpretar tais contratos tipados e renderizá-los em componentes *SVG* reutilizáveis, preservando coerência visual entre páginas distintas. O mesmo princípio é adotado nos *endpoints* de exemplo: a *API* fornece entradas de referência coerentes com os catálogos disponíveis, e a interface apenas pré-preenche os formulários e dispara o fluxo normal de cálculo, sem manter resultados estáticos desacoplados do motor computacional.

4.4 Validação de Cálculos e Especificação de Retornos

Dentre as etapas fundamentais no desenvolvimento de *softwares*, a fase de testagem apresenta papel de grande relevância, uma vez que possibilita a verificação sistemática da conformidade dos resultados obtidos com aqueles previstos teoricamente. No contexto deste trabalho, essa etapa é essencial para garantir a confiabilidade dos cálculos e das rotinas computacionais empregadas em aplicações de Engenharia Química.

Uma abordagem eficiente para a realização desses testes consiste na utilização da biblioteca *pytest*, que permite a implementação de testes automatizados desenvolvidos para assegurar o correto funcionamento dos módulos do sistema. Por meio dessa ferramenta, é possível validar, de forma reproduzível e padronizada, os retornos das funções responsáveis pelos cálculos de propriedades, balanços e demais operações recorrentes da área.

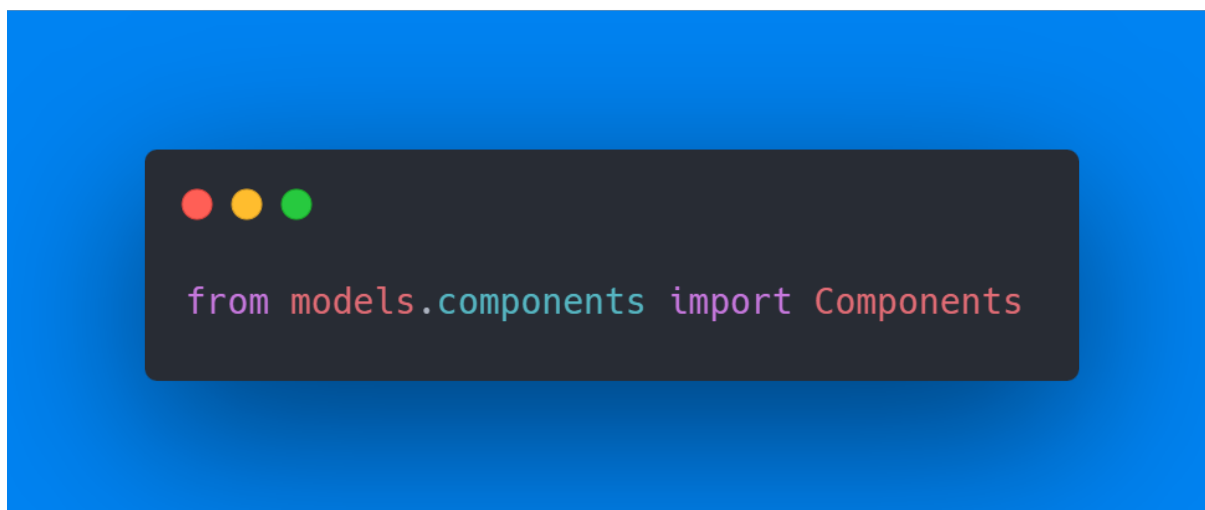
Com o objetivo de detalhar o funcionamento do *software* desenvolvido, as funcionalidades de cada módulo são descritas a seguir, acompanhadas de evidências da interface *frontend* e de instruções para o consumo direto das rotinas por meio da linguagem *Python*, permitindo tanto a utilização via interface gráfica quanto a integração do sistema com outros projetos e aplicações computacionais.

4.4.1 Componentes

Para iniciar a biblioteca local com o módulo de componentes, o código da figura abaixo deve ser inserido, assegurando que o *Python* consiga identificar as funções dentro da classe *Components* contida no arquivo *./models/componentes.py*.

Tendo que o passo a passo para consumir via *API* ou *Python* o projeto seja demasiado técnico, será apenas brevemente comentado abaixo o modo de se fazer, depois havendo maior realce apenas aos retornos e comentários correlatos.

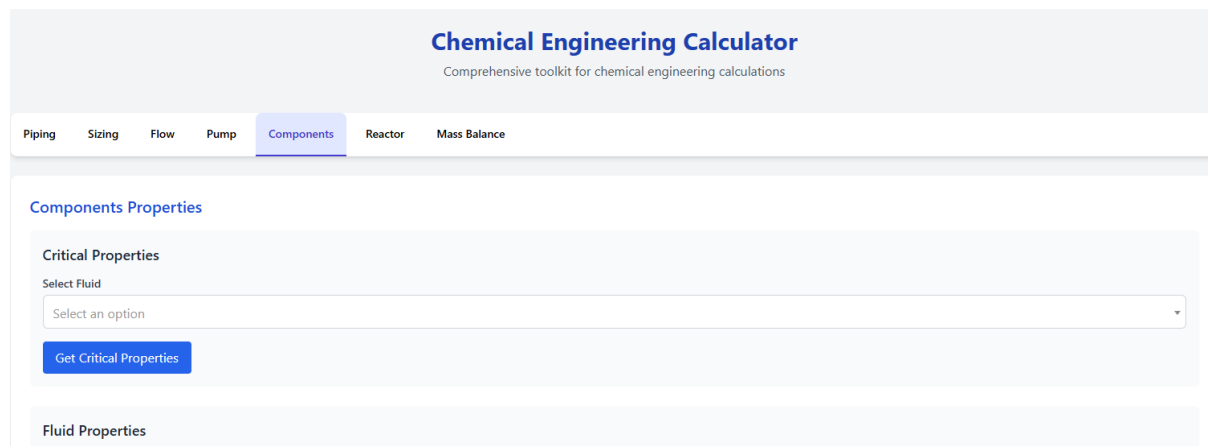
Figura 41 – Inicialização da biblioteca de componentes via *Python*



Fonte: Elaboração própria (2026).

No *frontend*, basta acessar a aba *Components* e rolar a página para visualizar as funcionalidades.

Figura 42 – Inicialização da biblioteca de componentes via *frontend*

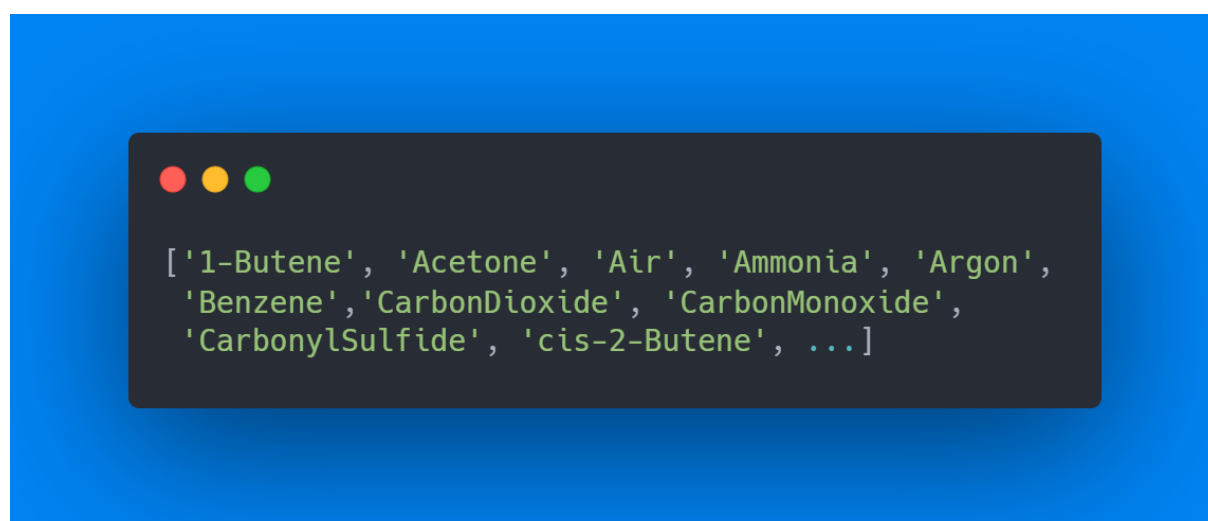


Fonte: Elaboração própria (2026).

4.4.1.1 Listar Componentes

A função `list_all_components()` interage diretamente com a biblioteca termodinâmica *CoolProp* para listar todas as substâncias puras e pseudo-puras disponíveis para cálculo. O teste retornou um total de 124 fluidos cadastrados, abrangendo desde hidrocarbonetos simples e refrigerantes até fluidos inorgânicos como água e amônia. Esta listagem valida a capacidade do sistema em operar com uma vasta gama de componentes químicos industriais.

Figura 43 – Listagem parcial de componentes disponíveis na biblioteca termodinâmica



Fonte: Elaboração própria (2026).

4.4.1.2 Valor de Propriedade

Ao solicitar as propriedades termofísicas para o componente *Water* (água) nas condições de 300 K e 101325 Pa (1 atm), o sistema retornou valores consistentes com a literatura. A massa específica calculada foi de 996,56 kg/m³ e a viscosidade de $8,54 \cdot 10^{-4}$ Pa · s. Nota-se que o retorno inclui explicitamente as unidades físicas, utilizando a biblioteca *Pint*, garantindo a consistência dimensional dos cálculos subsequentes.

Figura 44 – Propriedades termofísicas da água calculadas a 300K e 101,3kPa.

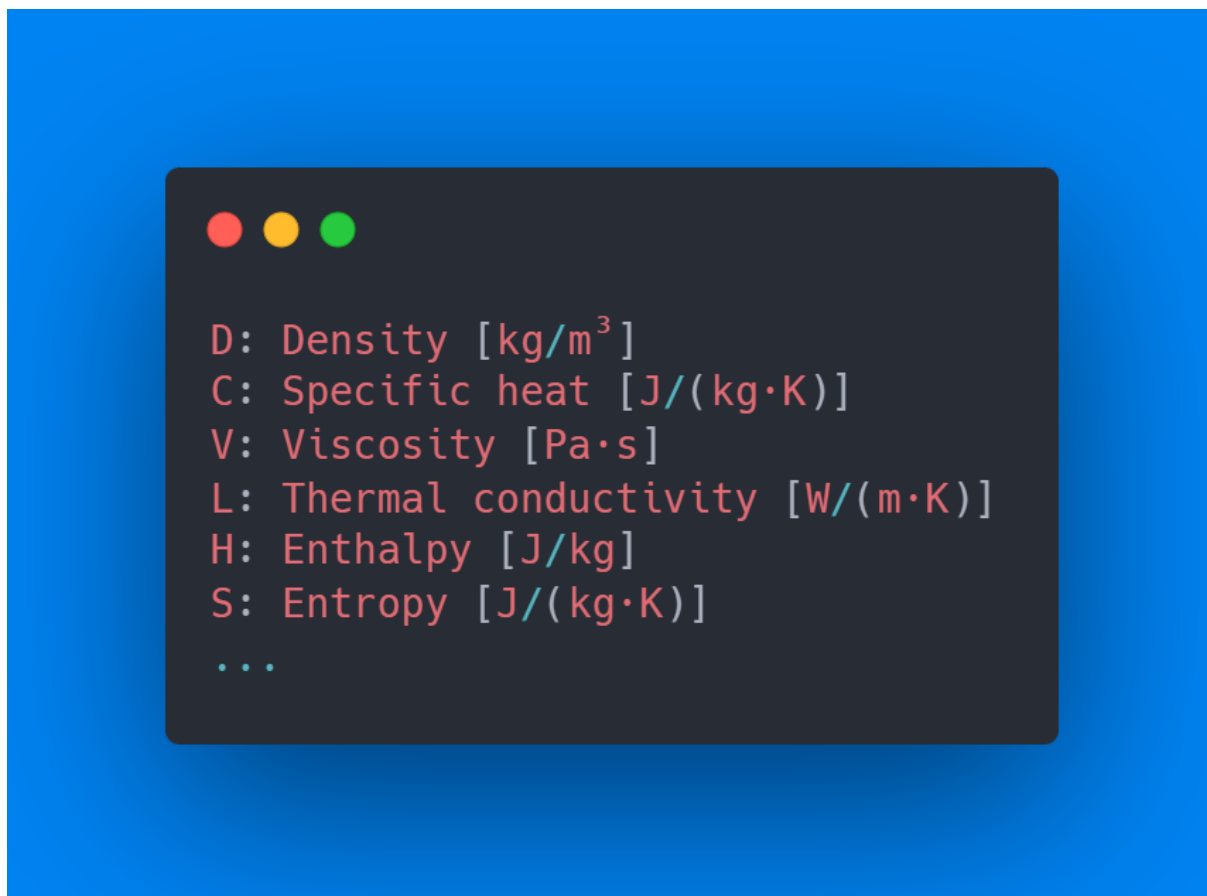


Fonte: Elaboração própria (2026).

4.4.1.3 Nome de Propriedades

A função `get_property_names()` expõe o mapeamento interno entre as chaves de solicitação da *API* (como 'D', 'H', 'V') e suas respectivas descrições físicas e unidades no SI. Esta estrutura organizacional permite que o usuário identifique rapidamente quais variáveis termodinâmicas podem ser requisitadas ao modelo, facilitando a configuração de simulações de balanço de massa e energia.

Figura 45 – Mapeamento de chaves de propriedades e suas unidades padronizadas.

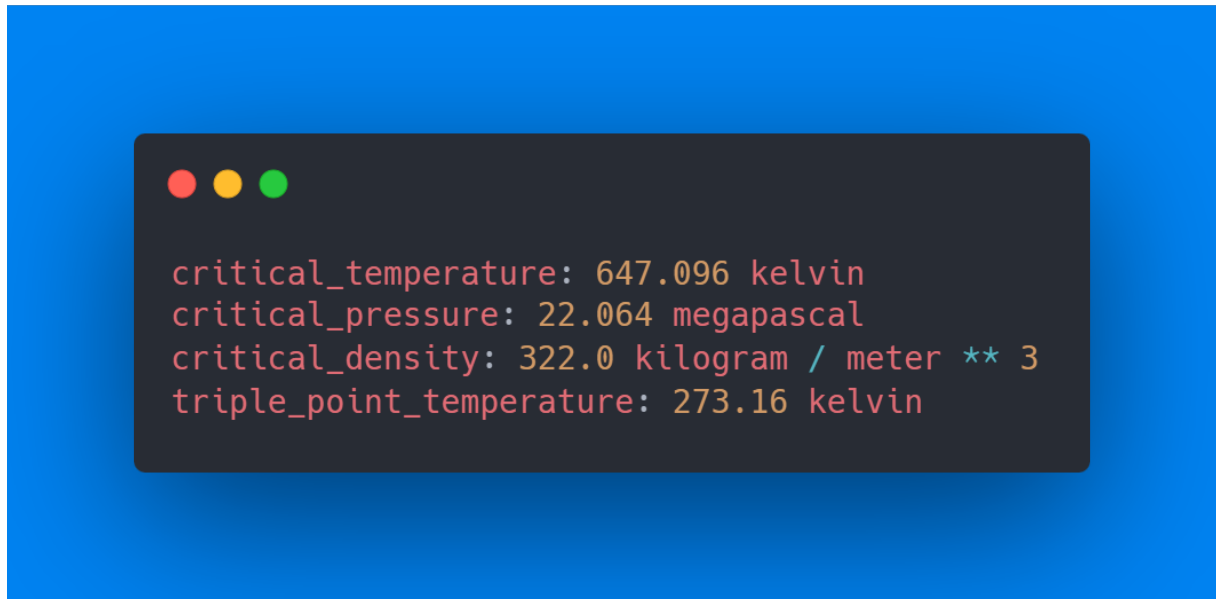


Fonte: Elaboração própria (2026).

4.4.1.4 Propriedades Críticas

A consulta das propriedades críticas para a água retornou uma temperatura crítica (T_c) de 647,1 K e uma pressão crítica (P_c) de 22,06 MPa. Esses parâmetros fixos são fundamentais para o cálculo de propriedades reduzidas e para a verificação dos limites de fase da substância nas equações de estado. O sistema também forneceu os dados do ponto triplo, expandindo a utilidade da ferramenta para análises de equilíbrio de fases.

Figura 46 – Propriedades críticas e do ponto triplo calculadas para a água.



Fonte: Elaboração própria (2026).

4.4.1.5 Opções de Propriedades de Misturas

Para sistemas multicomponentes, a função `get_property_mixture_names()` delimita o subconjunto de propriedades que podem ser calculadas. Diferentemente das substâncias puras, observa-se que certas propriedades de transporte específicas ou tensão superficial podem não estar disponíveis dependendo do modelo de mistura (*HEOS*) empregado. A listagem confirma a disponibilidade das variáveis de estado essenciais (entalpia, entropia, densidade) necessárias para o dimensionamento de reatores e tubulações.

Figura 47 – Propriedades termodinâmicas disponíveis para cálculo em misturas.



Fonte: Elaboração própria (2026).

4.4.1.6 Valor de Propriedades de Misturas

A simulação de uma mistura binária equimolar de Água (50%) e Etanol (50%) a $300K$ e $1atm$ resultou em uma densidade de mistura de $814,04 \text{ kg/m}^3$. O código utiliza internamente a regra de mistura baseada na energia livre de Helmholtz (HEOS) do *CoolProp* para ponderar as interações moleculares entre os componentes. Este valor intermediário entre a densidade da água pura (997 kg/m^3) e do etanol puro (789 kg/m^3) demonstra a eficácia do algoritmo em prever o comportamento não-ideal da solução.

Figura 48 – Propriedades termodinâmicas disponíveis para cálculo em misturas.



Fonte: Elaboração própria (2026).

4.4.2 Hidráulicos

4.4.2.1 Perda de Carga

Para a validação do cálculo de perda de carga, utilizou-se o método de Darcy–Weisbach, aplicável a fluxos incompressíveis em qualquer regime de escoamento. A equação base para o cálculo é a Equação 4.26.

Onde a perda de carga localizada ($\sum h_{local}$) é calculada através do comprimento equivalente (L_{eq}) dos acessórios:

$$L_{total} = L + \sum (L_{eq} \cdot D) \quad (4.34)$$

Teste A: Tubulação Reta (Sem Acessórios)

- **Dados:** $L = 100m$, $D = 50mm(0,05m)$, $V = 2,0m/s$, $f = 0,02$
- **Cálculo Manual:**

$$h_f = 0,02 \cdot \frac{100}{0,05} \cdot \frac{2^2}{2 \cdot 9,80665} = 40 \cdot 0,2039 = 8,1577 \text{ m} \quad (4.35)$$

- **Software:** 8,1577 m
- **Status:** [MATCH] (Desvio: 0,00%)

Teste B: Com Acessórios (2 Curvas 90º Raio Longo)

- **Dados Adicionais:** 2 Cotovelos ($L_{eq}/D = 16$).
- **Comprimento Equivalente:** $L_{eq} = 2 \cdot (16 \cdot 0,05) = 1,6m$

- **Comprimento Total:** 101,6m
- **Cálculo Manual:**

$$h_f = 0,02 \cdot \frac{101,6}{0,05} \cdot \frac{2^2}{2 \cdot 9,80665} = 8,2882 \text{ m} \quad (4.36)$$

- **Software:** 8,2882 m
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.2 Reynolds

O Número de Reynolds (Re) foi validado utilizando as duas formas de entrada suportadas pela *API*: viscosidade dinâmica (μ) e cinemática (ν).

Método 1: Viscosidade Dinâmica (Equação 4.21)

- **Dados:** $\rho = 998 \text{ kg/m}^3$, $V = 2,0 \text{ m/s}$, $D = 0,05 \text{ m}$, $\mu = 1,002 \cdot 10^{-3} \text{ Pa} \cdot \text{s}$
- **Cálculo Manual:** $Re = \frac{998 \cdot 2 \cdot 0,05}{0,001002} = 99.600,8$
- **Software:** 99.600,8
- **Status:** [MATCH] (Desvio: 0,00%)

Método 2: Viscosidade Cinemática (Equação 4.22)

- **Dados:** $V = 2,0 \text{ m/s}$, $D = 0,05 \text{ m}$, $\nu = 1,004 \cdot 10^{-6} \text{ m}^2/\text{s}$
- **Cálculo Manual:** $Re = \frac{2 \cdot 0,05}{1,004 \cdot 10^{-6}} = 99.601,6$
- **Software:** 99.601,6
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.3 Fator de Atrito

O fator de atrito de Darcy (f) foi validado comparando métodos iterativos e explícitos.

A. Consistência de Colebrook-White (COLEBROOK, 1939) Equação implícita (padrão, definida na Equação 4.23): Verificou-se a convergência da solução numérica da *API* comparando o Lado Esquerdo (LHS) e Direito (RHS) da equação.

- **Dados:** $\varepsilon = 0,045 \text{ mm}$, $D = 50 \text{ mm}$, $Re = 100.000$
- **Software (f):** 0,021614
- **Check:** $LHS = 6,802$; $RHS = 6,802$
- **Status:** [CONVERGÊNCIA CONFIRMADA]

B. Equação de Swamee-Jain (Explícita) (SWAMEE; JAIN, 1976) (ver Equação 4.24)

- **Cálculo Manual:** $f = 0,02199$
- **Software:** 0,02199
- **Status:** [MATCH]

C. Equação de Haaland (Explícita) (HAALAND, 1983) (ver Equação 4.25)

- **Cálculo Manual:** $f = 0,02161$
- **Software:** 0,02161
- **Status:** [MATCH]

4.4.2.4 Diâmetro Real

A função de seleção de diâmetro comercial verifica o próximo diâmetro nominal disponível numa tabela de *Schedule*.

- **Entrada:** Diâmetro Calculado = 45,0 mm. *Schedule* = SCH40.
- **Lógica Manual:** Consultar tabela SCH40. Diâmetros disponíveis: ..., 32, 40, 50, 65... O próximo maior que 45 é 50.
- **Software:** 50,0 mm
- **Status:** [MATCH]

4.4.2.5 Diâmetro Calculado

O dimensionamento preliminar baseia-se na equação da continuidade para tubos circulares.

(ver Equação 4.1)

- **Dados:** $Q = 0,005 m^3/s$, $V = 1,5 m/s$
- **Cálculo Manual:** $D = \sqrt{\frac{4 \cdot 0,005}{\pi \cdot 1,5}} = \sqrt{0,004244} = 0,06515 \text{ m}$ (65,15 mm)
- **Software:** 65,15 mm
- **Status:** [MATCH] (Desvio: 0,00%)

4.4.2.6 NPSH Disponível

O cálculo do *Net Positive Suction Head* disponível ($NPSH_a$) utiliza a equação de Bernoulli aplicada à sucção da bomba:

Utilizando a Equação 4.29

- **Dados:** $P_{man} = 0,5 \text{ kgf/cm}^2$, $P_{atm} = 1 \text{ atm}$, $P_{vap} = 0,02 \text{ kgf/cm}^2$, $\gamma = 9806 \text{ N/m}^3$, $h_f = 1 \text{ m}$, $\Delta Z = 2 \text{ m}$
- **Termos de Pressão (m.c.a):** $H_{press} = \frac{(0,5+1,033) \cdot 98.066,5}{9.806,6} = 15,33 \text{ m}$, $H_{vap} = 0,2 \text{ m}$
- **Cálculo Manual:** $15,33 + 2,0 + 0,11 - 1,0 - 0,2 = 16,24 \text{ m}$
- **Software:** $16,24 \text{ m}$
- **Status:** [MATCH] (Desvio: $< 0,01\%$)

4.4.2.7 Head

A carga manométrica total da bomba (H_{man}) é calculada pela diferença de energia entre sucção e descarga:

Utilizando a Equação 4.28

- **Dados:** $\Delta P = 101.325 \text{ Pa}$, $\Delta Z = 10 \text{ m}$, $V_1 = 1 \text{ m/s}$, $V_2 = 2 \text{ m/s}$, $h_f = 5 \text{ m}$
- **Cálculo Manual:** $\frac{101.325}{9806} + 10 + \frac{4-1}{19,6} + 5 = 10,33 + 10 + 0,15 + 5 = 25,48 \text{ m}$
- **Software:** $25,48 \text{ m}$
- **Status:** [MATCH] (Desvio: $0,00\%$)

4.4.2.8 Diâmetro Hidráulico

O cálculo do diâmetro hidráulico (D_h) foi validado para diferentes geometrias, fundamental para cálculos de perda de carga em dutos não circulares.

A relação geral é definida na Equação 4.30.

1. Circular

- **Dados:** $D = 50 \text{ mm}$
- **Manual:** $D_h = D = 50 \text{ mm}$
- **Software:** 50 mm [MATCH]

2. Retangular

- **Dados:** $L = 40mm, H = 30mm$
- **Manual:** $A = 1200, P = 140 \rightarrow D_h = \frac{4800}{140} = 34,29 \text{ mm}$
- **Software:** 34,29mm [MATCH]

3. Anular (Tubo Concêntrico)

- **Dados:** $D_{ext} = 100mm, D_{int} = 50mm$
- **Manual:** $D_h = D_{ext} - D_{int} = 50mm$
- **Software:** 50mm [MATCH]

4. Triangular

- **Dados:** Lados 3, 4, 5 (Triângulo Retângulo). $A = 6, P = 12$.
- **Manual:** $D_h = \frac{4 \cdot 6}{12} = 2,0mm$
- **Software:** 2,0mm [MATCH]

5. Canal Circular (Meia Seção)

- **Dados:** $D = 100mm$, Nível $H = 50mm$ (Metade).
- **Manual:** Para meia seção cheia, $A = \pi R^2/2, P = \pi R$. Logo $D_h = \frac{4(\pi R^2/2)}{\pi R} = 2R = D = 100mm$.
- **Software:** 100,0mm [MATCH]

4.4.3 Balanços de Massa

4.4.3.1 Misturador Ideal

A validação do balanço de massa foi realizada considerando um misturador ideal (Mixer) alimentado por duas correntes puras.

Definição do Caso Teste:

- **Corrente 1 (S1):** 100 kg/h de Água pura ($z_{\text{agua}} = 1,0$).
- **Corrente 2 (S2):** 100 kg/h de Etanol puro ($z_{\text{etanol}} = 1,0$).
- **Operação:** Mistura adiabática sem reação química ($S1 + S2 \rightarrow S3$).

Cálculo Manual:

1. Balanço Global:

$$F_{S3} = F_{S1} + F_{S2} = 100 + 100 = 200 \text{ kg/h} \quad (4.37)$$

2. Balanço por Componente (Água):

$$F_{S3} \cdot z_{S3,agua} = F_{S1} \cdot z_{S1,agua} + F_{S2} \cdot z_{S2,agua} \quad (4.38)$$

$$200 \cdot z_{S3,agua} = 100 \cdot 1,0 + 100 \cdot 0,0 \implies z_{S3,agua} = 0,5 \quad (4.39)$$

Comparação com o *Software*: O código de validação foi executado conforme a Figura 49.

Figura 49 – Código de validação do Balanço de Massa (Misturador)

```
demo_mass_balance.py

import sys
import os

# Add parent directory to path to allow imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from models.mass_balance import MassBalance
from demo.utils import print_header, print_result

def demo_mass_balance():
    print_header("Validação Balanço de Massa - Misturador")

    # Definição do Problema: Mistura de Água e Etanol
    # Corrente 1: 100 kg/h (Água Pura)
    # Corrente 2: 100 kg/h (Etanol Puro)
    # Misturador -> Corrente 3

    mb = MassBalance(components=["Água", "Etanol"])

    # 1. Adicionar Correntes
    # Stream 1: Entrada, 100 kg/h, 100% Água
    mb.add_stream("S1", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 1.0, "Etanol":
0.0})

    # Stream 2: Entrada, 100 kg/h, 100% Etanol
    mb.add_stream("S2", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 0.0, "Etanol":
1.0})

    # Stream 3: Saída
    mb.add_stream("S3", ["Água", "Etanol"], -1)

    print(">>> Dados de Entrada:")
    print("S1: F=100 kg/h, z=[Água: 1.0, Etanol: 0.0]")
    print("S2: F=100 kg/h, z=[Água: 0.0, Etanol: 1.0]")

    # 2. Solver
    results = mb.get_results()

    # 3. Resultados
    print("\n>>> Resultados Computacionais:")
    s3 = results["S3"]
    print(f"S3 Flow Rate: {s3['flow_rate']} kg/h")
    print(f"S3 Compositions: {s3['compositions']}")

    # Validação
    valid, msg = mb.validate_results(results)
    if valid:
        print("\n[VALIDAÇÃO] O balanço de massa fechou corretamente.")
    else:
        print(f"\n[ERRO] Falha na validação: {msg}")

if __name__ == "__main__":
    demo_mass_balance()
```

Tabela 3 – Validação Balanço de Massa (Misturador)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Vazão <i>S3</i> (<i>kg/h</i>)	200,0	200,0	0,00
Fração Água <i>S3</i>	0,50	0,50	0,00
Fração Etanol <i>S3</i>	0,50	0,50	0,00

Fonte: Elaboração própria (2026).

4.4.3.2 Reator de Conversão

Neste caso, avalia-se a geração e consumo de espécies químicas em um sistema reacional simples.

- **Reação:** $A \rightarrow B$ (Estequiometria 1:1).
- **Alimentação:** 100 *mol/s* de A puro.
- **Conversão** (X_A): 40%.

Cálculo Manual: Pela estequiometria, para cada mol de A consumido, gera-se 1 mol de B.

$$F_{A,out} = F_{A,in}(1 - X_A) = 100(1 - 0,40) = 60 \text{ mol/s} \quad (4.40)$$

$$F_{B,out} = F_{A,in}X_A = 100(0,40) = 40 \text{ mol/s} \quad (4.41)$$

A vazão molar total permanece constante ($60 + 40 = 100$), resultando nas frações molares:

$$y_A = \frac{60}{100} = 0,6; \quad y_B = \frac{40}{100} = 0,4 \quad (4.42)$$

Validação: O *software* retornou: $F_{Product} = 100 \text{ mol/s}$, com composição $z_A = 0,6$ e $z_B = 0,4$, confirmando a conservação de massa e estequiometria.

4.4.3.3 Divisor de Corrente (*Splitter*)

Simula-se uma operação de separação física onde uma corrente é dividida em duas (Reciclo e Produto) sem alteração de composição, típica em loops de reciclo.

- **Entrada no Divisor:** 100 *kg/h* (mistura 50% A, 50% B).
- **Razão de Divisão (Reciclo):** 0,2 ou 20%.

Cálculo Manual:

$$F_{Reciclo} = 0,2 \cdot 100 = 20 \text{ kg/h} \quad (4.43)$$

$$F_{Produto} = (1 - 0,2) \cdot 100 = 80 \text{ kg/h} \quad (4.44)$$

A composição deve permanecer inalterada: $z_A = 0,5$; $z_B = 0,5$ em ambas as saídas.

Validação: Conforme apresentado na saída expandida do *software* (Figura 50), os valores de vazão foram exatos (20 e 80 *kg/h*) e a composição manteve-se constante, validando a lógica do divisor.

Figura 50 – Resultados expandidos da validação de Balanço de Massa (Misturador, Reator e *Splitter*)

```
import sys
import os

# Add parent directory to path to allow imports
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from models.mass_balance import MassBalance
from demo.utils import print_header, print_result

def demo_mixer():
    print_header("1. Validação Misturador")
    # Caso: Mistura de Água (S1) e Etanol (S2)
    mb = MassBalance(components=["Água", "Etanol"])
    mb.add_stream("S1", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 1.0, "Etanol": 0.0})
    mb.add_stream("S2", ["Água", "Etanol"], 1, flow_rate=100.0, compositions={"Água": 0.0, "Etanol": 1.0})
    mb.add_stream("S3", ["Água", "Etanol"], -1)

    results = mb.get_results()
    s3 = results["S3"]
    print(f"S1 (In): 100 kg/h [Água: 1.0]")
    print(f"S2 (In): 100 kg/h [Etanol: 1.0]")
    print(f"S3 (Out): Flow={s3['flow_rate']:.2f} kg/h, Comp={s3['compositions']}")

def demo_reactor():
    print_header("2. Validação Reator de Conversão")
    # Caso: Reação A -> B com 40% de conversão
    # Alimentação: 100 mol/s de A puro
    mb = MassBalance(components=["A", "B"])
    mb.add_stream("Feed", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 1.0, "B": 0.0})
    mb.add_stream("Product", ["A", "B"], -1)

    # Reação: A -> B (Estequiometria: -1 A, +1 B)
    # Conversão de A = 0.40
    mb.add_reaction({"A": -1, "B": 1}, key_component="A", conversion=0.40)

    results = mb.get_results()
    prod = results["Product"]
    print(f"Feed (In): 100 mol/s [A: 1.0]")
    print(f"Reaction: A -> B (X=40%)")
    print(f"Product (Out): Flow={prod['flow_rate']:.2f} mol/s, Comp={prod['compositions']}")

def demo_recycle():
    print_header("3. Validação Reciclo com Purga")

    mb = MassBalance(components=["A", "B"])
    mb.add_stream("Feed", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 1.0})

    mb_split = MassBalance(components=["A", "B"])
    mb_split.add_stream("S_Main", ["A", "B"], 1, flow_rate=100.0, compositions={"A": 0.5, "B": 0.5})
    mb_split.add_stream("S_Recycle", ["A", "B"], -1)
    mb_split.add_stream("S_Product", ["A", "B"], -1)

    # Split: 20% Reciclo, 80% Produto
    mb_split.add_split("S_Main", "S_Recycle", "S_Product", fraction=0.2)

    res = mb_split.get_results()
    rec = res["S_Recycle"]
    prod = res["S_Product"]

    print(f"Main (In): 100 kg/h [A:0.5, B:0.5]")
    print(f"Split Fraction: 0.2 (20% Reciclo)")
    print(f"Recycle (Out): Flow={rec['flow_rate']:.2f}, Comp={rec['compositions']}")
    print(f"Product (Out): Flow={prod['flow_rate']:.2f}, Comp={prod['compositions']}")

if __name__ == "__main__":
    demo_mixer()
    print("-" * 30)
    demo_reactor()
    print("-" * 30)
    demo_recycle()
```

Fonte: Elaboração própria (2026).

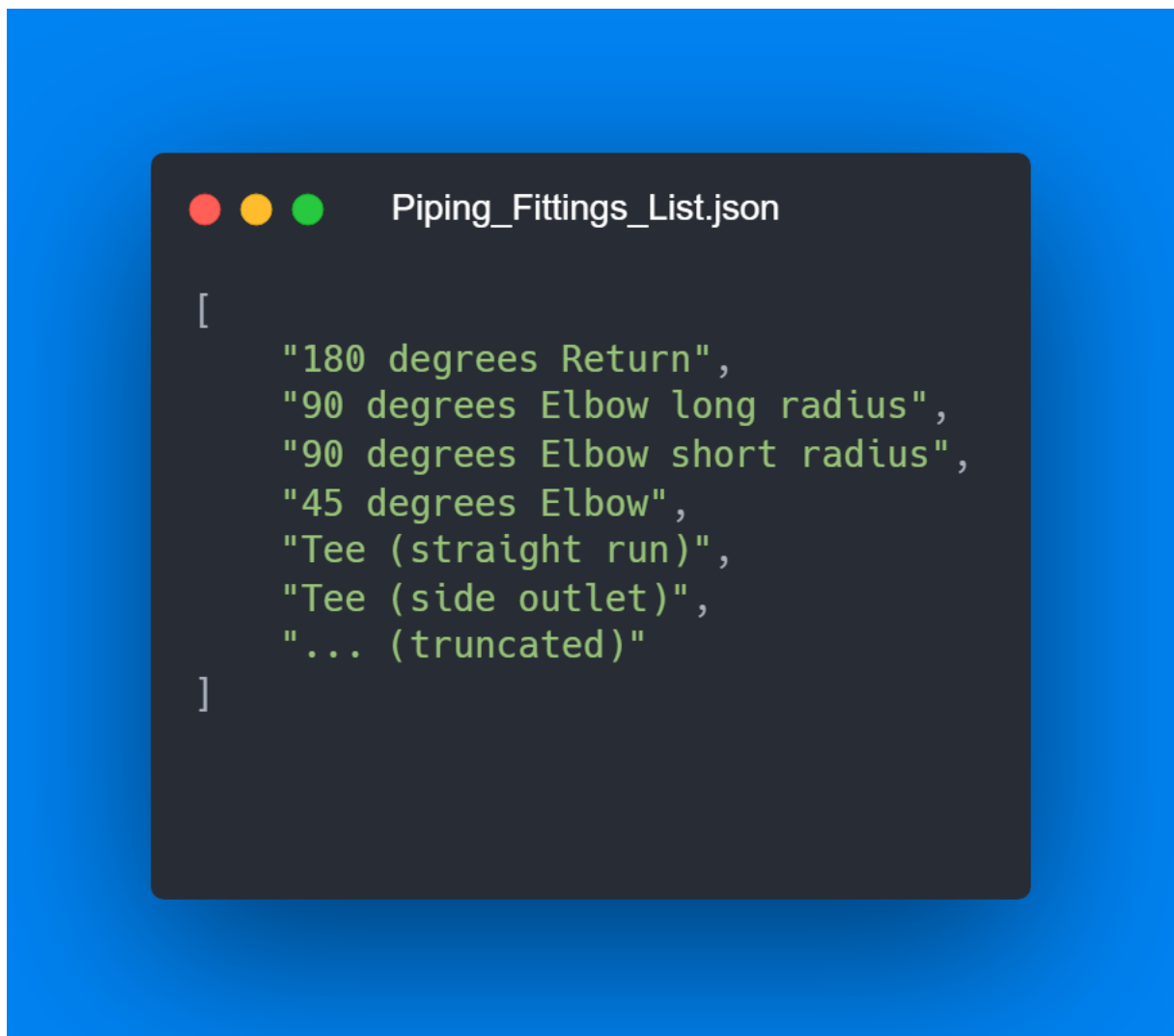
4.4.4 Tubulações

O módulo de tubulações (*Piping*) atua como um banco de dados de engenharia, fornecendo especificações padronizadas para acessórios, materiais e dimensões de tubos. A validação deste módulo consiste na verificação da conformidade dos dados retornados com as tabelas normativas padrão.

4.4.4.1 Acessórios

A função *fittings()* retorna uma lista de todos os acessórios hidráulicos cadastrados no sistema. Esta funcionalidade permite que outros módulos (como o de Perda de Carga) validem a entrada de dados do usuário e utilizem os coeficientes corretos.

Figura 51 – Lista parcial de acessórios disponíveis (JSON)

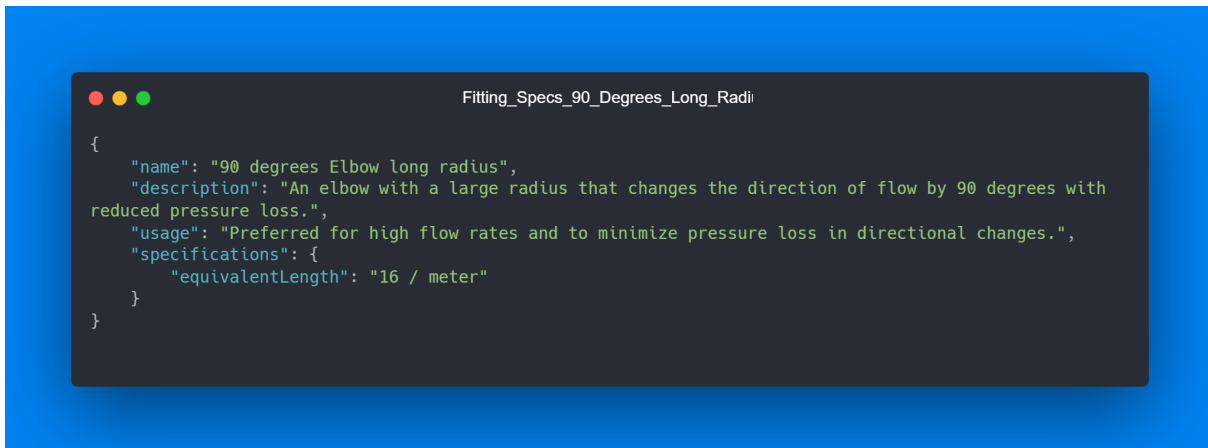


Fonte: Elaboração própria (2026).

4.4.4.2 Especificação de Acessórios

A função *fitting_specifications(nome)* retorna os detalhes técnicos de um acessório, incluindo seu comprimento equivalente adimensional (L_{eq}/D) ou coeficiente de perda de carga (K). Para validação, utilizou-se o acessório *90 degrees Elbow long radius* (Cotovelo 90º Raio Longo).

Figura 52 – Especificações do acessório Cotovelo 90º Raio Longo (JSON)



Fonte: Elaboração própria (2026).

4.4.4.3 Composição

A função *compositions()* lista os materiais de tubulação disponíveis, associando-os às suas respectivas rugosidades absolutas, parâmetro crítico para o cálculo do Fator de Atrito de Darcy.

Figura 53 – Lista parcial de materiais de tubulação (JSON)

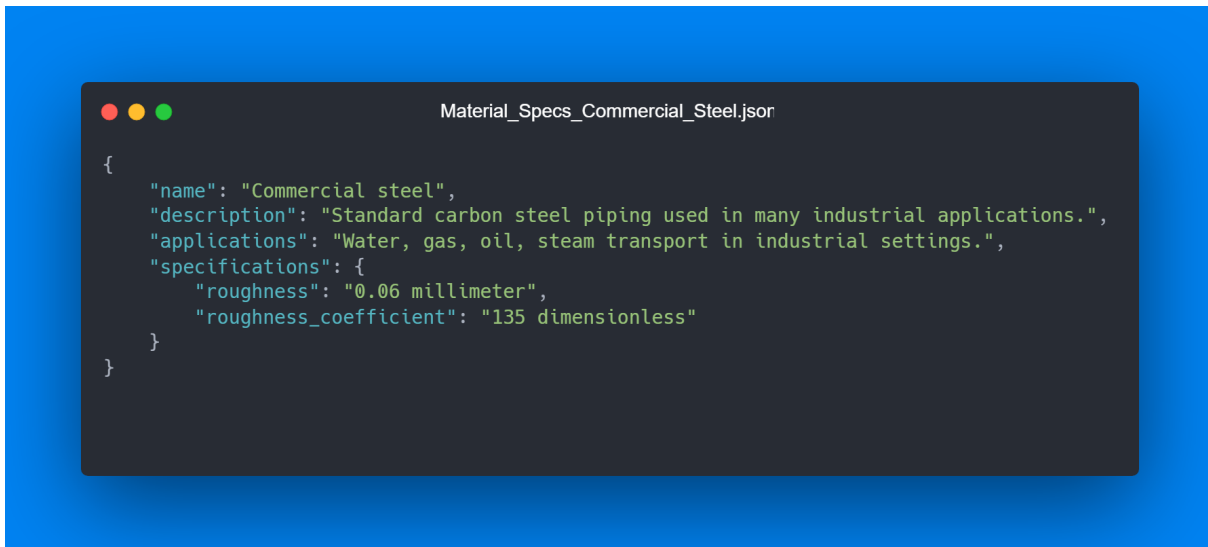


Fonte: Elaboração própria (2026).

4.4.4.4 Especificação da Composição

A função *composition_specifications(nome)* retorna as propriedades de um material específico. Para validação, verificou-se o material *Commercial steel* (Aço comercial).

Figura 54 – Especificações do material Aço Comercial (JSON)



Fonte: Elaboração própria (2026).

4.4.4.5 Schedules

A função *schedules()* fornece as classes de pressão/espessura disponíveis (ex: SCH40, SCH80), permitindo a seleção adequada conforme a norma ANSI/ASME B36.10M.

Figura 55 – Lista de classes de pressão (*Schedules*) disponíveis (JSON)

Fonte: Elaboração própria (2026).

4.4.4.6 Diâmetros

A função *diameters(schedule)* lista os diâmetros nominais (DN) disponíveis para uma determinada classe (*schedule*).

Figura 56 – Lista parcial de diâmetros para SCH40 (JSON)



Fonte: Elaboração própria (2026).

4.4.4.7 Especificação de Diâmetros

A função *diameter_specifications(schedule, dn)* retorna as dimensões reais do tubo, incluindo diâmetro externo, espessura de parede e diâmetro interno (calculado). Para validação, utilizou-se um tubo de *Schedule 40* com Diâmetro Nominal 50 (aprox. 2 polegadas).

Figura 57 – Especificações do tubo SCH40 DN50 (JSON)



Fonte: Elaboração própria (2026).

4.4.5 Reatores

Esta seção apresenta a validação detalhada dos modelos de reatores *CSTR* (*Continuous Stirred-Tank Reactor*) e *PFR* (*Plug Flow Reactor*). Para garantir a precisão dos algoritmos, foram realizados cálculos manuais passo a passo para cenários de projeto e desempenho, comparando-os com os resultados obtidos pela ferramenta computacional.

Para todos os casos, adotou-se uma reação irreversível de primeira ordem $A \rightarrow B$ ocorrendo em fase líquida isotérmica, com os seguintes parâmetros globais:

- Constante cinética da reação (k): $0,05 \text{ s}^{-1}$
- Concentração inicial do reagente A (C_{A0}): 1000 mol/m^3
- Vazão volumétrica de alimentação (Q ou v_0): $0,01 \text{ m}^3/\text{s}$
- Taxa molar de alimentação (F_{A0}): 10 mol/s

4.4.5.1 CSTR: Reator Tanque Agitado Continuamente

A equação de projeto (balanço molar) para um *CSTR* operando em regime estacionário é a Equação 4.10.

Para uma reação de primeira ordem em fase líquida (densidade constante), a taxa de reação é dada por:

$$-r_A = kC_A = kC_{A0}(1 - X) \quad (4.45)$$

Substituindo na equação de projeto e sabendo que $F_{A0} = C_{A0}v_0$:

$$V_{CSTR} = \frac{C_{A0}v_0X}{kC_{A0}(1 - X)} = \frac{v_0X}{k(1 - X)} \quad (4.46)$$

Cálculo Manual (Design): Considerando uma conversão alvo de $X = 0,85$ (85%):

$$V_{manual} = \frac{0,01 \cdot 0,85}{0,05 \cdot (1 - 0,85)} \quad (4.47)$$

$$V_{manual} = \frac{0,0085}{0,05 \cdot 0,15} = \frac{0,0085}{0,0075} \approx 1,1333 \text{ m}^3 \quad (4.48)$$

Comparação com o Código: O módulo *Reactor* foi executado com os mesmos parâmetros.

Tabela 4 – Validação *CSTR* ($X = 0,85$)

Parâmetro	Cálculo Manual	Software	Erro (%)
Volume (m^3)	1,1333	1,1333	0,0000

Fonte: Elaboração própria (2026).

Também validou-se o cálculo inverso (Desempenho), onde fornece-se $V = 1,1333m^3$ e espera-se $X = 0,85$. O código retornou exatamente 0,8500.

4.4.5.2 PFR: Reator Tubular

A equação de projeto para um *PFR* é obtida através do balanço diferencial de massa (Equação 4.14).

Substituindo a cinética de primeira ordem:

$$V = F_{A0} \int_0^X \frac{dX}{kC_{A0}(1 - X)} = \frac{F_{A0}}{kC_{A0}} \int_0^X \frac{dX}{1 - X} \quad (4.49)$$

Como $\frac{F_{A0}}{C_{A0}} = v_0$:

$$V = \frac{v_0}{k} [-\ln(1 - X)]_0^X = -\frac{v_0}{k} \ln(1 - X) \quad (4.50)$$

Cálculo Manual (Design): Considerando uma conversão alvo de $X = 0,90$ (90%):

$$V_{manual} = -\frac{0,01}{0,05} \ln(1 - 0,90) \quad (4.51)$$

$$V_{manual} = -0,2 \cdot \ln(0,10) \quad (4.52)$$

Sabendo que $\ln(0,10) \approx -2,302585$:

$$V_{manual} = -0,2 \cdot (-2,302585) \approx 0,4605 \text{ m}^3 \quad (4.53)$$

Comparação com o Código:

Tabela 5 – Validação *PFR* ($X = 0,90$)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Volume (m^3)	0,4605	0,4605	0,0000

Fonte: Elaboração própria (2026).

4.4.5.3 *PFR* com Reciclo

Em reatores com reciclo, parte da corrente de saída retorna à entrada. Para uma razão de reciclo R (vazão de reciclo / vazão de saída do sistema), a conversão na entrada do reator (X_{in}) é diferente de zero, pois há mistura de reagentes frescos ($X = 0$) com reagentes reciclados ($X = X_{out}$).

A relação de balanço de massa no ponto de mistura fornece:

$$X_{in} = \frac{R}{R+1} X_{out} \quad (4.54)$$

A equação de volume torna-se:

$$V = (R+1)F_{A0} \int_{X_{in}}^{X_{out}} \frac{dX}{-r_A} \quad (4.55)$$

Integrando para primeira ordem:

$$V = (R+1) \frac{v_0}{k} \ln \left(\frac{1 - X_{in}}{1 - X_{out}} \right) \quad (4.56)$$

Cálculo Manual (Design): Considerando uma conversão de saída $X_{out} = 0,90$ e razão de reciclo $R = 2,0$.

Passo 1: Calcular X_{in} .

$$X_{in} = \frac{2,0}{2,0+1} \cdot 0,90 = \frac{2}{3} \cdot 0,90 = 0,60 \quad (4.57)$$

Passo 2: Calcular o Volume.

$$V_{manual} = (2+1) \cdot \frac{0,01}{0,05} \cdot \ln \left(\frac{1 - 0,60}{1 - 0,90} \right) \quad (4.58)$$

$$V_{manual} = 3 \cdot 0,2 \cdot \ln \left(\frac{0,40}{0,10} \right) = 0,6 \cdot \ln(4) \quad (4.59)$$

Sabendo que $\ln(4) \approx 1,38629$:

$$V_{manual} = 0,6 \cdot 1,38629 \approx 0,8318 \text{ m}^3 \quad (4.60)$$

Comparação com o Código:

Tabela 6 – Validação *PFR* com Reciclo ($R = 2, 0$; $X = 0, 90$)

Parâmetro	Cálculo Manual	<i>Software</i>	Erro (%)
Volume (m^3)	0,8318	0,8318	0,0000

Fonte: Elaboração própria (2026).

Os testes confirmam que as rotinas computacionais implementam corretamente as equações diferenciais e algébricas fundamentais da engenharia de reações, incluindo cenários complexos de integração numérica com limites variáveis como no caso do reciclo.

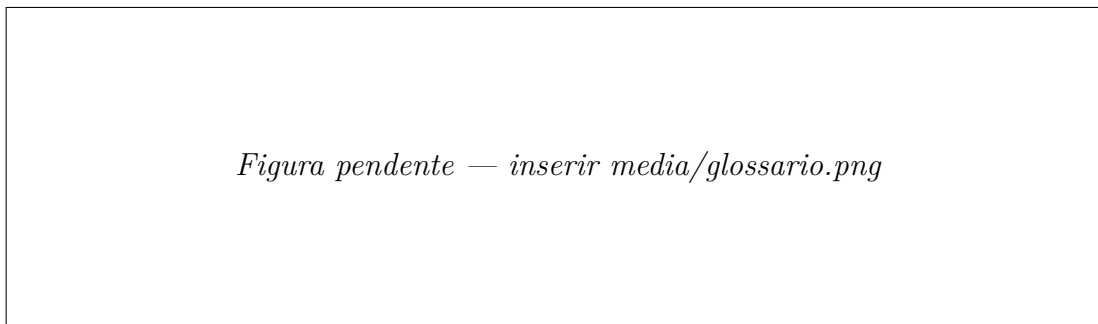
4.5 Recursos Didáticos

Além dos módulos de cálculo, o DCOU incorpora uma camada de recursos voltados ao aprendizado conceitual, que diferenciam a ferramenta de uma simples calculadora. Esses recursos articulam cada resultado numérico ao seu fundamento teórico e ao fenômeno físico correspondente, apoiando estudantes e engenheiros em formação. As subseções a seguir descrevem o glossário técnico, as explicações teóricas embutidas, as visualizações interativas, os cenários de exemplo e as trilhas de aprendizagem.

4.5.1 Glossário

O glossário reúne 49 termos técnicos organizados em oito categorias (*Hidráulica*, *Dimensionamento*, *Bombas*, *Reatores*, *Componentes*, *Balanço*, *Termodinâmica* e *Transferência de calor*), cada um acompanhado de definição e, quando pertinente, da equação correspondente renderizada matematicamente por meio da biblioteca *KaTeX*. Um campo de busca filtra os termos em tempo real, e as categorias se reorganizam dinamicamente conforme os resultados, permitindo que o estudante localize rapidamente conceitos como número de *Reynolds*, fator de atrito de *Darcy*, *NPSH* ou equação de *Antoine*. Na interface final, o glossário também passou a atuar como recurso de apoio ao fluxo de navegação, aparecendo integrado ao painel de recursos para que o usuário compreenda o papel dessa área sem sair da experiência principal.

Figura 58 – Glossário técnico com busca e equações renderizadas

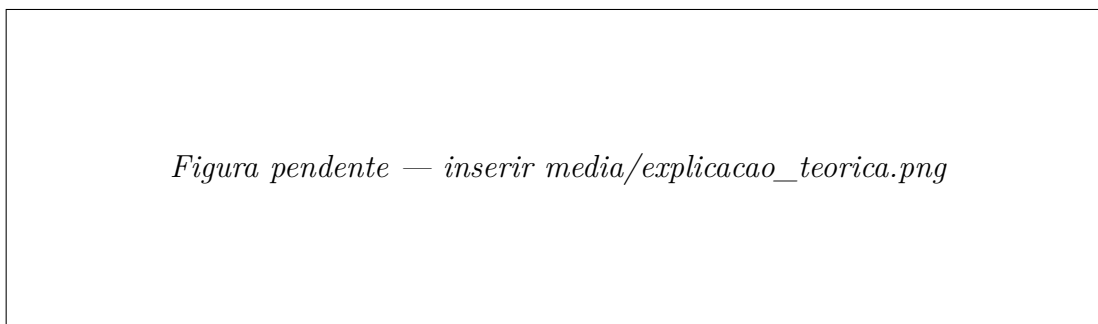


Fonte: Elaboração própria (2026).

4.5.2 Explicações Teóricas Embutidas

Cada calculadora dispõe de blocos teóricos retráteis (“Como funciona”), que apresentam a fundamentação do cálculo sem afastar o usuário da interface. Na versão atual, esses blocos não se limitam às calculadoras escalares de diâmetro, número de *Reynolds*, fator de atrito, diâmetro hidráulico, perda de carga, *NPSH* disponível e altura manométrica: eles também acompanham visualizações específicas, como o diagrama de *Moody*, as curvas de bomba, o mapa de eficiência com *BEP*, o diagrama *T-x-y* binário, o *McCabe-Thiele*, o envelope de fase, a curva de pressão de vapor, o diagrama ternário e os painéis de reatores. Em conjunto, esses blocos reúnem equações, regras de leitura, interpretação física das variáveis e referências bibliográficas coerentes com cada fenômeno (WHITE, 2018; PERRY; GREEN, 2007; FOGLER, 2009; SMITH et al., 2018; MCCABE; SMITH; HARRIOTT, 2005; POLING; PRAUSNITZ; O’CONNELL, 2001; SEADER; HENLEY; ROPER, 2016; KARASSIK et al., 2008; BELL et al., 2014).

Figura 59 – Exemplo de explicação teórica embutida em uma calculadora



Fonte: Elaboração própria (2026).

4.5.3 Visualizações Interativas

Para tornar tangíveis os fenômenos físicos, o *frontend* incorpora uma camada ampla de visualizações contextuais construída em componentes reativos de *SVG*. Essa abordagem permite que os gráficos respondam imediatamente às alterações de entrada e se integrem à mesma arquitetura modular dos formulários e resultados. Atualmente, a aplicação reúne

mais de vinte visualizações distribuídas entre gráficos numéricos com eixos padronizados, diagramas conceituais, mapas de processo e indicadores de operação. Entre os exemplos, destacam-se o perfil de velocidade na seção do duto; a régua de regime de escoamento e o diagrama de *Moody*; as curvas de perda de carga em função da vazão, a margem de *NPSH* e a decomposição da altura manométrica; os diagramas de *Levenspiel*, *T-x-y*, *y-x*, envelope de fase e *McCabe-Thiele*; além de representações de apoio como o gráfico de *Arrhenius*, o perfil espacial de *PFR*, o mapa de eficiência da bomba, os diagramas ternários, a superfície de propriedades e os fluxos de reciclo e purga. Em conjunto, essas visualizações reforçam a leitura dos resultados e aproximam a interpretação numérica do comportamento físico representado.

O valor didático dessa biblioteca de visualizações está em tornar observáveis relações que, em um cálculo puramente tabular, permaneceriam implícitas. No módulo de escoamento, por exemplo, o perfil de velocidade e a régua de regime ajudam a relacionar o número de *Reynolds* com a forma do escoamento e com a mudança de correlações de atrito. No módulo de bombas, as curvas do sistema, a decomposição da altura manométrica, o indicador de *NPSH* e o mapa de eficiência com destaque para o *Best Efficiency Point (BEP)* permitem discutir simultaneamente energia mecânica, perdas, faixa operacional e risco de cavitação. Nos módulos de propriedades e reatores, diagramas como *McCabe-Thiele*, envelope de fase, curva de pressão de vapor, *Levenspiel* e *Arrhenius* transformam resultados intermediários do *backend* em objetos gráficos que favorecem a interpretação física das equações, e não apenas a obtenção do valor final.

4.5.3.1 Diagrama de Moody

O diagrama de *Moody* foi incorporado para tornar explícita a relação entre número de *Reynolds*, rugosidade relativa e fator de atrito de *Darcy*. Na interface, o usuário não recebe apenas o valor final de f : o ponto operacional é sobreposto à família de curvas, permitindo compreender por que materiais distintos e regimes distintos conduzem a perdas de carga diferentes. Como o *backend* já devolve o Re calculado, a rugosidade relativa e o fator de atrito coerentes com a correlação adotada, a visualização permanece rastreável ao modelo hidráulico e favorece a leitura do cálculo em chave física, e não apenas algorítmica (WHITE, 2018; COLEBROOK, 1939; SWAMEE; JAIN, 1976; HAALAND, 1983).

4.5.3.2 Curva da Bomba, Margem de *NPSH* e Mapa de Eficiência

A curva da bomba e do sistema foi adicionada para discutir o ponto operacional como resultado do equilíbrio entre a energia fornecida pelo equipamento e a resistência hidráulica da instalação. Em vez de apresentar apenas uma altura manométrica calculada, a interface mostra como a interseção entre as curvas se desloca quando a vazão ou as perdas variam, o que reforça a interpretação operacional do problema (FOX; PRITCHARD; MCDONALD, 2011). Complementarmente, o gráfico de margem de *NPSH* foi incluído

para representar visualmente a folga entre $NPSH_d$ e $NPSH_r$, permitindo discutir cavitação de modo imediato; já o mapa de eficiência com destaque para o *Best Efficiency Point* (*BEP*) evidencia se a condição corrente está próxima da região de melhor rendimento ou deslocada para uma faixa menos favorável, inclusive com indicação aproximada da região de cavitação. Nesses casos, o *backend* devolve não apenas valores escalares, mas também a malha de eficiência relativa, a curva do sistema, o ponto de operação e os marcadores críticos que estruturam a leitura didática (KARASSIK et al., 2008; FOX; PRITCHARD; MCDONALD, 2011).

4.5.3.3 Diagrama de Levenspiel

O diagrama de *Levenspiel* foi mantido como peça central do módulo de reatores porque ele transforma a comparação entre *CSTR* e *PFR* em uma relação geométrica facilmente interpretável pelo estudante. Na implementação atual, o gráfico destaca os pontos operacionais de ambos os modelos e compara a demanda volumétrica associada a uma mesma conversão, deixando visível por que o *PFR* tende a requerer menor volume em cinéticas positivas quando comparado ao *CSTR*. Sua presença na monografia é particularmente importante porque ele não atua como ornamento visual: ele traduz em forma gráfica a própria equação de projeto dos reatores ideais, articulando diretamente cálculo e interpretação (LEVENSPIEL, 2000; FOGLER, 2009).

4.5.3.4 Gráfico de Arrhenius

O gráfico de *Arrhenius* foi incluído para apoiar a compreensão da dependência da constante cinética com a temperatura. Ao linearizar a relação entre k e T no plano $1000/T$ versus $\ln(k)$, a visualização permite discutir sensibilidade térmica, energia de ativação e aceleração reacional sem exigir que o estudante permaneça apenas na manipulação simbólica da equação. Seu papel pedagógico é evidenciar que pequenas alterações de temperatura podem produzir mudanças expressivas na taxa de reação, o que se conecta diretamente à análise dos módulos de *CSTR* e *PFR* (FOGLER, 2009).

4.5.3.5 Perfis Espaciais no *PFR*

Os perfis espaciais no *PFR* foram adicionados para evitar uma leitura excessivamente agregada do reator tubular. Em vez de resumir o comportamento do sistema a um único ponto de saída, a interface exibe séries de concentração e temperatura ao longo do volume relativo do reator, acompanhadas de um esquema que marca estações intermediárias calculadas pelo *backend*. Isso ajuda a consolidar a ideia de que o *PFR* deve ser interpretado como um campo espacial de estados, no qual cada posição do reator opera sob composição distinta (FOGLER, 2009; LEVENSPIEL, 2000).

4.5.3.6 Diagrama T - x - y Binário

O diagrama T - x - y binário foi incluído para representar o equilíbrio líquido-vapor em misturas binárias a pressão fixa. A presença simultânea das curvas de bolha e de orvalho permite discutir início de vaporização, início de condensação, coexistência entre fases e volatilidade relativa sem depender apenas de tabelas termodinâmicas. Como o *backend* calcula os pontos de equilíbrio e entrega as séries já estruturadas para renderização, o estudante observa o comportamento do sistema a partir de dados consistentes com o mesmo motor termodinâmico usado nas demais consultas de propriedades (SMITH et al., 2018).

4.5.3.7 Diagrama de McCabe-Thiele

O diagrama de *McCabe-Thiele* foi incorporado para explicitar a construção gráfica do número teórico de estágios em uma separação binária por destilação. Na aplicação, a curva de equilíbrio, a diagonal, as linhas de enriquecimento e esgotamento, a linha q e os degraus são todos montados a partir dos dados termodinâmicos e operacionais resolvidos no *backend*, o que permite ao usuário relacionar composição de topo, composição de fundo, condição da alimentação e razão de refluxo com a contagem de estágios. O valor didático do gráfico está em mostrar que o resultado de projeto não emerge de uma “caixa-preta”, mas de uma sequência geométrica interpretável (MCCABE; SMITH; HARRIOTT, 2005; SMITH et al., 2018).

4.5.3.8 Envelope de Fase e Curva de Pressão de Vapor

O envelope de fase e a curva de pressão de vapor foram adicionados para apoiar a leitura termodinâmica de limites de estabilidade e mudança de fase. O envelope destaca a fronteira entre regiões de líquido, vapor e coexistência, incluindo marcadores de ponto crítico e ponto tríplice; a curva de pressão de vapor, por sua vez, mostra como a pressão de saturação varia com a temperatura, auxiliando discussões sobre volatilidade, ebulição e risco de cavitação. Em ambos os casos, a utilidade pedagógica está em associar propriedades antes tratadas de forma tabular a regiões operacionais visualmente distinguíveis (POLING; PRAUSNITZ; O'CONNELL, 2001; SMITH et al., 2018).

4.5.3.9 Diagrama Ternário

O diagrama ternário foi incorporado para representar composições de correntes com três componentes sem a necessidade de introduzir, neste estágio, um módulo completo de separações multicomponentes. Na implementação atual, o *backend* normaliza as frações molares, projeta a corrente no plano ternário e devolve a geometria do triângulo, das linhas-guia e do ponto da corrente pronta para renderização. Didaticamente, isso permite mostrar que cada vértice representa um componente puro, cada lado uma mistura binária

e o interior uma composição ternária, o que facilita a leitura de misturas e a preparação conceitual para problemas de extração e equilíbrio mais avançados (SEADER; HENLEY; ROPER, 2016; DEVOE, 2014).

4.5.3.10 Superfície T - P de Propriedades

Por fim, a superfície T - P de propriedades foi adicionada para evidenciar que grandezas termodinâmicas não dependem de forma linear das variáveis de estado. Ao colorir e discretizar o valor de uma propriedade em função de temperatura e pressão, a visualização transforma uma consulta isolada em um campo interpretável, útil para comparar tendências, identificar regiões de maior gradiente e compreender a sensibilidade do modelo termodinâmico às condições impostas. Como essa superfície é construída a partir do mesmo núcleo de propriedades do *CoolProp*, ela reforça a coerência entre cálculo puntual e exploração gráfica (BELL et al., 2014; POLING; PRAUSNITZ; O'CONNELL, 2001).

Figura 60 – Diagrama de *Moody* com o ponto operacional do problema destacado

Friction Factor

Roughness Type

☒ Material Composition ☐ Custom Value

Material Composition

Commercial steel

Diameter Type

☒ Pipe Schedule ☐ Custom Value

Pipe Schedule

SCH40

Diameter (mm)

15 mm

Reynolds Number

100000

Method

ColebrookWhite

Calculate Friction Factor

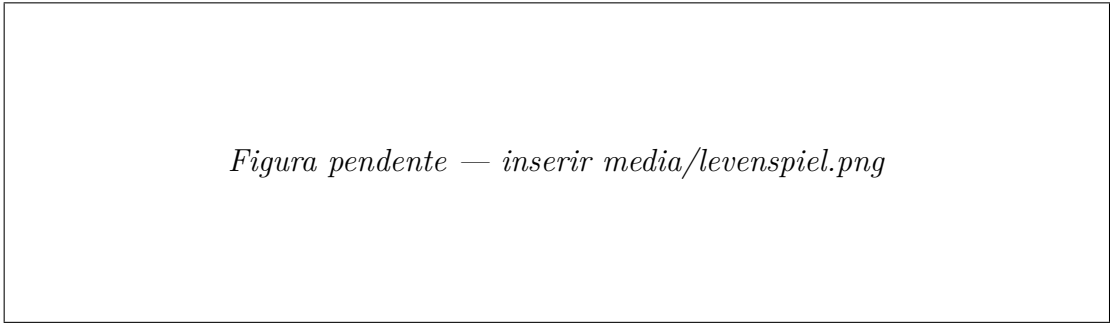
Limpar campos

Friction Factor

Property	Value	Units
Diameter	0.0294801	dimensionless

Fonte: Elaboração própria (2026).

Figura 61 – Diagrama de *Levenspiel* comparando reatores *CSTR* e *PFR*



Fonte: Elaboração própria (2026).

4.5.4 Cenários de Exemplo

Além das visualizações, o sistema oferece cenários de exemplo providos pela própria *API* para acelerar a exploração didática dos módulos. Em vez de armazenar resultados prontos no *frontend*, esses *endpoints* retornam conjuntos coerentes de entradas para escoamento, dimensionamento, bombas, propriedades de componentes e outros contextos, respeitando os catálogos e limites disponíveis no *backend*. A interface utiliza esses dados para pré-preencher os formulários e, em seguida, executa o mesmo fluxo de cálculo usado em situações reais.

Essa estratégia possui valor pedagógico relevante por dois motivos. Primeiro, reduz a barreira inicial para o estudante, que passa a observar um caso funcional antes de montar um cenário próprio do zero. Segundo, preserva a rastreabilidade entre entradas, equações e visualizações, pois os exemplos não contornam o motor computacional: eles apenas selecionam condições iniciais representativas, como um caso de transição em número de *Reynolds*, um sistema com margem de *NPSH* explícita ou uma separação binária que justifique a construção de um diagrama de *McCabe-Thiele*.

4.5.5 Trilhas de Aprendizagem

A página inicial organiza o uso dos módulos em três trilhas de aprendizagem, que sugerem uma sequência didática de estudo: **Transporte de Fluidos** (Tubulações → Dimensionamento → Escoamento → Bombas); **Reatores Ideais** (Componentes → Reatores *CSTR/PFR*); e **Balanco de Massa** (Componentes → Balanco). As trilhas orientam o estudante a percorrer os módulos em uma ordem coerente com a construção dos conceitos, em vez de utilizá-los de forma isolada. No desenho atual da interface, essas trilhas convivem com cartões de acesso rápido para os módulos principais, reforçando a intenção de uso de cada área antes da abertura do cálculo.

Figura 62 – Trilhas de aprendizagem apresentadas na página inicial

Figura pendente — inserir media/trilhas.png

Fonte: Elaboração própria (2026).

5 Resultados e Discussões

O desenvolvimento do *software* demonstrou que é possível integrar conceitos fundamentais da engenharia química com tecnologias modernas de programação para produzir uma ferramenta acessível, modular e capaz de automatizar cálculos recorrentes da área.

Do ponto de vista funcional, o *backend* em *FastAPI* apresentou desempenho satisfatório, respondendo rapidamente aos cálculos mesmo em operações envolvendo chamadas reiteradas ao módulo de propriedades termodinâmicas (*CoolProp*) ou a rotinas numéricas como métodos iterativos de Brent e integrações do *SciPy*.

A validação de entradas pelo *Pydantic* mostrou-se fundamental para evitar inconsistências numéricas e garantir segurança nos cálculos, reduzindo significativamente erros comuns como unidades incorretas, tipos inválidos ou parâmetros ausentes.

No *frontend*, o uso de *React*, *TypeScript* e *Vite*, associado a *Tailwind CSS*, primitivas acessíveis de *@base-ui/react*, componentes compostos no padrão *shadcn*, *React Router* e elementos visuais próprios, resultou em uma interface modular, responsiva e adequada ao propósito educacional do sistema. Essa arquitetura favoreceu a separação entre apresentação, estado e integração com a *API*, permitindo encapsular cada categoria de cálculo em componentes e serviços tipados. Bibliotecas como *KaTeX*, *Lucide React* e *Sonner* complementam a interface com renderização matemática, ícones contextuais e notificações discretas. Além disso, a criação de uma camada compartilhada para grades, eixos, marcadores e rótulos numéricos padronizou parte relevante das visualizações em *SVG*, o que melhora a consistência gráfica e simplifica a evolução dos componentes. Na interface consolidada, a navegação evidencia o papel pedagógico de cada área, com uma *home* orientada por trilhas, atalhos diretos para os módulos e indicadores visuais de processo que reforçam a interpretação do regime de escoamento.

Para além da automação de cálculos, a incorporação de recursos didáticos consolidou o DCOU como um ambiente de ensino, e não apenas como uma calculadora. As explicações teóricas embutidas em cada módulo, o glossário técnico, os cenários de exemplo providos pela *API* e o conjunto ampliado de visualizações interativas aproximam o resultado numérico do conceito que o fundamenta, ao apresentar o comportamento dos fenômenos físicos de forma contextualizada. O uso de contratos tipados para exemplos e gráficos também mostrou-se eficaz para manter coerência entre cálculo, apresentação e interpretação pedagógica em diferentes módulos.

A comparação dos resultados obtidos com valores de referência da literatura e com *softwares* consolidados evidenciou boa concordância para todos os módulos implementados. Cálculos de *Reynolds*, perdas de carga, seleção de diâmetros e estimativas de *NPSH* apresentaram diferenças mínimas em relação às tabelas clássicas. Para reatores *CSTR* e *PFR*, os valores de conversão e volume mostraram-se consistentes com soluções de Fogler

e Levenspiel, reforçando a confiabilidade das integrações numéricas.

Embora o *software* esteja plenamente funcional para os módulos desenvolvidos, observou-se que o sistema ainda pode ser expandido significativamente. A ausência de módulos como trocadores de calor, destiladores, absorção, secagem e operações com múltiplas reações limita sua aplicabilidade em projetos de maior complexidade. Do ponto de vista de usabilidade e de maturidade funcional, futuras melhorias incluem exportação de relatórios, ampliação de gráficos especializados e integração com bancos de dados externos de propriedades físicas, de modo a ampliar o alcance técnico da ferramenta.

Ainda assim, o objetivo de criar uma ferramenta precisa, acessível, gratuita e colaborativa foi alcançado. O fato de o projeto ser *open-source* torna natural que seu ciclo de evolução dependa da contribuição da comunidade acadêmica e profissional, que poderá incluir novos módulos, otimizar algoritmos e expandir a base de dados.

6 Guia de Uso da Ferramenta

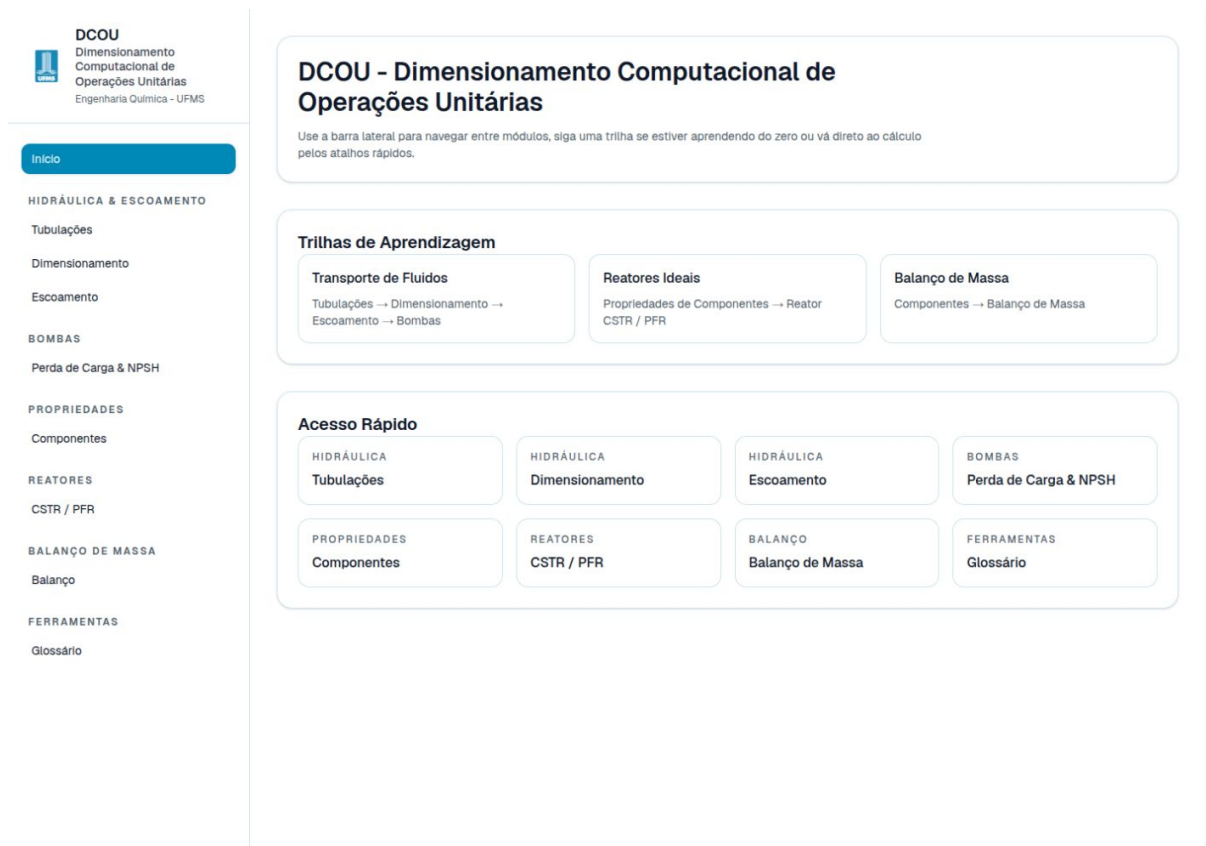
Este capítulo apresenta um roteiro de uso do DCOU para discentes e docentes de Engenharia Química. A finalidade é mostrar, em sequência, como acessar a aplicação, navegar entre os módulos, carregar exemplos, executar cálculos e interpretar resultados numéricos e gráficos. As telas foram verificadas na aplicação local em ambiente desktop, no endereço *web* local na data de 08/07/2026, podendo haver diferenças visuais e de funcionalidade conforme atualização no servidor produtivo.

Em todos os módulos que disponibilizam o botão **Carregar exemplo**, as capturas deste capítulo foram feitas a partir dos exemplos fornecidos pela própria ferramenta. Quando a tela possuía botão de execução, como **Calcular Reynolds**, **Calcular CSTR**, **Calcular PFR** ou **Calcular Balanço de Massa**, o cálculo foi executado antes da captura. Dessa forma, o fluxo documentado reproduz o uso real esperado: carregar um caso coerente, observar as entradas, executar o cálculo e interpretar o resultado.

6.1 Acesso e Tela Inicial

O acesso local é feito abrindo o navegador e informando a URL [<http://localhost:5173>](http://localhost:5173). Em produção, o endereço atualmente utilizado é [<https://tcc.joao.baza.dev.br>](https://tcc.joao.baza.dev.br). A tela inicial apresenta a identidade do DCOU, o menu lateral de módulos, as trilhas de aprendizagem e os cartões de acesso rápido. Para uma atividade de ensino, recomenda-se escolher a trilha compatível com o conteúdo da aula: transporte de fluidos, reatores ideais ou balanço de massa.

Figura 63 – Tela inicial com trilhas de aprendizagem e cartões de acesso rápido



Fonte: Elaboração própria (2026).

Na leitura didática, o estudante deve observar que a aplicação não é organizada como uma calculadora única. Cada módulo representa uma etapa conceitual: catálogos de dados, dimensionamento, análise de regime, propriedades, reatores, balanço e glossário. Para o docente, essa organização permite conduzir uma aula por sequência de fenômenos, em vez de apresentar apenas resultados isolados.

6.2 Tubulações e Acessórios

O módulo **Tubulações** reúne dados usados posteriormente nos cálculos hidráulicos. Na aba **Composições**, o exemplo carregado seleciona uma composição de tubulação e mostra suas propriedades catalogadas. Essa tela deve ser usada antes de problemas de perda de carga para discutir como material e rugosidade afetam a modelagem.

Figura 64 – Aba Composições do módulo de tubulações com exemplo carregado

DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensionamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORRES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Tubulações e Acessórios

Consulte materiais, schedules, diâmetros nominais e conexões usadas nos módulos hidráulicos da aplicação.

Sucesso
Exemplo carregado com sucesso.

Composições Schedules e Diâmetros Conexões

Composições

Como funciona - Composições

Composição

Aço galvanizado

Propriedade	Valor	Unidade
Nome	Aço galvanizado	-
Descrição	Tubulação de aço revestida com zinco para prevenir corrosão.	-
Aplicações	Sistemas de água potável, sprinklers e irrigação.	-
Roughness	0,16	mm
Roughness Coefficient	125	-

Fonte: Elaboração própria (2026).

Na aba **Schedules e Diâmetros**, o usuário observa diâmetros nominais e dimensões comerciais. Essa etapa conecta o diâmetro calculado pelo modelo matemático com a seleção real de tubulações disponíveis.

Figura 65 – Aba Schedules e Diâmetros com diâmetro comercial do exemplo

DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensionamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Tubulações e Acessórios

Consulte materiais, schedules, diâmetros nominais e conexões usadas nos módulos hidráulicos da aplicação.

Sucesso
Exemplo carregado com sucesso.

Composições Schedules e Diâmetros Conexões

Schedules e Diâmetros

Como funciona - Schedules e Diâmetros

Schedule

Schedule 40

Schedule padrão utilizado na maioria das aplicações comerciais e industriais.

Diâmetro nominal

125 mm

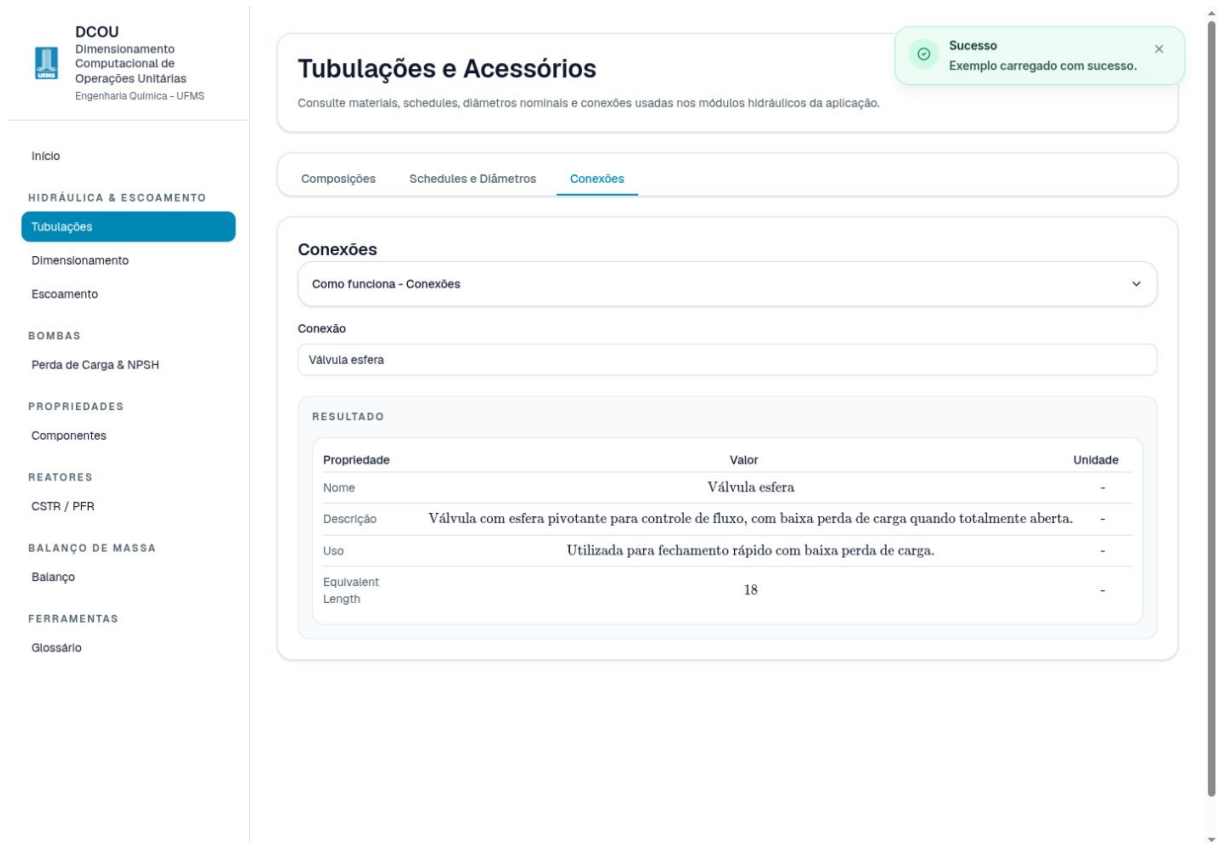
RESULTADO

Propriedade	Valor	Unidade
External Diameter	141,3	mm
Thickness	6,55	mm
Weight	21,77	kg/m
Max Pressure	$5,75712 \times 10^6$	Pa

Fonte: Elaboração própria (2026).

Na aba **Conexões**, o exemplo mostra o cadastro de acessórios e seus coeficientes. A interpretação esperada é que válvulas, cotovelos, entradas e saídas de tanque não são detalhes secundários: eles entram diretamente no cálculo de perdas localizadas.

Figura 66 – Aba Conexões com acessório carregado pelo exemplo



DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensãoamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Tubulações e Acessórios

Consulte materiais, schedules, diâmetros nominais e conexões usadas nos módulos hidráulicos da aplicação.

Composições Schedules e Diâmetros **Conexões**

Conexões

Como funciona - Conexões

Conexão

Válvula esfera

RESULTADO

Propriedade	Valor	Unidade
Nome	Válvula esfera	-
Descrição	Válvula com esfera pivotante para controle de fluxo, com baixa perda de carga quando totalmente aberta.	-
Uso	Utilizada para fechamento rápido com baixa perda de carga.	-
Equivalent Length	18	-

Sucesso
Exemplo carregado com sucesso.

Fonte: Elaboração própria (2026).

6.3 Dimensionamento

No módulo **Dimensionamento**, a aba **Diâmetro Calculado** solicita vazão volumétrica e velocidade de projeto. Com o exemplo carregado, o estudante pode acionar **Calcular diâmetro** e observar a tabela de resultado. A pergunta didática central é: o valor calculado é um requisito hidráulico, mas a escolha final ainda precisa ser compatibilizada com um diâmetro comercial.

Figura 67 – Aba Diâmetro Calculado com exemplo executado

Dimensionamento de Tubulação

Calcule o diâmetro hidráulico a partir da vazão e da velocidade de projeto e, em seguida, selecione o próximo diâmetro comercial do schedule adotado.

Diâmetro Calculado | Diâmetro Real

Diâmetro Calculado

Como funciona - Cálculo de Diâmetro

Vazão (m³/s)
0,0166667

Velocidade de projeto (m/s)
1,5

Calcular diâmetro | **Limpar campos**

RESULTADO

Propriedade	Valor	Unidade
Diâmetro calculado	118,94173	mm

Perfil de Velocidade - Duto Circular

Turbulento - $Re = 1,78413 \times 10^5$ | Perfil estimado para água a 20 °C - D = 118,94173 mm - V = 1,5 m/s

Fonte: Elaboração própria (2026).

Na aba **Diâmetro Real**, a ferramenta compara o diâmetro calculado com o schedule selecionado e retorna o diâmetro comercial imediatamente aplicável. O discente deve observar a diferença entre cálculo contínuo e seleção discreta de catálogo.

Figura 68 – Aba Diâmetro Real com seleção de diâmetro comercial

Dimensionamento de Tubulação

Calcule o diâmetro hidráulico a partir da vazão e da velocidade de projeto e, em seguida, selecione o próximo diâmetro comercial do schedule adotado.

Sucesso
Exemplo carregado com sucesso.

Diâmetro Calculado **Diâmetro Real**

Diâmetro Real

Como funciona - Diametro Nominal Comercial

Diâmetro calculado (mm)

118,94

Schedule

Schedule 40

Schedule padrão utilizado na maioria das aplicações comerciais e industriais.

Obter diâmetro real

RESULTADO		
Propriedade	Valor	Unidade
Diâmetro real	125	mm

Fonte: Elaboração própria (2026).

6.4 Escoamento Interno

O módulo **Escoamento** começa pela aba **Número de Reynolds**. Depois de carregar o exemplo e executar o cálculo, a tela mostra o valor de Reynolds e a régua de regime. A interpretação não deve parar no número: o marcador indica se o escoamento é laminar, de transição ou turbulento, afetando as correlações posteriores.

Figura 69 – Aba Número de Reynolds com resultado e régua de regime

Dimensionamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORRES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Número de Reynolds

Como funciona - Número de Reynolds

Diâmetro característico (mm)

13,843

Velocidade média (m/s)

3,923

Densidade (kg/m³)

0,65688

Viscosidade dinâmica (Pa.s)

0,000011963

Viscosidade cinemática (m²/s)

Calcular Reynolds

RESULTADO

Propriedade	Valor	Unidade
Número de Reynolds	3186,10467	-

Regime do escoamento

Escala linear de Reynolds de 100 a 10.000. O marcador mostra a posição atual.

Laminar

Transição

Turbulento

100

500

1000

2300

4000

6000

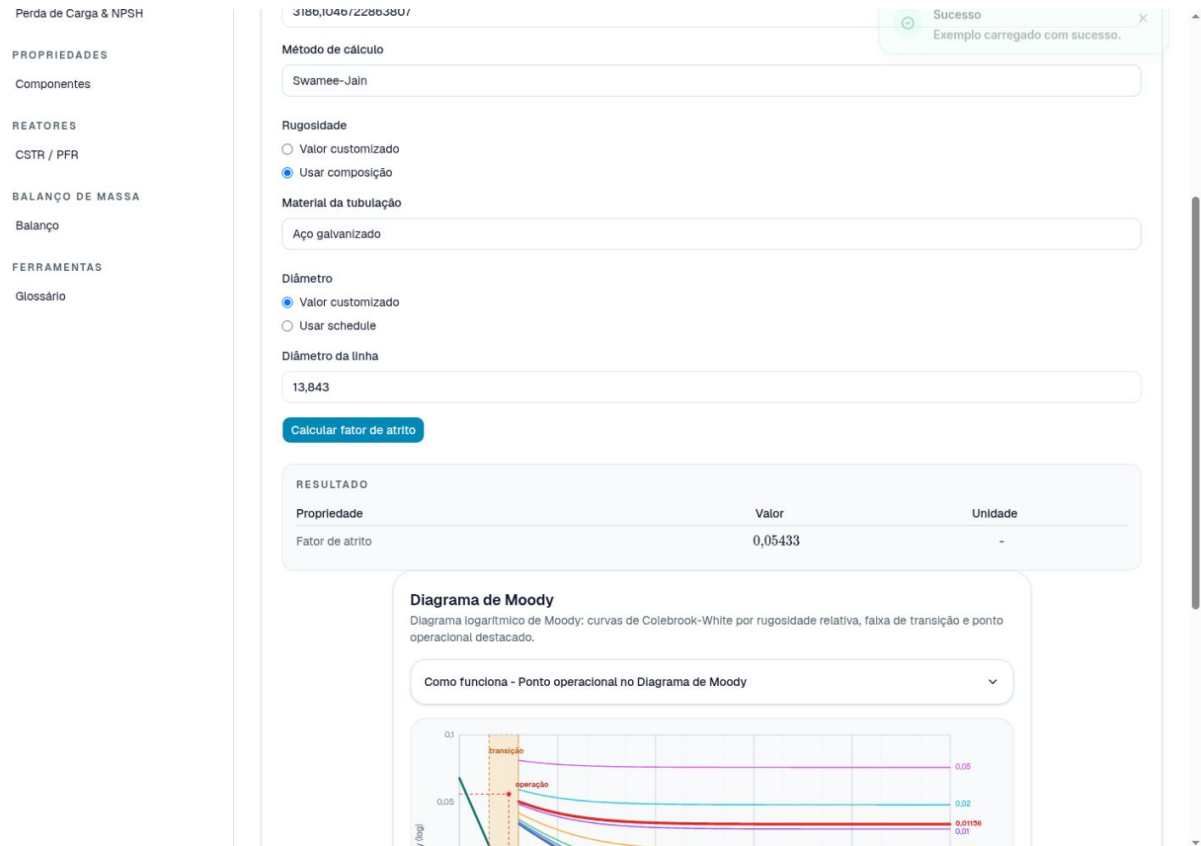
8000

10000

Fonte: Elaboração própria (2026).

Na aba **Fator de Atrito**, o exemplo calcula o fator de Darcy e exibe o diagrama de *Moody*. Essa tela deve ser usada para discutir a influência simultânea do número de Reynolds e da rugosidade relativa.

Figura 70 – Aba Fator de Atrito com diagrama de Moody



Fonte: Elaboração própria (2026).

Na aba **Diâmetro Hidráulico**, o exemplo evidencia que geometrias não circulares também podem ser tratadas por uma dimensão característica. O estudante deve observar a relação entre geometria, área molhada e perímetro molhado.

Figura 71 – Aba Diâmetro Hidráulico com pré-visualização geométrica

Interface da ferramenta para cálculo de Diâmetro Hidráulico.

Menu Lateral:

- Início
- HIDRÁULICA & ESCOAMENTO
 - Tubulações
 - Dimensionamento
 - Escoamento**
- BOMBAS
 - Perda de Carga & NPSH
- PROPRIEDADES
 - Componentes
- REATORES
 - CSTR / PFR
- BALANÇO DE MASSA
 - Balanço
- FERRAMENTAS
 - Glossário

Topo da Interface:

- Abas: Número de Reynolds, Fator de Atrito, **Diâmetro Hidráulico**
- Notificação: Sucesso. Exemplo carregado com sucesso.

Diâmetro Hidráulico

Como funciona - Diâmetro Hidráulico

Forma geométrica: Canal circular

Diâmetro (m): 0,125

Altura (m): 0,08333

Calcular diâmetro hidráulico

Pré-visualização geométrica
Segmento circular preenchido, com D, h e R destacados.

D = 0,125 h = 0,08333 R = 0,0625

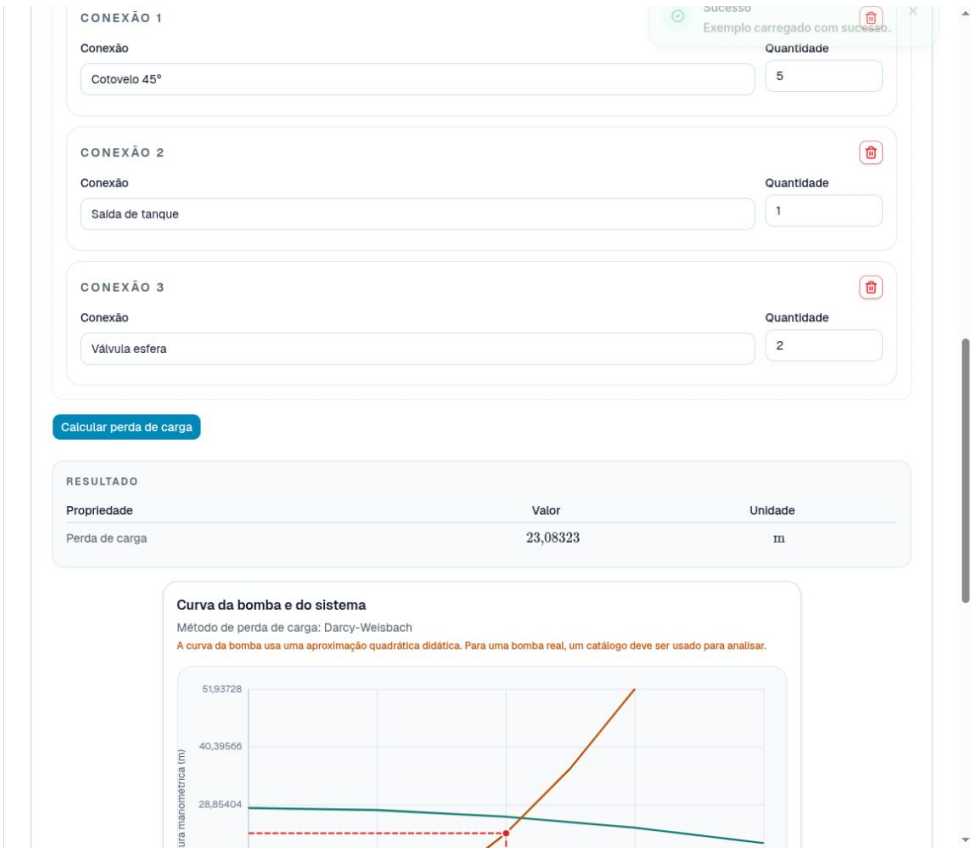
Propriedade	Valor	Unidade
Diâmetro hidráulico	0,09746	m

Fonte: Elaboração própria (2026).

6.5 Perda de Carga e Bombas

Na aba **Perda de Carga**, o exemplo combina comprimento de linha, diâmetro, vazão, material, Reynolds, método de fator de atrito e acessórios. Ao executar o cálculo, a ferramenta retorna a perda de carga e apresenta curvas didáticas do sistema. O docente pode explorar a tela alterando material, vazão ou conexões e observando o deslocamento do ponto de operação.

Figura 72 – Aba Perda de Carga com exemplo e curva do sistema



Fonte: Elaboração própria (2026).

Na aba **NPSH Disponível**, o exemplo calcula a margem entre o *NPSH* disponível e o requerido. O ponto de atenção para o estudante é a cavitação: valores próximos ao limite indicam risco operacional e exigem revisão das condições de sucção.

Figura 74 – Aba Altura Manométrica com decomposição dos termos de energia

REATORES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Elevação 1 (m)
0

Elevação 2 (m)
5

Velocidade 1 (m/s)
0

Velocidade 2 (m/s)
3

Massa específica do fluido (kg/m³)
1000

Perda de carga total (m)
23,08

Calcular altura manométrica

RESULTADO

Propriedade	Valor	Unidade
Altura manométrica	28,53887	m

Decomposição

H total = 28,53887 m

PARCELA	VALOR	PORCENTAGEM
$\Delta P / (\rho g)$	0	0%
Δz	5	17,51996%
$\Delta V^2 / (2g)$	0,45887	1,60789%
$-\hat{h}_f$	-23,08	80,87215%

Fonte: Elaboração própria (2026).

6.6 Propriedades de Componentes

O módulo **Componentes** reúne consultas termodinâmicas e visualizações de equilíbrio. Na aba **Propriedades Críticas**, o exemplo recupera propriedades críticas do fluido selecionado. Essa tela é útil para introduzir limites de validade e regiões próximas ao ponto crítico.

Figura 75 – Aba Propriedades Críticas com fluido do exemplo

DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensionamento

Esgoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REACTORES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Propriedades de Componentes

Consulte propriedades críticas, fluidos puros, misturas e equilíbrio binário usando os serviços já expostos pela API.

Propriedades Críticas | Fluido Puro | Misturas | Equilíbrio Binário | Envelope de Fase

Propriedades Críticas

Como funciona - Propriedades Críticas

Fluido crítico

Air

Obter propriedades críticas

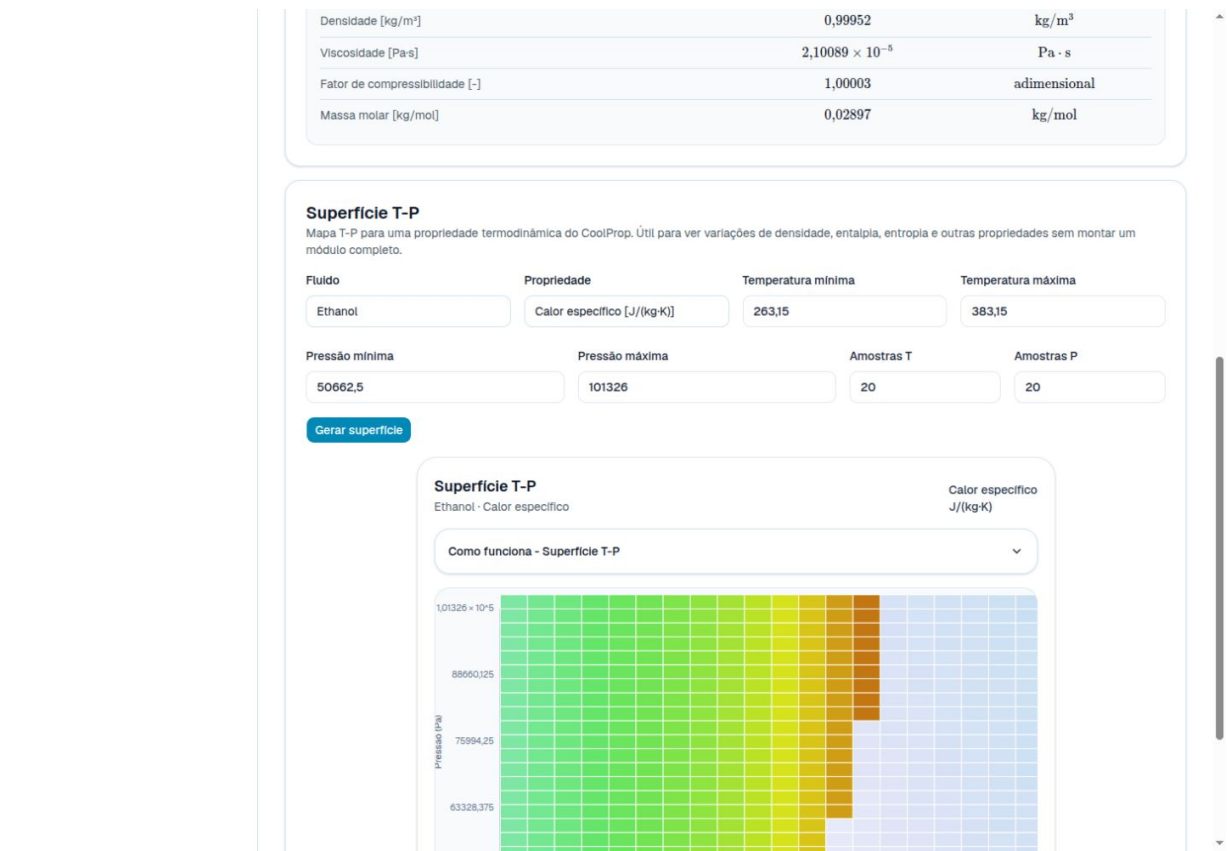
Propriedade	Valor	Unidade
Temperatura crítica	132,5306	K
Pressão crítica	$3,786 \times 10^6$	Pa
Densidade crítica	342,68456	kg/m ³
Temperatura do ponto triplo	59,75	K
Pressão do ponto triplo	5264,18107	Pa

Sucesso
Exemplo carregado com sucesso.

Fonte: Elaboração própria (2026).

Na aba **Fluido Puro**, o exemplo mostra propriedades de estado e, na mesma tela, a superfície T - P . O discente deve observar que propriedades como densidade, entalpia e entropia variam com temperatura e pressão, não apenas com uma variável isolada.

Figura 76 – Aba Fluido Puro com propriedades e superfície T-P



Fonte: Elaboração própria (2026).

Na aba **Misturas**, a ferramenta calcula propriedades de mistura e apresenta a visualização ternária quando aplicável. Essa tela apoia discussões sobre composição, normalização de frações e leitura de sistemas multicomponentes.

Figura 77 – Aba Misturas com exemplo e diagrama ternário

Methanol

Fração molar 3
0,01

Remover fluido

Adicionar fluido

Temperatura da mistura
303,15

Pressão da mistura
101325

Propriedades da mistura

Densidade [kg/m³] x Massa molar [kg/mol] x Viscosidade [Pa·s] x Calor específico [J/(kg·K)] x Seleccione propriedades

Calcular mistura

COMPOSIÇÃO DA MISTURA

Water: 0.7 Ethanol: 0.29 Methanol: 0.01

RESULTADO

Propriedade	Valor	Unidade
Densidade [kg/m³]	842,63486	kg/m³
Massa molar [kg/mol]	0,02629	kg/mol
Viscosidade [Pa·s]	$9,14928 \times 10^{-4}$	Pa·s
Calor específico [J/(kg·K)]	3900,39747	joule/K/kg

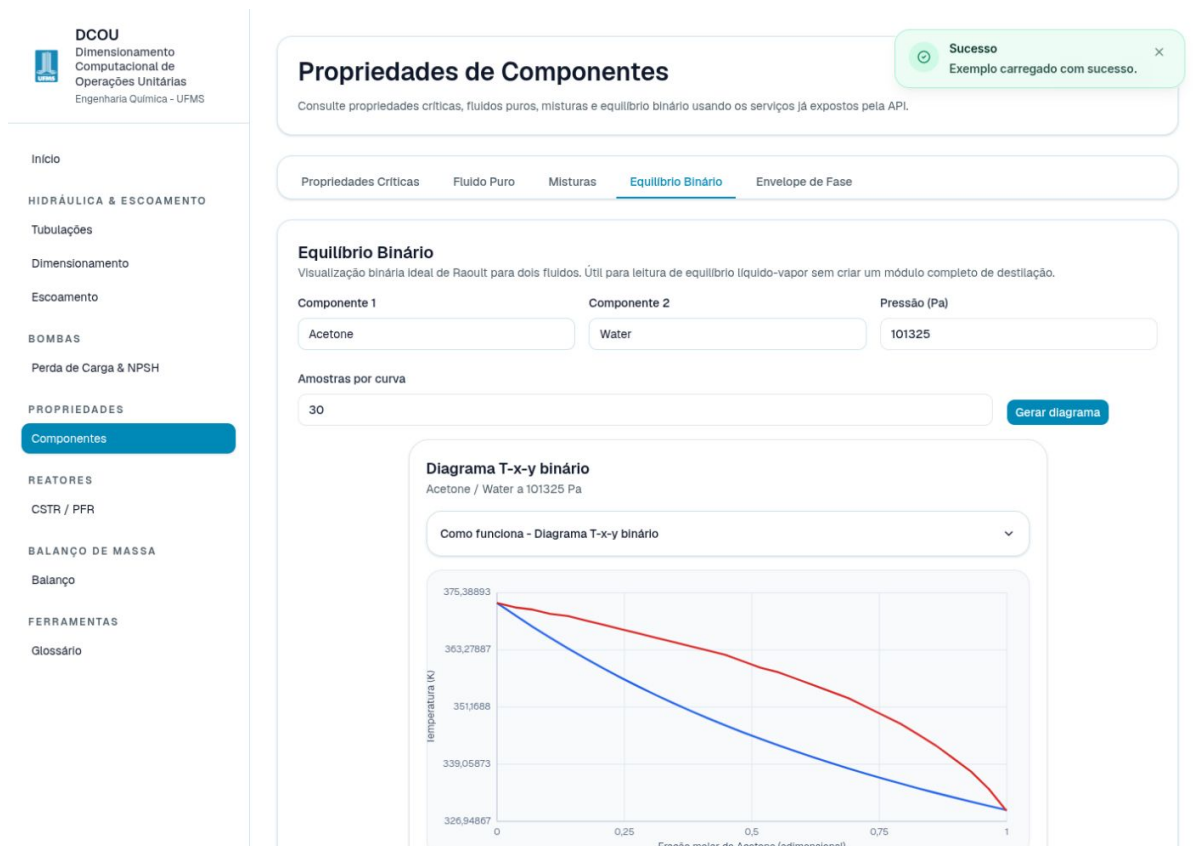
Diagrama Ternário

Orbitação de uma corrente de três componentes. Útil para misturas e separações multicomponentes com adicionar um fluido novo.

Fonte: Elaboração própria (2026).

Na aba **Equilíbrio Binário**, o exemplo gera o diagrama de equilíbrio líquido-vapor. O estudante deve acompanhar as curvas de bolha e orvalho e relacioná-las às composições das fases em equilíbrio.

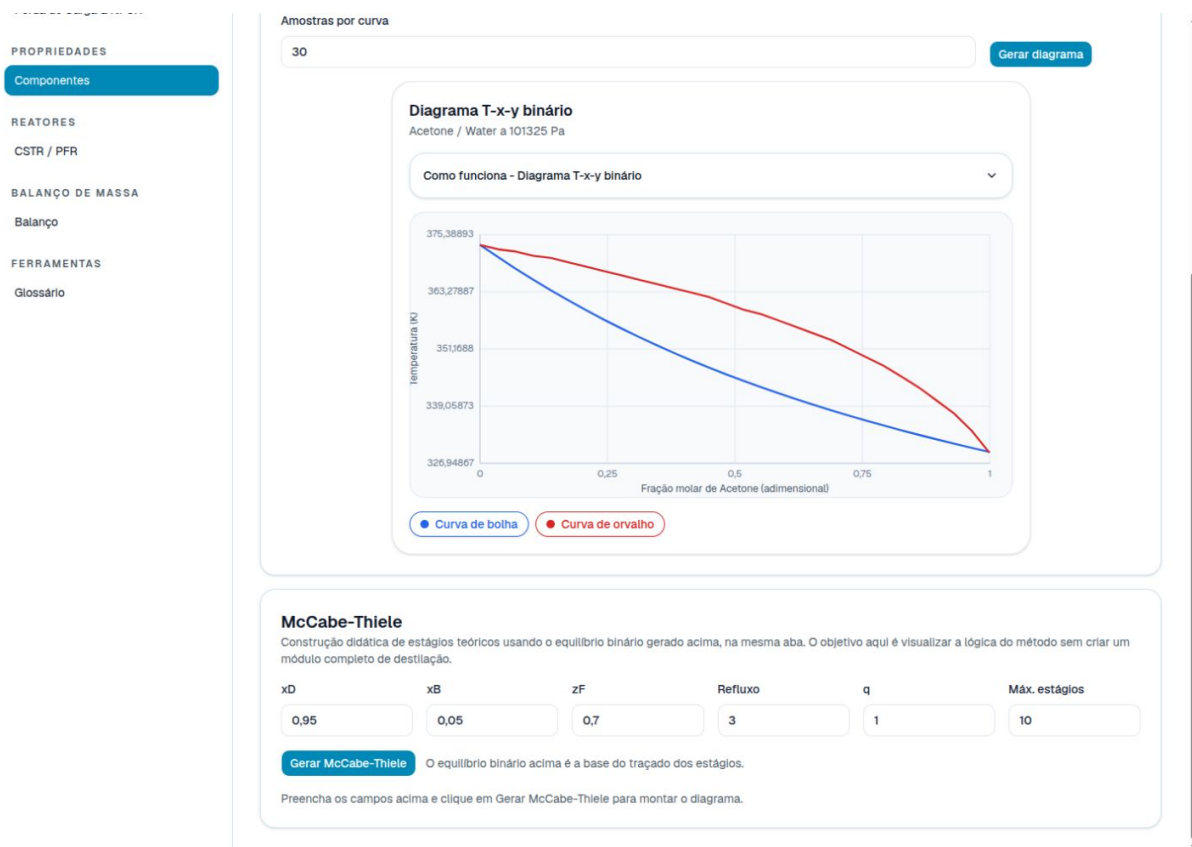
Figura 78 – Aba Equilíbrio Binário com diagrama T-x-y



Fonte: Elaboração própria (2026).

Na mesma área de equilíbrio binário, o bloco **McCabe-Thiele** permite visualizar estágios teóricos de separação. A construção gráfica torna visíveis a curva de equilíbrio, linhas operacionais e degraus de separação.

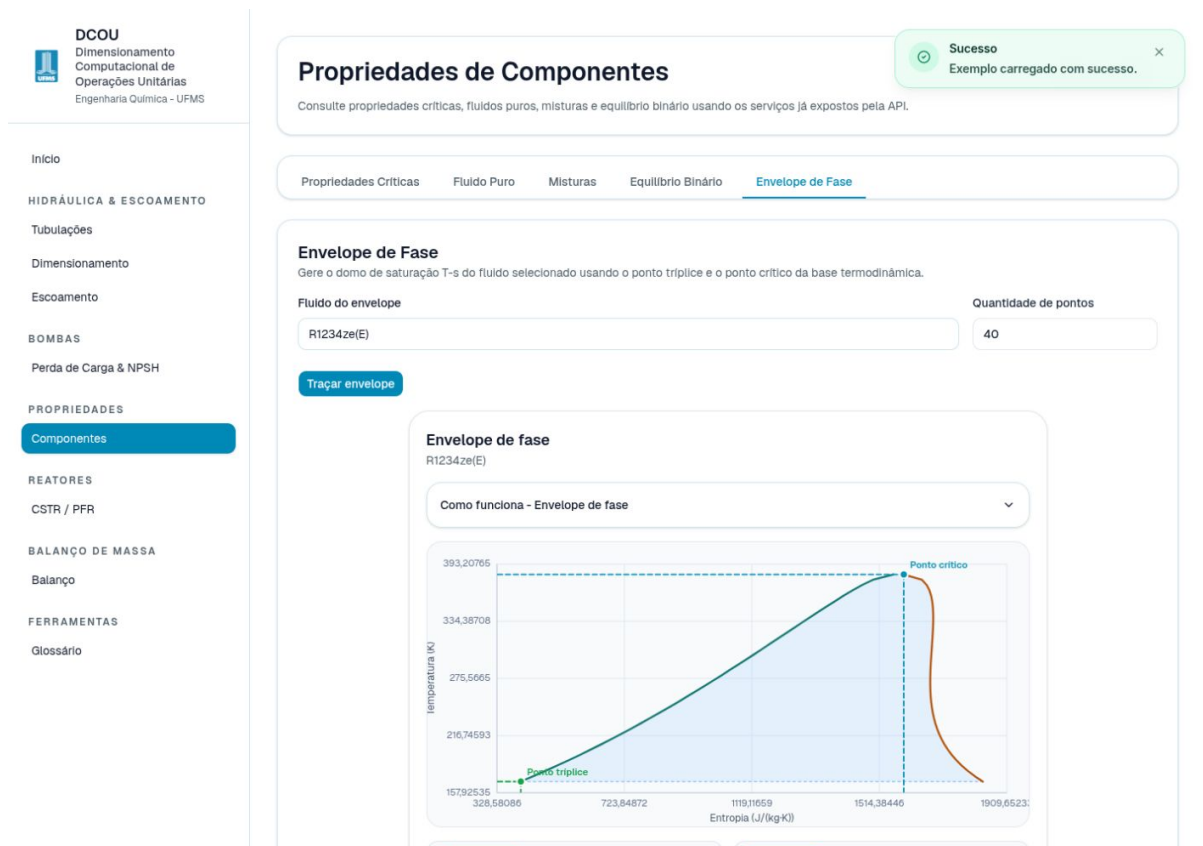
Figura 79 – Bloco McCabe-Thiele gerado a partir do exemplo binário



Fonte: Elaboração própria (2026).

Na aba **Envelope de Fase**, o exemplo apresenta a fronteira entre regiões de fase. Essa tela é adequada para discutir ponto crítico, mudança de fase e limites operacionais de um componente.

Figura 80 – Aba Envelope de Fase com curva gerada pelo exemplo



Fonte: Elaboração própria (2026).

6.7 Reatores Ideais

O módulo **CSTR / PFR** deve ser usado após carregar o exemplo, pois o caso preenche componentes, estequiometria, ordem de reação e condições operacionais. Na aba **CSTR**, a ferramenta calcula conversão, taxa de reação, concentrações de saída, tempo de residência e volume. O discente deve observar que, no CSTR, a composição de saída representa a composição no interior do reator perfeitamente misturado.

Figura 81 – Aba CSTR com exemplo calculado

COMPONENTE 1

Componente: Water Estado: Líquido Vazão de entrada (m^3/s): 1,2

Concentração molar (mol/m^3): 5000 Coef. estequiométrico: -1 Ordem de reação: 1

COMPONENTE 2

Componente: Ethanol Estado: Líquido Vazão de entrada (m^3/s): 0

Concentração molar (mol/m^3): 0 Coef. estequiométrico: 1 Ordem de reação: 0,5

Calcular CSTR

RESULTADO CSTR

Propriedade	Valor	Unidade
Conversão	0,36885	-
Taxa de reação	737,69344	$\text{mol}/\text{m}^3/\text{s}$
Concentrações na saída [Water]	2103,84426	mol/m^3
Concentrações na saída [Ethanol]	1229,48907	mol/m^3
Fator de diluição	0	-
Taxa molar na entrada (limitante)	6000	mol/s
Vazão de saída	1,2	m^3/s
Tempo de residência	2,5	s
Volume	3	m^3

Fonte: Elaboração própria (2026).

Na aba **PFR**, o exemplo calcula o reator tubular e apresenta resultados numéricos e perfis associados. A comparação com o CSTR deve enfatizar que o PFR descreve uma evolução espacial das variáveis, e não uma mistura perfeitamente uniforme.

Figura 82 – Aba PFR com exemplo calculado

5000 -1 1

COMPONENTE 2

Componente: Ethanol Estado: Líquido Vazão de entrada (m³/s): 0

Concentração molar (mol/m³): 0 Coef. estequiométrico: 1 Ordem de reação: 0,5

Calcular PFR

Calculando PFR...

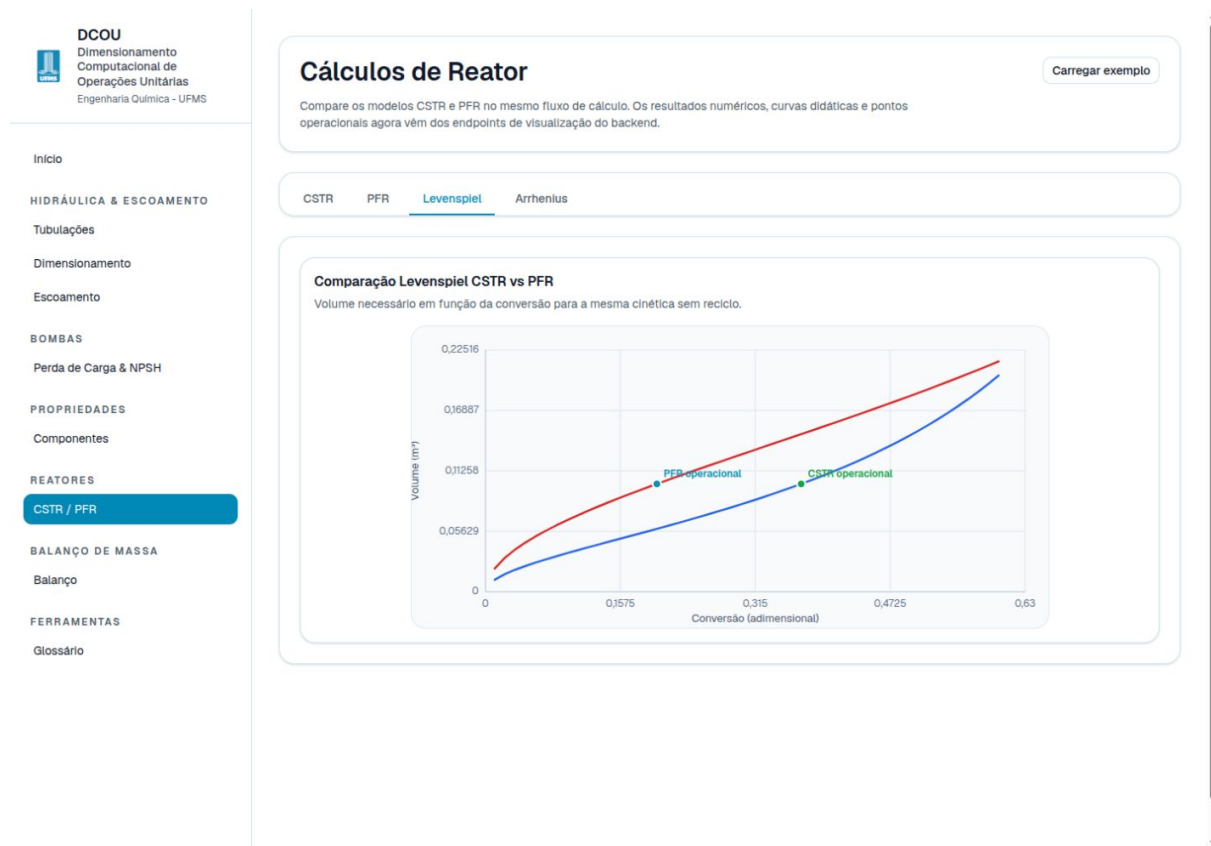
RESULTADO PFR

Propriedade	Valor	Unidade
Conversão	0,19994	-
Taxa molar na entrada (limitante)	6000	mol/s
Vazão de saída	1,2	m³/s
Volume	3	m³
Taxa de reação	688,47703	mol/m³/s
Concentrações na saída [Water]	2666,87321	mol/m³
Concentrações na saída [Ethanol]	666,46012	mol/m³
Fator de diluição (1 + ε·P·O·T)	1,5	-
Tempo de residência	2,5	s
Fator de diluição	0	-

Fonte: Elaboração própria (2026).

Na aba **Levenspiel**, a aplicação compara graficamente CSTR e PFR para o mesmo caso. Essa tela ajuda a discutir por que áreas sob a curva se relacionam ao volume requerido e como a geometria do gráfico expressa a equação de projeto.

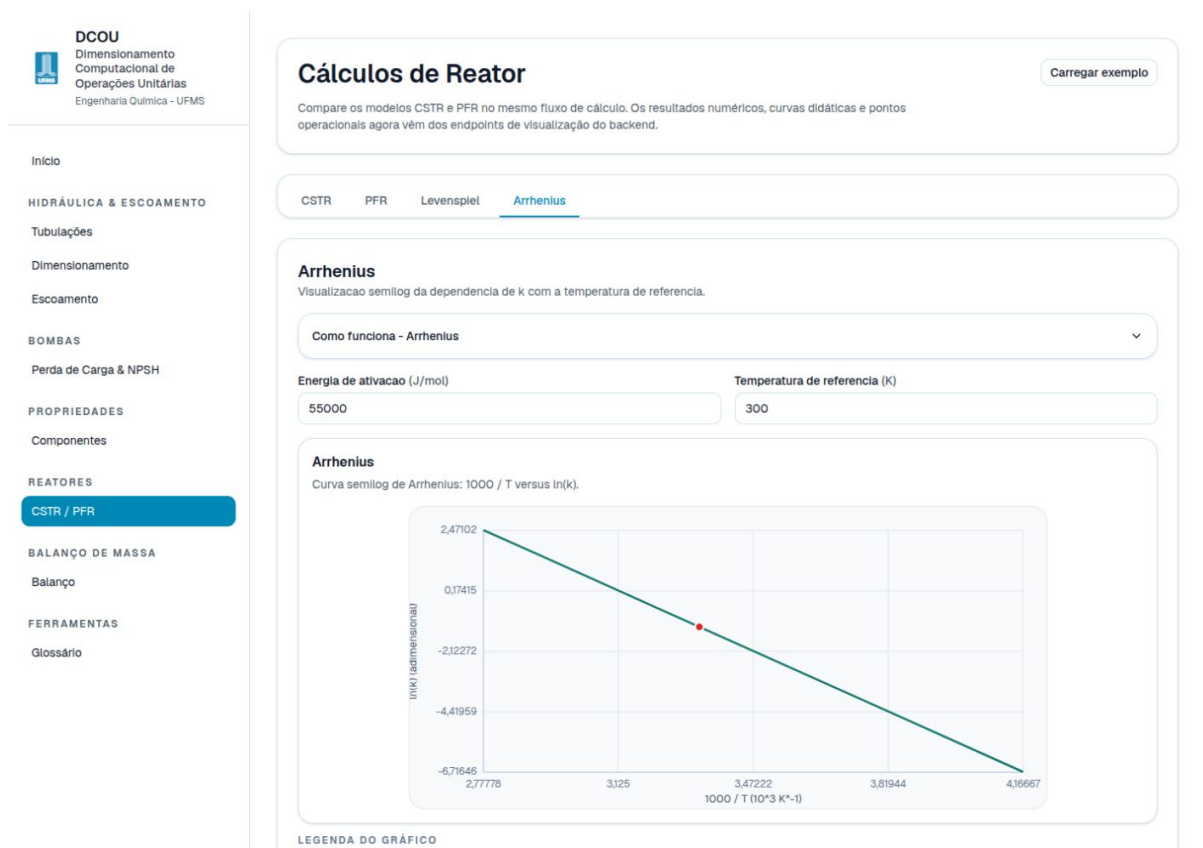
Figura 83 – Aba Levenspiel comparando CSTR e PFR



Fonte: Elaboração própria (2026).

Na aba **Arrhenius**, o exemplo mostra a dependência da constante cinética com a temperatura. O objetivo didático é destacar que variações térmicas podem alterar fortemente a velocidade de reação.

Figura 84 – Aba Arrhenius com curva semilogarítmica do exemplo

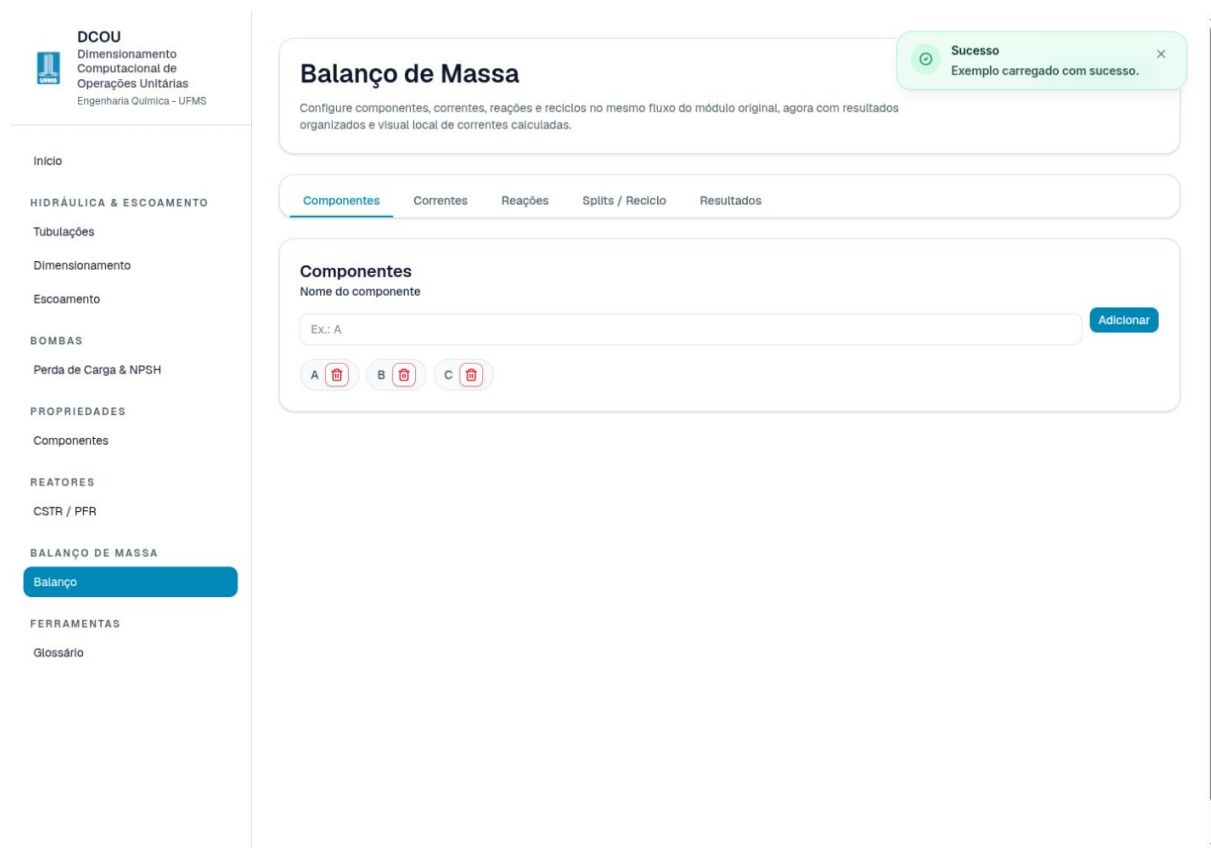


Fonte: Elaboração própria (2026).

6.8 Balanço de Massa

O módulo **Balanço de Massa** é organizado como um fluxo de montagem do problema. A aba **Componentes** define as espécies presentes. Essa etapa deve ser conferida antes de montar correntes e reações, pois todas as demais abas dependem dessa lista.

Figura 85 – Aba Componentes do balanço de massa com exemplo carregado



Fonte: Elaboração própria (2026).

Na aba **Correntes**, o exemplo lista entradas e saídas com vazões e composições. O estudante deve verificar sentido da corrente, vazão total e composição antes de avançar.

Figura 86 – Aba Correntes com alimentação, produto e reciclo do exemplo

DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensionamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORRES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Balanço de Massa

Sucesso
Exemplo carregado com sucesso.

Componentes **Correntes** Reações Splits / Reciclo Resultados

Correntes Adicionar Corrente

CORRENTE 1

Nome: Alimentacao_Fresca Direção: Entrada Vazão: 100

A: 0,8 B: 0,2 C: 0

CORRENTE 2

Nome: Saida_Do_Reator Direção: Saida Vazão:

A: B: C:

CORRENTE 3

Nome: Reciclo Direção: Entrada Vazão:

A: B: C:

Fonte: Elaboração própria (2026).

Na aba **Reações**, a ferramenta organiza estequiometria e conversão. O docente pode usar essa tela para reforçar a diferença entre balanços com e sem reação química.

Figura 87 – Aba Reações com estequiometria do exemplo

Balanço de Massa

Configure componentes, correntes, reações e reciclos no mesmo fluxo do módulo original, agora com resultados organizados e visual local de correntes calculadas.

Componentes Correntes **Reações** Splits / Reciclo Resultados

Reações Adicionar Reação

REAÇÃO 1

Componente-chave	Conversão (0-1)
A	0,7
A	
B	
C	1

Fonte: Elaboração própria (2026).

Na aba **Splits / Reciclo**, o exemplo configura divisão de correntes e reciclo. Essa tela ajuda a discutir fluxos internos do processo, que normalmente dificultam a resolução manual.

Figura 88 – Aba Splits / Reciclo com configuração do exemplo

DCOU
Dimensionamento
Computacional de
Operações Unitárias
Engenharia Química - UFMS

Início

HIDRÁULICA & ESCOAMENTO

Tubulações

Dimensionamento

Escoamento

BOMBAS

Perda de Carga & NPSH

PROPRIEDADES

Componentes

REATORES

CSTR / PFR

BALANÇO DE MASSA

Balanço

FERRAMENTAS

Glossário

Balanço de Massa

Sucesso
Exemplo carregado com sucesso.

Configure componentes, correntes, reações e reciclos no mesmo fluxo do módulo original, agora com resultados organizados e visual local de correntes calculadas.

Componentes Correntes Reações **Splits / Reciclo** Resultados

Splits / Reciclo Adicionar Split

SPLIT 1

Corrente pal
Saída_Do_Reator

Corrente de reciclo
Reciclo

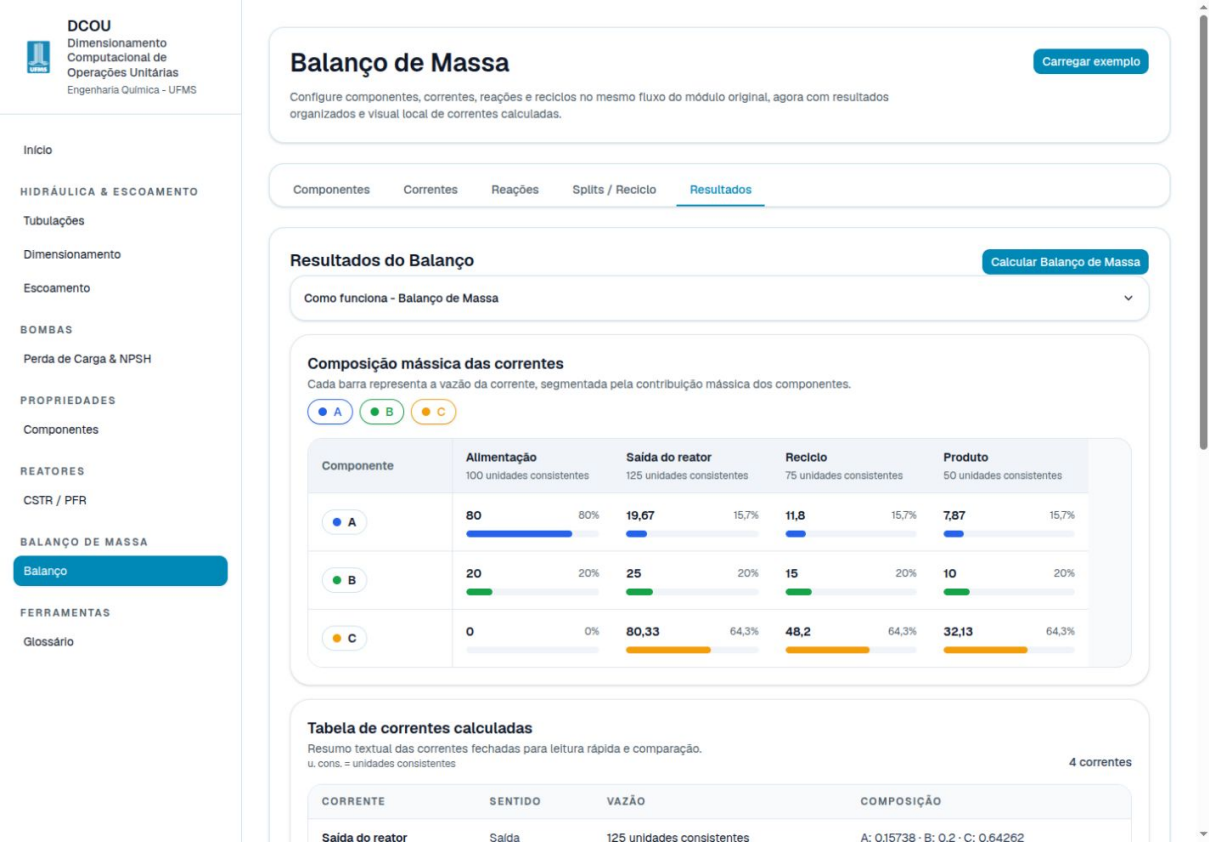
Corrente de purga
Produto

Fração de reciclo (0-1)
0,6

Fonte: Elaboração própria (2026).

Na aba **Resultados**, após executar **Calcular Balanço de Massa**, a ferramenta apresenta composição mássica das correntes, tabela de correntes calculadas e comparações detalhadas. A interpretação recomendada é verificar se as vazões fecham e se as frações dos componentes fazem sentido físico.

Figura 89 – Aba Resultados do balanço de massa após cálculo do exemplo

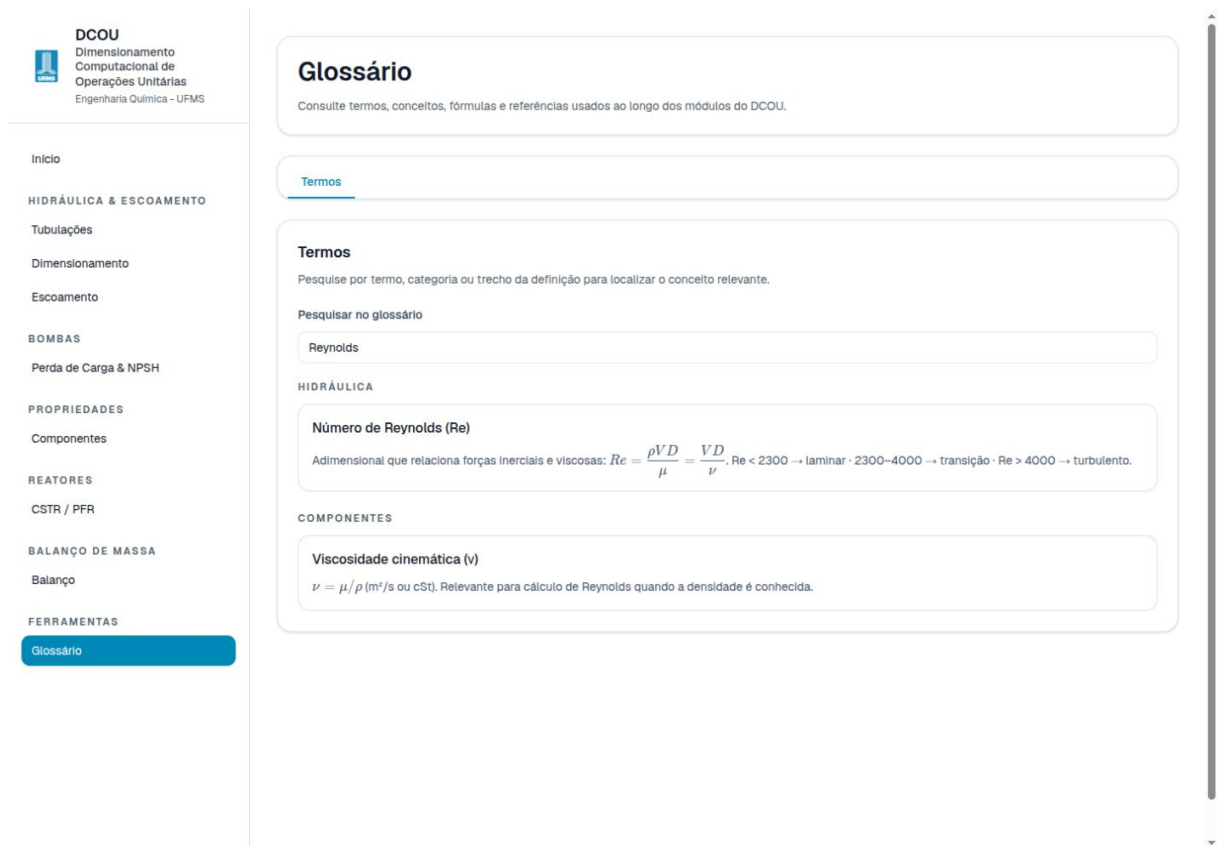


Fonte: Elaboração própria (2026).

6.9 Glossário e Validações

O **Glossário** apoia a leitura dos resultados durante o uso dos módulos. No exemplo de busca por *Reynolds*, a ferramenta filtra o termo principal e conceitos relacionados, como viscosidade cinemática. O estudante deve usar essa tela quando precisar revisar rapidamente definições e equações.

Figura 90 – Glossário com busca por Reynolds



Fonte: Elaboração própria (2026).

Além da consulta conceitual, a ferramenta possui validações de entrada. Campos obrigatórios, valores numéricos incompatíveis e combinações incompletas de parâmetros geram notificações, evitando que o usuário interprete ausência de dado como resultado físico. Em atividades didáticas, recomenda-se carregar o exemplo, executar o cálculo, registrar o resultado e então alterar apenas uma variável por vez. Essa prática permite comparar cenários mantendo rastreável a relação entre hipótese, entrada, equação e visualização.

Como sequência mínima de uso em aula ou estudo dirigido, recomenda-se:

1. abrir a tela inicial e escolher a trilha ou o módulo de interesse;
2. carregar o exemplo disponível no módulo;
3. conferir unidades, hipóteses e parâmetros preenchidos;
4. executar o cálculo ou geração gráfica da aba atual;
5. interpretar a tabela de resultados junto das visualizações;
6. navegar para as abas relacionadas, como CSTR/PFR, Levenspiel, Arrhenius, NPSH ou resultados de balanço;
7. alterar uma variável por vez para testar cenários;

8. consultar o glossário e os painéis **Como funciona** sempre que a interpretação física precisar ser reforçada.

Assim, o DCOU pode ser usado como ambiente de exploração guiada. O valor didático está em manter, no mesmo fluxo, entradas, catálogos, cálculos, visualizações, validações e explicações conceituais.

7 Conclusões

O presente trabalho alcançou com êxito o objetivo geral de desenvolver o DCOU, um *software web open-source* que reúne, em uma mesma plataforma, o dimensionamento de operações unitárias em engenharia química e uma camada de recursos didáticos. Do lado do cálculo, foram implementados módulos de tubulações, escoamento, bombas, propriedades termofísicas, reatores *CSTR* e *PFR* e balanços de massa. Foi demonstrada precisão e utilidade prática, validadas com referências consolidadas da literatura, sobre uma arquitetura cliente-servidor moderna e acessível via *API*. Do lado didático, o glossário técnico, as explicações teóricas embutidas, os cenários de exemplo e a biblioteca de visualizações interativas consolidaram a ferramenta como apoio efetivo ao aprendizado conceitual.

A ferramenta desenvolvida preenche uma lacuna no acesso a simulações didáticas e gratuitas, especialmente em contextos acadêmicos e para profissionais autônomos, ao reunir a automação de cálculos rotineiros com recursos que promovem ativamente o aprendizado conceitual. O caráter *open-source* é fundamental para sua completude futura, permitindo contribuições comunitárias que expandam o repertório de módulos e aprimorem sua robustez, tornando-o uma plataforma escalável para o ensino e a prática da engenharia química na era da Indústria 4.0.

Como perspectivas, sugere-se a adição de funcionalidades como simulações multifásicas, otimização de processos, integração com bancos de dados remotos e compatibilidade com o padrão de interface CAPE-OPEN, consolidando o *software* como uma alternativa viável a ferramentas proprietárias.

Referências

- ALVES, G. N. et al. Uso do software aberto dwsim na simulação de biorrefinarias: desafios e aprendizados. In: *Anais do XXX Congresso de Iniciação Científica da UNICAMP*. Campinas: UNICAMP, 2022.
- ASPEN TECHNOLOGY. *Aspen HYSYS*. Bedford, MA, 2026. Acesso em: 8 jul. 2026. Disponível em: <<https://www.aspentech.com/en/products/engineering/aspen-hysys>>.
- ASPEN TECHNOLOGY. *Aspen Plus*. Bedford, MA, 2026. Acesso em: 8 jul. 2026. Disponível em: <<https://www.aspentech.com/en/products/engineering/aspen-plus>>.
- BELL, I. H. et al. Pure and pseudo-pure fluid thermophysical property evaluation and the open-source thermophysical property library coolprop. *Industrial & Engineering Chemistry Research*, ACS Publications, v. 53, n. 6, p. 2498–2508, 2014.
- COLEBROOK, C. F. Turbulent flow in pipes, with particular reference to the transition region between the smooth and rough pipe laws. *Journal of the Institution of Civil Engineers*, Thomas Telford-ICE Virtual Library, v. 11, n. 4, p. 133–156, 1939.
- DATA COR. *CHEMCAD Process Simulation Software*. Florham Park, NJ, 2026. Acesso em: 8 jul. 2026. Disponível em: <<https://www.datacor.com/products/chemcad>>.
- DEVOE, H. *Thermodynamics and Chemistry*. 2. ed. [s.n.], 2014. Version 5. Acesso em: 8 jul. 2026. Disponível em: <<https://www2.chem.umd.edu/thermobook/>>.
- DWSIM. *DWSIM – Open-Source Chemical Process Simulator*. 2026. Acesso em: 8 jul. 2026. Disponível em: <<https://dwsim.org/>>.
- FELDER, R. M.; ROUSSEAU, R. W. *Princípios Elementares dos Processos Químicos*. 3. ed. Rio de Janeiro: LTC, 2005.
- FOGLER, H. S. *Elementos de Engenharia das Reações Químicas*. 4. ed. Rio de Janeiro: LTC, 2009.
- FOX, R. W.; PRITCHARD, P. J.; MCDONALD, A. T. *Fox and McDonald's Introduction to Fluid Mechanics*. 8. ed. Hoboken: Wiley, 2011.
- HAALAND, S. E. Simple and explicit formulas for the friction factor in turbulent pipe flow. *Journal of Fluids Engineering*, American Society of Mechanical Engineers, v. 105, n. 1, p. 89–90, 1983.
- KARASSIK, I. J. et al. (Ed.). *Pump Handbook*. 4. ed. New York: McGraw-Hill, 2008.
- LEVENSPIEL, O. *Engenharia das Reações Químicas*. 3. ed. São Paulo: Edgard Blücher, 2000.
- MCCABE, W. L.; SMITH, J. C.; HARRIOTT, P. *Unit Operations of Chemical Engineering*. 7. ed. New York: McGraw-Hill, 2005.
- PERRY, R. H.; GREEN, D. W. (Ed.). *Perry's Chemical Engineers' Handbook*. 8. ed. New York: McGraw-Hill, 2007.

- POLING, B. E.; PRAUSNITZ, J. M.; O'CONNELL, J. P. *The Properties of Gases and Liquids*. 5. ed. New York: McGraw-Hill, 2001.
- SEADER, J. D.; HENLEY, E. J.; ROPER, D. K. *Separation Process Principles: With Applications Using Process Simulators*. 4. ed. Hoboken: Wiley, 2016.
- SMITH, J. M. et al. *Introduction to Chemical Engineering Thermodynamics*. 8. ed. New York: McGraw-Hill Education, 2018.
- SWAMEE, P. K.; JAIN, A. K. Explicit equations for pipe-flow problems. *Journal of the Hydraulics Division*, ASCE, v. 102, n. 5, p. 657–664, 1976.
- WHITE, F. M. *Mecânica dos Fluidos*. 8. ed. Porto Alegre: AMGH, 2018.